# MATH7432 - Computing and Algorithms for Statistics

Gaetano Romano

Invalid Date

# Table of contents


**Preface** **5**
- Overview . . . 5
- How to use these notes . . . 6
- Prerequisites . . . 6
- Sources . . . 7

**1 Week 1: Introduction to R and Python Programming** **8**
- 1.1 R, Python and RStudio . . . 9
  - 1.1.1 Getting help . . . 10
- 1.2 A glorified calculator . . . 11
- 1.3 Variables and assignment . . . 13
  - 1.3.1 Types, briefly . . . 16
- 1.4 Vectors . . . 18
  - 1.4.1 Sequences and repetitions . . . 19
  - 1.4.2 Vectorised arithmetic . . . 21
  - 1.4.3 Recycling, and its footgun . . . 24
  - 1.4.4 Indexing: getting elements out . . . 26
  - 1.4.5 A first plot . . . 30
- 1.5 Saving your work: csv files and the working directory . . . 32
- 1.6 Matrices and a taste of linear algebra . . . 34
  - 1.6.1 Indexing a matrix . . . 38
  - 1.6.2 Matrix arithmetic: `*` is not what you think . . . 39
  - 1.6.3 Inverses, determinants and linear systems . . . 41
  - 1.6.4 An eigenvalue teaser . . . 44
- 1.7 Challenges . . . 46
- 1.8 What you can do now . . . 47

**2 Week 2: Logic, Loops and Iteration** **49**
- 2.1 Comparisons . . . 49
  - 2.1.1 The floating-point trap . . . 52
- 2.2 Logical operators . . . 54
  - 2.2.1 `&&` / `and` versus `&` . . . 57
  - 2.2.2 `any()`, `all()` and `%in%` / `in` . . . 57
- 2.3 Logical indexing . . . 59

2.4 Conditional statements . . . 63
2.4.1 Vectorised choice: `ifelse()` / `np.where()` . . . 65
2.5 `for` loops . . . 66
2.5.1 Loops with state . . . 68
2.5.2 Grow or preallocate? . . . 70
2.6 `while`, `break` and `next` . . . 72
2.7 When not to loop . . . 74
2.8 A first taste of gradient descent . . . 77
2.8.1 The idea: follow the slope downhill . . . 77
2.8.2 The algorithm in code . . . 79
2.8.3 The step size is a dial, not a detail . . . 81
2.8.4 Beyond the quadratic . . . 83
2.9 Challenges . . . 86

**3 Week 3: Functions and Functional Programming 87**
3.1 Writing your own functions . . . 87
3.1.1 Arguments and defaults . . . 89
3.1.2 Return values . . . 92
3.2 Scoping: where variables live . . . 95
3.3 The `...` / `**kwargs` argument . . . 99
3.4 A brief look at recursion . . . 100
3.5 Functions are values . . . 102
3.5.1 Lambdas . . . 104
3.5.2 Function factories and currying . . . 105
3.5.3 map: one function, every element . . . 110
3.5.4 `Reduce` / `functools.reduce`: fold a collection to one value . . . 115
3.6 Packaging gradient descent . . . 117
3.7 Challenges . . . 121
3.8 Looking ahead . . . 122

**4 Week 4: Data Frames and Plotting 123**
4.1 The tidyverse (and friends in Python) . . . 123
4.2 Data frames . . . 124
4.2.1 Building one . . . 124
4.2.2 Columns, rows and cells . . . 125
4.2.3 Getting to know a data frame . . . 129
4.3 Tibbles (and the pandas index) . . . 133
4.4 Reading and writing data . . . 136
4.5 The pipe `|>` (and method chaining) . . . 138
4.6 Data manipulation with dplyr (and pandas) . . . 139
4.6.1 filter and select . . . 140
4.6.2 mutate and arrange . . . 142
4.6.3 group_by and summarise . . . 144

4.6.4 Joining data frames . . . 147
4.7 Plotting with ggplot2 (and plotnine) . . . 148
4.7.1 Scatter plots . . . 148
4.7.2 Mapping vs setting aesthetics . . . 152
4.7.3 Histograms and boxplots . . . 155
4.7.4 Line plots . . . 159
4.7.5 Facets . . . 161
4.7.6 Labels, themes and saving . . . 163
4.8 Challenges . . . 165

**5 Week 5: Introduction to Optimisation for Statistics 168**
5.1 Gradient descent, again . . . 168
5.1.1 Estimation is optimisation . . . 172
5.1.2 Stochastic gradient descent . . . 181
5.2 Fixing the geometry: preconditioning and BFGS . . . 190
5.2.1 Why the step size is hard . . . 190
5.2.2 Preconditioning . . . 195
5.2.3 BFGS: learning the Hessian as you go . . . 200
5.3 Where you will see all this again . . . 207

**References 209**

# Preface

Welcome to MATH7432, *Computing and Algorithms for Statistics*: a Master's module at Lancaster University about the computational machinery that makes modern statistics run. Statistics on paper is elegant; statistics in practice is code. This module teaches you to write that code well, and to understand the algorithms it implements. Everything is in both R and Python.

Every code chunk in these notes appears twice, side by side: **R on the left, Python on the right**, translated line for line. You are expected to **learn both languages**. R is the lingua franca of statistics, Python of machine learning and production data science, and the modern statistician needs both in their toolset! Working statisticians cross between the two languages constantly, and the side-by-side layout teaches you to make that hop. One practical consequence: the side-by-side layout needs horizontal room, so read the notes on a wide screen (to be fair, it can work on a phone on a single column layout, but it kinda misses the point, and you probably want your phone for doomscrolling rather then reading the notes).

## Overview

The module will follow:

- **Weeks 1–4: Programming in R and Python** (each week: 1h lecture + 2h lab). We start from zero and build up both languages: variables, vectors and matrices (Chapter 1); logic, loops and iteration (Chapter 2); functions and functional programming (Chapter 3); data frames and plotting (Chapter 4). A running example threads through these weeks: **gradient descent**, the workhorse optimiser behind half of modern statistics and essentially all of machine learning. It appears in week 2 as a humble loop and grows up alongside your programming skills.
- **Week 5: Optimisation for statistics** (2h lecture + 1h lab). The payoff chapter, where the programming tools meet the statistics: estimation as loss minimisation, stochastic gradient descent, preconditioning, and quasi-Newton methods (BFGS); see Chapter 5.

**These notes cover weeks 1–5** The materials for weeks 6-10 will be released separately later in term, and will roughly cover the following topics:

- **Weeks 6–7: The bootstrap.** Nonparametric, parametric and semi-parametric bootstraps; standard errors, confidence intervals, and bias correction. Week 6 also hosts the **in-class test**.
- **Weeks 8–10: Markov chain Monte Carlo.** Random-walk Metropolis, Metropolis–Hastings, Hamiltonian Monte Carlo, convergence diagnostics, and Gibbs sampling.

## How to use these notes

The intended workflow:

- **Before the lecture:** skim the week's chapter. You don't need to understand everything: the lecture will give you the overview and the why.
- **During the lecture:** follow along, ask questions. You can take notes, but I will cover the same materials I have in these documents, so you don't need to copy everything down.
- **In the lab:** work through the chapter at your own pace, copy-pasting the code into RStudio and running it yourself. Change things. Break things. Exercises appear throughout each chapter exactly where the relevant idea is fresh: do them as you meet them! If you get stuck, ask one of us for help.
- **Which language for the exercises?** Since you are building both, use the exercises to train the weaker one. If you already know one of the two languages, do the exercises in the other; if you know neither, alternate languages across the exercises. In every case, check the solutions for both languages.
- **Afterwards:** worked solutions are released on Moodle once all of the week's labs have ended.

Code you are meant to run appears in grey boxes; its output appears just below.

## Prerequisites

You might find that the rate of learning is faster than you expect. Well, it is. Since this is a Master's module, to proceed at the required speed we kind-of assume that you have:

- **A good grasp of year 1 mathematics**. We will use calculus and linear algebra concepts (gradients, matrices, eigenvalues). Most of you from Lancaster will have covered these, but if you haven't you need to catch up. If you don't understand part of the maths, please ask as soon as possible.
- **Knowledge of basic statistics and probability**. We will use concepts such as random variables, distributions, and likelihoods. Again, as above, if you are not familiar with these, please get in touch.

- **Programming**. Most importantly, we assume that you have **some** programming experience. Many of you will have done some programming at some point, but if you haven't, don't worry: the first four weeks start from zero, however they progress quickly. If you have programmed before, the early weeks will be quick revision (although, if you have never never programmed before, please get in touch, so we can make sure you are not left behind).

## Sources

These notes build on, and reuse material from, two earlier Lancaster modules:

- Gaetano Romano and Eduard Campillo-Funollet, *MATH245 Computational Mathematics*, Lancaster University (also the source of the side-by-side R/Python format).
- *MATH550/SCC.461 R Programming* (Statistics in Practice / Programming for Data Scientists), Lancaster University, https://www.lancaster.ac.uk/~romano/teaching/2324MATH550/.

This sets of notes essentially descends from the lineage of Debbie Costain, Clement Lee, Chris Nemeth, Tom Palmer, Florian Pein, Eduard Campillo-Funollet, Gaetano Romano and Stuart Sharples. I hope I haven't missed anyone (and if I have, please let me know).

For going deeper into R than we do here, the canonical further reading is Hadley Wickham, *Advanced R*, 2nd edition, https://adv-r.hadley.nz/: free online, and the source of several ideas and examples in these notes.

Right then. Open RStudio (which, as week 1 explains, speaks both of our languages) and let's begin with Chapter 1.

# 1 Week 1: Introduction to R and Python Programming

Welcome to MATH7432! Over the next five weeks we will learn how to program, in both R and Python, and how to use them to build the computational tools that modern statistics runs on. By the end of the module you will have implemented, from scratch, algorithms that estimate statistical models: the same family of algorithms that trains neural networks. This week we start from zero: no programming background is assumed.

Before the code, a very brief explainer. A **computer** is a device with a processor (which does arithmetic, very fast these days), memory (which holds the numbers the processor is currently working on, very expensive these days) and storage (which holds files that survive a reboot). A **program** is a list of instructions for the processor. Processors only understand *machine code* (streams of 0s and 1s), which humans are famously bad at writing, so we write programs in a **programming language**: a formal language that is readable by humans and translated for the machine. R and Python, the two languages of this module, are both *high-level*, *interpreted* languages: high-level means they read almost like mathematics, and interpreted means each line is translated and executed on the spot, so you can have a live conversation with your computer: type a command, get an answer, type the next one. This makes them ideal for data analysis, where you rarely know step 10 before seeing the result of step 3.

A bit of history:

- **R** was created in 1993 by Ross Ihaka and Robert Gentleman in Auckland as an open-source descendant of the S language, and it has since become the lingua franca of statistics: free, open source, with thousands of contributed packages covering essentially every statistical method ever published (available on `CRAN`).
- **Python** was created in 1991 by Guido van Rossum in the Netherlands, and is named after the comedy group Monty Python, not the snake. It is a free, open-source, general-purpose language with a philosophy of readability. Through the libraries `numpy` and `pandas`, which we will lean on heavily, it has become the other lingua franca of scientific computing, and the language most machine learning is written in.

At this day, in my opinion, there's not a clear cut on which language is better for statistics: depending on what you will do, you might need one or the other, or both. For instance, R is very popular in medical statistics and epidemiology, while Python is the language of choice for machine learning and deep learning.

So (fortunately/unfortunately) the modern statistician needs both in their toolset! In this module you will learn *both of them*, side by side. Every example in these notes appears in both languages, translated line for line, and the translation is itself part of the course: hopping between the two until the crossing feels routine is exactly the skill we are after.

Throughout these notes you will meet chunks of code with their output printed underneath: **side by side in the two languages, R always on the left, Python always on the right** (on a narrow screen the columns stack, and the bold label above each chunk tells you which is which). Here is the layout in action:

**R**

```
# I am a comment: R ignores everything after a '#'. The line below does the work.
print("Beep bop beep")
```

```
[1] "Beep bop beep"
```

**Python**

```
# I am a comment: Python ignores everything after a '#'. The line below does the work.
print("Beep bop beep")
```

```
Beep bop beep
```

The notes are designed for the labs: read a section, copy the code into RStudio, run it, break it, fix it, and do the exercises as you meet them.

## 1.1 R, Python and RStudio

You will interact with R through **RStudio**, a program that wraps R in a friendly interface. Both are installed on the university machines via AppsAnywhere; for your own laptop, install R from r-project.org first and RStudio from posit.co second. When you open RStudio you will see several panels:

- the **console** (look for the `>` prompt): type a command, press `Enter`, R answers immediately, and it's what executes your commands;
- the **script editor**: a text file of commands you are preparing. If you don't see this, click on `file > new` and select your language of choice. Once you have some code, press `Ctrl + Enter` (`Cmd + Enter` on a Mac) on a line to send it to the console;
- the **environment** panel: lists the objects you have created so far;
- a miscellaneous panel showing files, plots and help pages.

**Note for Python:** everything in this section applies on the Python side too. RStudio speaks Python as well, and it is what we use in the labs for both languages: a Python script ends in `.py` instead of `.R`, the Python console prompt is `>>>` rather than `>`, and on your own laptop you will want Python from python.org (the university machines can install this via R Studio). Comments are `#` in Python too, so everything below about commenting transfers unchanged.

The console forgets, a saved script does not. A good workflow, and the one to use in the labs, is: type your commands in a script, run them with `Ctrl + Enter`, and save the script as you go. Future you, who returns after a week, will be grateful.

Future you will be even more grateful for **comments**. Any text after a `#` on a line is ignored, in both languages, which makes it the place to explain *why* the code does what it does:

**R**

```
# convert a temperature from Celsius to Fahrenheit
20 * 9 / 5 + 32
```

```
[1] 68
```

**Python**

```
# convert a temperature from Celsius to Fahrenheit
20 * 9 / 5 + 32
```

```
68.0
```

We comment code generously in these notes, and you should do the same in yours. One of the hardest tasks in programming is to understand somebody else's code, and this include yours 5 years from now. Comments are your best friend. Note how LLMs generated code is full of comments! I personally find this helps a lot.

### 1.1.1 Getting help

Every R function has a help page: `?seq` (or `help(seq)`) opens the documentation for `seq`. The *Description*, *Usage* and *Arguments* sections are the ones to read first, and the *Examples* at the bottom can be copy-pasted.

Python's equivalent is `help()` (e.g. `help(len)`, or `help(np.arange)` once numpy is loaded), and the online documentation for numpy and pandas is excellent. Not all of Python ocumentation is great however, for anything the help pages cannot answer, the internet is your best friend. Generally LLMs are great at answering questions about usage as well, in particular if you point them to the current documentation.

## 1.2 A glorified calculator

At its heart, the processor is a very fast calculator, and the console gives you direct access to it. All the basic operations are here:

Table 1.1: Basic mathematical operations in the two languages. `pi` is built into R; Python keeps it in the `math` library.

| Operation | R | Python |
|---|---|---|
| Add / subtract | `2 + 3`, `2 - 3` | `2 + 3`, `2 - 3` |
| Multiply / divide | `2 * 3`, `2 / 3` | `2 * 3`, `2 / 3` |
| Power | `2^10` | `2**10` |
| Integer division | `7 %/% 2` | `7 // 2` |
| Modulus (remainder) | `7 %% 2` | `7 % 2` |
| Square root | `sqrt(2)` | `math.sqrt(2)` |
| Natural log | `log(2)` | `math.log(2)` |
| Exponential | `exp(1)` | `math.exp(1)` |
| Trigonometry | `sin(pi / 2)`, `cos(2 * pi)` | `math.sin(math.pi / 2)`, `math.cos(2 * math.pi)` |

**Note for Python:** the `math.` prefix in the right-hand column means those functions live in the **math** *library*, which must be loaded (once, at the top of your script) with `import math`. That includes the constant `math.pi`, since bare `pi` does not exist in Python. Loading libraries is everyday Python life; numpy and pandas arrive later this chapter in exactly the same way. And one trap deserves its own sentence: **the power operator in Python is `**`**, because `^` exists there too but means something entirely different (bitwise XOR): `2^10` runs happily in Python and returns `8`.

**R**

```
3 + 5 * 2
```

```
[1] 13
```

**Python**

```
3 + 5 * 2
```

```
13
```

In the R output, the `[1]` tells you that `13` is the first element of the answer: for now answers have only one element, but that changes very soon (Python just prints the bare number). Operations follow the usual algebraic precedence (powers, then multiplication and division, then addition and subtraction, left to right), and parentheses override it:

**R**

```
3 + 7 * 5 / 3 - 2
```

```
[1] 12.66667
```

```
(3 + 7) * 5 / (3 - 2)
```

```
[1] 50
```

**Python**

```
3 + 7 * 5 / 3 - 2
```

```
12.666666666666666
```

```
(3 + 7) * 5 / (3 - 2)
```

```
50.0
```

So any formula you can write on paper, you can type. Take, for instance:

$$\frac{\sqrt{3/4}}{\frac{1}{3} - \frac{2}{\pi^2}}.$$

Here's in code:

**R**

```
sqrt(3 / 4) / (1 / 3 - 2 / pi^2)
```

```
[1] 6.626513
```

**Python**

```python
import math   # sqrt, log, pi & co. live in the math library

math.sqrt(3 / 4) / (1 / 3 - 2 / math.pi**2)
```

```
6.626513139875386
```

> Exercise 1.1: A glorified calculator
>
> Use the console to evaluate:
>
> a. $\dfrac{49^2 - 542}{42 + 63^2}$
> b. $\sqrt{\pi^3} - \cos(2\pi)$
> c. whether 2026 is divisible by 7, and whether it is divisible by 1013 (the `%%` operator in R, and `%` in Python, is your friend: what should the remainder be?);
> d. each person's share when a £147.68 dinner bill is split among 7 people.

## 1.3 Variables and assignment

A calculator with no memory forces you to re-type intermediate results. Our languages instead let you store any result in a **variable** (a labelled box in memory) using the assignment operator: `<-` in R (read it as "gets"), the plain `=` in Python:

**R**

```r
x <- 20 - 16
y <- 4 + 5
x - y
```

```
[1] -5
```

**Python**

```python
x = 20 - 16
y = 4 + 5
x - y
```

```
-5
```

**Note on `<-` vs `=`:** R also accepts `=` for assignment, and fingers fresh from the Python column will type it on reflex, but the R community's convention is `<-`, and we follow it in these notes. Python offers no choice: assignment is `=`, and `<-` is not assignment at all there. In Python `x <- 3` parses as "`x` less than `-3`", a comparison that quietly returns `True` or `False`. Beware when switching columns.

Typing a variable's name prints its content, and the environment panel of RStudio lists everything you have created:

**R**

```
x
```

```
[1] 4
```

```
y
```

```
[1] 9
```

**Python**

```
x
```

```
4
```

```
y
```

```
9
```

Three things to know about assignment. First, both languages are case-sensitive, so `x` and `X` are different names, and asking for a variable that does not exist is an error:

**R**

```
X
```

```
Error:
! object 'X' not found
```

**Python**

```
X
```

```
NameError: name 'X' is not defined
```

Second, assigning to an existing variable silently overwrites it; neither language asks "are you sure?":

**R**

```
x <- -100
x
```

```
[1] -100
```

**Python**

```
x = -100
x
```

```
-100
```

Third, a variable can appear on both sides of the assignment. The statement below is nonsense as an equation but perfectly sensible as an instruction: *take the current value of* $x$*, add 1, store the result back in* $x$:

**R**

```
x <- x + 1
x
```

```
[1] -99
```

**Python**

```
x = x + 1
x
```

```
-99
```

This little idiom is the beating heart of every iterative algorithm we will write in this module, so stare at it until it feels normal.

Variables also make code readable. Compare the formula from earlier with:

**R**

```r
numerator   <- sqrt(3 / 4)
denominator <- 1 / 3 - 2 / pi^2
numerator / denominator
```

```
[1] 6.626513
```

**Python**

```python
numerator   = math.sqrt(3 / 4)
denominator = 1 / 3 - 2 / math.pi**2
numerator / denominator
```

```
6.626513139875386
```

Same result, but each piece now has a name, can be inspected separately, and can be reused. Name your variables like this: lowercase, words separated by `_`, meaning over brevity (a convention both language communities happen to agree on).

> Exercise 1.2: Compound interest, minimal effort
>
> You deposit £2000 in an account paying 3.5% interest per year. After $t$ years the balance is $2000 \times 1.035^t$.
>
> a. Create variables `deposit`, `rate` and `years` (set it at 10 years), and compute the final balance *using only the variables*, storing it in `balance`.
> b. How much interest did you earn in total?
> c. A rival bank offers 5%. Change `rate`, re-run your lines, and report the new balance. Notice how little typing part (c) took: that is why we use variables.

### 1.3.1 Types, briefly

Not everything in a variable is a number. Both languages distinguish several **types** of value, and R's `class()` (Python's `type()`) tells you which one you are holding:

**R**

```r
class(3.14)          # numeric: real numbers
```

```
[1] "numeric"
```

```r
class("statistics")  # character: text, always in quotes
```

```
[1] "character"
```

```r
class(TRUE)          # logical: TRUE or FALSE
```

```
[1] "logical"
```

### Python

```python
type(3.14)          # float: real numbers
```

```
<class 'float'>
```

```python
type("statistics")  # str: text, always in quotes
```

```
<class 'str'>
```

```python
type(True)          # bool: True or False
```

```
<class 'bool'>
```

Numbers you know. **Character** values (strings) are text wrapped in quotes: without the quotes, `statistics` would be the name of a variable. **Logical** values are the atoms of decision-making in a program; Chapter 2 is largely about them, so we park them here.

**Note for Python:** the type names differ (`float` for numeric, `str` for character, `bool` for logical) and so does the spelling of the logicals: R has `TRUE` and `FALSE` in full capitals (R is the language of anger), Python writes `True` and `False` with a capital initial only.

## 1.4 Vectors

Statistics is not done one number at a time: you have $n$ observations, and you want to act on all of them at once. R's fundamental data structure, the **vector**, is exactly that: an ordered collection of values of the same type. The workhorse for creating one is `c()`, short for *combine*.

**Note for Python:** plain Python has *lists* (`[1.62, 1.85]`, square brackets), but the object that matches R's vector, elementwise mathematics included, is the **array** from the **numpy** library, the foundation of all scientific Python. Load it once per script with `import numpy as np`; everything in it is then available as `np.something`, and `np.array([...])` is the equivalent of `c()`. Every vector and matrix in these notes goes through numpy. For the future, other modern Python libraries that use arrays and tensors (pandas, PyTorch, TensorFlow, etc.) are built on top or speak happily with numpy.

**R**

```r
heights <- c(1.62, 1.85, 1.71, 1.68, 1.79)   # heights in metres
heights
```

```
[1] 1.62 1.85 1.71 1.68 1.79
```

```r
length(heights)   # how many elements?
```

```
[1] 5
```

**Python**

```python
import numpy as np                                    # load numpy once per script

heights = np.array([1.62, 1.85, 1.71, 1.68, 1.79])    # heights in metres
heights
```

```
array([1.62, 1.85, 1.71, 1.68, 1.79])
```

```python
len(heights)   # how many elements?
```

```
5
```

Vectors of text work the same way (`c("Ada", "Bea", "Cal")`), but one vector cannot mix types: if you try `c(1, "two")`, R silently converts everything to character, worth knowing before it surprises you. (numpy plays it exactly the same: `np.array([1, "two"])` comes back as two strings.)

### 1.4.1 Sequences and repetitions

Typing long vectors by hand does not scale, so both languages give you generators. In R the colon operator builds integer sequences, `seq()` builds general ones, and `rep()` repeats; numpy's counterparts are `np.arange`, `np.linspace`, `np.tile` and `np.repeat`:

**R**

```r
1:10                                # integers from 1 to 10
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```r
5:-5                                # sequences can go downhill
```

```
 [1]  5  4  3  2  1  0 -1 -2 -3 -4 -5
```

```r
seq(from = 0, to = 1, by = 0.1)     # step size 0.1
```

```
 [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```r
seq(from = 0, to = 1, length.out = 5)  # exactly 5 equally spaced points
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```r
rep(c(1, 2), times = 3)             # repeat the whole vector 3 times
```

```
[1] 1 2 1 2 1 2
```

```r
rep(c(1, 2), each = 3)              # repeat each element 3 times
```

```
[1] 1 1 1 2 2 2
```

**Python**

```python
np.arange(1, 11)                    # integers from 1 to 10 (end point EXCLUDED)
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
np.arange(5, -6, -1)                # sequences can go downhill
```

```
array([ 5,  4,  3,  2,  1,  0, -1, -2, -3, -4, -5])
```

```
np.arange(0, 1.1, 0.1)              # step size 0.1 (stop just past 1 so 1 makes it in)
```

```
array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. ])
```

```
np.linspace(0, 1, 5)                # exactly 5 equally spaced points
```

```
array([0.  , 0.25, 0.5 , 0.75, 1.  ])
```

```
np.tile([1, 2], 3)                  # repeat the whole vector 3 times
```

```
array([1, 2, 1, 2, 1, 2])
```

```
np.repeat([1, 2], 3)                # repeat each element 3 times
```

```
array([1, 1, 1, 2, 2, 2])
```

**Note for Python:** `np.arange(a, b)` stops *one step before* `b`: the end point is excluded, which is why the integers 1 to 10 take `np.arange(1, 11)`. This end-exclusion habit runs right through Python (it returns in slicing, below), so burn it in now.

`seq()` is our first function with named arguments: `from`, `to`, `by`, `length.out`. Naming the arguments (rather than relying on their order) keeps calls readable; check `?seq` for the full story. Python has named (*keyword*) arguments too: `np.linspace(start = 0, stop = 1, num = 5)` reads just as well.

Exercise 1.3: Sequence like you mean it

Produce each of the following vectors in a single command, without typing out all the elements:

a. $0, 0.25, 0.5, \ldots, 3$
b. $10, 8, 6, 4, 2, 0$
c. $1, 1, 1, 2, 2, 2, 3, 3, 3$
d. $1, 2, 3, 1, 2, 3, 1, 2, 3$

e. 21 equally spaced points from $-1$ to 1.

### 1.4.2 Vectorised arithmetic

Arithmetic on vectors happens **elementwise**, with no extra effort on your part. A scalar is applied to every element:

**R**

```
x <- c(1, 2, 77, 2, -2, 14)
x + 2
```

```
[1]  3  4 79  4  0 16
```

```
x * 10
```

```
[1]  10  20 770  20 -20 140
```

```
x^2
```

```
[1]    1    4 5929    4    4  196
```

**Python**

```
x = np.array([1, 2, 77, 2, -2, 14])
x + 2
```

```
array([ 3,  4, 79,  4,  0, 16])
```

```
x * 10
```

```
array([ 10,  20, 770,  20, -20, 140])
```

```
x**2
```

```
array([   1,    4, 5929,    4,    4,  196])
```

Two vectors of the same length are combined element by element:

**R**

```r
y <- c(1, 1, 2, 2, 3, 3)
x + y
```

```
[1]  2  3 79  4  1 17
```

```r
x * y
```

```
[1]   1   2 154   4  -6  42
```

**Python**

```python
y = np.array([1, 1, 2, 2, 3, 3])
x + y
```

```
array([ 2,  3, 79,  4,  1, 17])
```

```python
x * y
```

```
array([  1,   2, 154,   4,  -6,  42])
```

And mathematical functions map over every element too:

**R**

```r
sqrt(c(1, 4, 9, 16))
```

```
[1] 1 2 3 4
```

```r
exp(c(0, 1, 2))
```

```
[1] 1.000000 2.718282 7.389056
```

**Python**

```python
np.sqrt(np.array([1, 4, 9, 16]))
```

```
array([1., 2., 3., 4.])
```

```
np.exp(np.array([0, 1, 2]))
```

```
array([1.        , 2.71828183, 7.3890561 ])
```

This style, one expression acting on $n$ numbers at once, is called **vectorised** code. It is how R (and numpy, which exists for precisely this) wants to be written: compact, close to the mathematical notation, and (as we will measure in Chapter 2) much faster than handling elements one at a time.

Some functions instead *summarise* a whole vector into one number, which covers a good share of day-one statistics:

**R**

```
sum(heights)
```

```
[1] 8.65
```

```
mean(heights)
```

```
[1] 1.73
```

```
max(heights)
```

```
[1] 1.85
```

```
min(heights)
```

```
[1] 1.62
```

```
which.max(heights)   # the POSITION of the largest element
```

```
[1] 2
```

**Python**

```
np.sum(heights)
```

```
np.float64(8.649999999999999)
```

```
np.mean(heights)
```

```
np.float64(1.7299999999999998)
```

```
np.max(heights)
```

```
np.float64(1.85)
```

```
np.min(heights)
```

```
np.float64(1.62)
```

```
np.argmax(heights)   # the INDEX of the largest element -- counted from 0!
```

```
np.int64(1)
```

(numpy also offers these as *methods* attached to the array itself, `heights.mean()`, a spelling you will meet everywhere in the wild; the function form keeps our columns aligned. And yes, `which.max` and `np.argmax` just disagreed by one about the same element: the next-but-one section explains why.)

### 1.4.3 Recycling, and its footgun

What if the two vectors have different lengths? R does not give up: it **recycles** the shorter vector, repeating it until it matches the longer one. Sometimes that is exactly what you want.

**Note for Python (recycling vs broadcasting):** numpy does *not* recycle. Its rule, called **broadcasting**, only stretches a scalar (or, once we reach matrices, a length-1 axis) to fit; two vectors of lengths 6 and 2 simply refuse to combine, with an error, not a warning. That is why the Python column below repeats the short array explicitly with `np.tile`. You lose a convenience but dodge many possible errors:

**R**

```r
1:6 + c(0, 100)      # c(0, 100) becomes 0, 100, 0, 100, 0, 100
```

```
[1]   1 102   3 104   5 106
```

```r
1:6 * c(1, -1)       # alternating signs, in one command
```

```
[1]  1 -2  3 -4  5 -6
```

**Python**

```python
np.arange(1, 7) + np.tile([0, 100], 3)   # numpy will NOT stretch a length-2 array: tile it y
```

```
array([  1, 102,   3, 104,   5, 106])
```

```python
np.arange(1, 7) * np.tile([1, -1], 3)    # alternating signs, repetition made explicit
```

```
array([ 1, -2,  3, -4,  5, -6])
```

R only complains when the longer length is not a multiple of the shorter, and even then it is a *warning*, not an error, and you still get an answer. numpy, meanwhile, refuses outright and names the two shapes it could not reconcile:

**R**

```r
1:6 + c(0, 10, 100, 1000)
```

```
Warning in 1:6 + c(0, 10, 100, 1000): longer object length is not a multiple of
shorter object length
```

```
[1]    1   12  103 1004    5   16
```

**Python**

```python
np.arange(1, 7) + np.array([0, 10, 100, 1000])
```

```
ValueError: operands could not be broadcast together with shapes (6,) (4,)
```

If your two vectors were *supposed* to be the same length and are not (say a typo dropped an observation, or an error in a cycle…), R may silently recycle and hand you plausible-looking garbage. When a warning like the one above appears, treat it as an error until proven innocent, and check your `length()`s.

Exercise 1.4: The recycling plant

For each command, think about the predicted output *before* running it, then check.

a. `1:6 + c(0, 10)`
b. `c(1, 2, 3) * c(1, -1)`
c. `1:8 / c(1, 2, 4, 8)`
d. `(1:3)^(1:6)`

On the Python side the 4 lines above will be:

a. `np.arange(1, 7) + np.tile([0, 10], 3)`
b. `np.tile(np.array([1, 2, 3]), 2) * np.tile(np.array([1, -1]), 3)`
c. `np.arange(1, 9) / np.tile(np.array([1, 2, 4, 8]), 2)`
d. `np.tile(np.arange(1, 4), 2) ** np.tile(np.arange(1, 7), 1)`

If you remove the `np.tile` calls, what happens?

### 1.4.4 Indexing: getting elements out

Every element of a vector has a position, and here the two languages part ways in the most consequential fashion of the whole module.

1-based vs 0-based indexing (read this twice)

R counts positions **from 1**, Python **from 0**: the first element is `x[1]` in R but `x[0]` in Python, and the element in position $k$ of your maths is `x[k]` in R but `x[k - 1]` in Python.
Negative indices are worse than different: they are *false friends*. In R, `x[-1]` means "everything **except** the first element". In Python, `x[-1]` means "count **from the end**": it *is* the last element (a beloved idiom, no `len(x) - 1` needed). The same characters, two entirely different results, and no error message to warn you. More translation bugs are born on this line than anywhere else: when in doubt, print and check.

**R**

```r
x <- c(5, 8, 1, 9, 3, 7)
x[1]           # first element
```

```
[1] 5
```

```r
x[3]            # third element
```

```
[1] 1
```

```r
x[length(x)]    # last element, whatever the length is
```

```
[1] 7
```

**Python**

```python
x = np.array([5, 8, 1, 9, 3, 7])
x[0]            # first element
```

```
np.int64(5)
```

```python
x[2]            # third element
```

```
np.int64(1)
```

```python
x[len(x) - 1]  # last element (see the note: x[-1] is slicker)
```

```
np.int64(7)
```

The index can itself be a vector, which lets you take slices, cherry-pick, or reorder:

**R**

```r
x[2:4]          # elements 2 to 4
```

```
[1] 8 1 9
```

```r
x[c(1, 5)]      # elements 1 and 5
```

```
[1] 5 3
```

```r
x[c(5, 1)]     # same elements, opposite order
```

```
[1] 3 5
```

**Python**

```python
x[1:4]         # elements 2 to 4: indices 1, 2, 3 -- the slice EXCLUDES its end
```

```
array([8, 1, 9])
```

```python
x[[0, 4]]      # elements 1 and 5
```

```
array([5, 3])
```

```python
x[[4, 0]]      # same elements, opposite order
```

```
array([3, 5])
```

(There is the end-exclusion again: Python's slice `a:b` runs from index `a` up to, but not including, index `b`, just like `np.arange`.)

A **negative** index means "everything except" in R, and something else entirely in Python:

**R**

```r
x[-1]          # drop the first element
```

```
[1] 8 1 9 3 7
```

```r
x[-c(2, 3)]    # drop the second and third
```

```
[1] 5 9 3 7
```

**Python**

```python
np.delete(x, 0)        # drop the first element -- x[-1] would NOT do this...
```

```
array([8, 1, 9, 3, 7])
```

```
np.delete(x, [1, 2])   # drop the second and third
```

```
array([5, 9, 3, 7])
```

```
x[-1]                  # ...because in Python this counts from the end
```

```
np.int64(7)
```

And indexing on the left of the assignment overwrites elements in place:

**R**

```
x[3] <- 100    # replace the third element
x
```

```
[1]   5   8 100   9   3   7
```

```
x[c(1, 2)] <- 0  # replace several at once
x
```

```
[1]   0   0 100   9   3   7
```

**Python**

```
x[2] = 100     # replace the third element
x
```

```
array([  5,   8, 100,   9,   3,   7])
```

```
x[[0, 1]] = 0  # replace several at once
x
```

```
array([  0,   0, 100,   9,   3,   7])
```

One teaser before we move on: in both languages, brackets also accept conditions, so `x[x > 50]` returns the elements larger than 50. How and why that works is the business of Chapter 2: resist the urge for one more week.

Exercise 1.5: Marking spree

Ten students sat a test, scoring (out of 100):

**R**

```r
marks <- c(63, 58, 71, 45, 82, 69, 77, 52, 88, 60)
```

**Python**

```python
marks = np.array([63, 58, 71, 45, 82, 69, 77, 52, 88, 60])
```

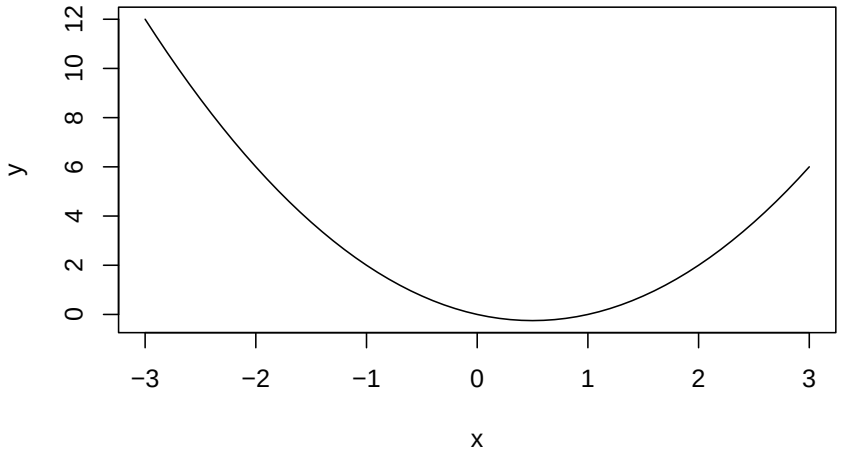
a. Compute the mean mark and the range (maximum minus minimum).
b. Which student (i.e. which *position*) got the top mark? Extract their mark using that position (in one command, without typing the position yourself).
c. Extract the marks of students 2, 5 and 9 in a single command.
d. Extract everyone *except* student 4, who has an exceptional-circumstances claim pending.
e. The lecturer, feeling generous, adds 3 marks to everyone. Compute the new vector and its mean. How does the new mean relate to the old one, and why did that not require any statistics?

### 1.4.5 A first plot

Two vectors of equal length are begging to be plotted against each other, and base R obliges with `plot()`; Python's counterpart is the **matplotlib** library:

**R**

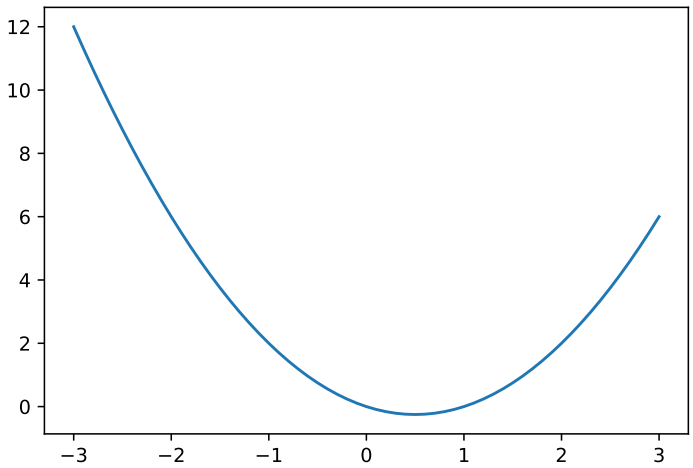
```r
x <- seq(-3, 3, by = 0.1)
y <- x^2 - x
plot(x, y, type = "l")   # type = "l" joins the points with a line
```




**Python**

```
import matplotlib.pyplot as plt   # Python's plotting library

x = np.arange(-3, 3.1, 0.1)
y = x**2 - x
plt.plot(x, y)    # plt.plot draws a line by default
plt.show()
```




That is genuinely all it takes to graph a function. (One asymmetry: base R plots points unless you ask for a line with `type = "l"`; matplotlib's `plt.plot` draws a line unless you ask otherwise; `plt.scatter` is its point-plotter.) We will keep using quick one-liners like this

as visual checks, and do plotting *properly* (labels, colours, themes, publication quality) in Chapter 4.

## 1.5 Saving your work: csv files and the working directory

Variables live in memory, and memory is wiped when your session closes. To keep results, or to share them with anyone who is not inside your session, you write them to a file on disk. The standard exchange format for data is the **csv** (comma-separated values) file, a plain text table that every spreadsheet and statistics package can read:

**R**

```r
lancaster_rain <- c(12.5, 8.0, 0.0, 3.2, 21.7)   # mm of rain, Mon-Fri. The 0.0 is a typo, su
write.csv(lancaster_rain, file = "week1_rain.csv", row.names = FALSE)
```

**Python**

```python
import pandas as pd                                # Python's table library

lancaster_rain = np.array([12.5, 8.0, 0.0, 3.2, 21.7])   # mm of rain, Mon-Fri. The 0.0 is a
pd.DataFrame(lancaster_rain).to_csv("week1_rain_py.csv", index=False)
```

**Note for Python:** csv duty falls to **pandas**, Python's data-frame library (imported as `pd` by near-universal convention). pandas thinks in tables, so the array is wrapped in a `DataFrame` on its way to disk, and `index=False` is the counterpart of R's `row.names = FALSE` (note that in the example above the Python column writes to `week1_rain_py.csv`: a different file name so we don't overwrite the R one).

But *where* did that file go? Into your **working directory**: the folder on disk where your session currently "stands". Any file name without a folder in front of it is read from, or written to, this directory.

**R**

```r
getwd()          # print the current working directory
```

**Python**

```python
import os
os.getcwd()      # print the current working directory
```

You can point elsewhere with a **path**: `"data/week1_rain.csv"` means "inside the folder `data` under the working directory", and `"../week1_rain.csv"` means "one folder up".

**R**

```
write.csv(lancaster_rain, file = "data/week1_rain.csv")  # needs a 'data' folder to exist
write.csv(lancaster_rain, file = "../week1_rain.csv")    # parent directory
```

**Python**

```
pd.DataFrame(lancaster_rain).to_csv("data/week1_rain_py.csv")  # needs a 'data' folder to ex
pd.DataFrame(lancaster_rain).to_csv("../week1_rain_py.csv")    # parent directory
```

> ⚠ Hard-coded filepaths
>
> Rather than hard-coding long paths like `"C:/Users/you/Desktop/stuff/..."` (which break the moment the script moves to another machine), the sane workflow is to keep your scripts and data in one folder and make that the working directory (in RStudio: *Session > Set Working Directory > To Source File Location*).

Reading the file back in is the mirror image:

**R**

```
rain_from_file <- read.csv("week1_rain.csv")
rain_from_file
```

```
     x
1 12.5
2  8.0
3  0.0
4  3.2
5 21.7
```

**Python**

```
rain_from_file = pd.read_csv("week1_rain_py.csv")
rain_from_file
```

```
      0
0  12.5
1   8.0
2   0.0
3   3.2
4  21.7
```

The printout looks slightly different from a vector: it has a column name (`x` in R; pandas defaults to `0`) and numbered rows (from 0 in Python, naturally). That is because `read.csv()` (and equally pandas' `read_csv()`) returns a **data frame**: the table-shaped cousin of the vector, and the single most important object in applied statistics. It gets the star treatment in Chapter 4; for now, csv out and csv in is all we need.

Exercise 1.6: There and back again (a csv's tale)

a. Create a vector `temps` with the daily maximum temperatures $4.2, 6.8, 7.5, 3.1, 5.0, 8.4, 6.9$ (°C, a week in a Lancaster spring).
b. Write it to a file called `week1_temps.csv` (Python: `week1_temps_py.csv`, so the two languages stay out of each other's way), without row names (pandas: without the index).
c. Find the file on disk and open it with a plain text editor (Notepad, TextEdit), *not* Excel. Describe what you see: how are the values laid out?
d. Read the file back into a new variable and check the values survived the round trip.
e. What does the file look like if you forget `row.names = FALSE` (`index=False`) in part (b)?

## 1.6 Matrices and a taste of linear algebra

You know matrices from linear algebra; both our languages know them too, and they are very good at them. A matrix in R is a vector wrapped onto rows and columns by `matrix()`; in numpy, a matrix is simply a 2-D array. Before the first example, one crucial difference in the defaults: given the same numbers, **R fills a matrix column by column** (the *column-major* convention, inherited from Fortran), while **numpy fills it row by row** (the *row-major* convention of the C language). Same six numbers, two different matrices:

**R**

```r
A <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)   # fills column by column (default)
A
```

```
     [,1] [,2] [,3]
```

```
[1,]    1    3    5
[2,]    2    4    6
```

**Python**

```
A = np.array([1, 2, 3, 4, 5, 6]).reshape(2, 3)   # fills row by row (default)
A
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

Both languages can also fill the other way; you just have to say so. In R, `byrow = TRUE` switches to filling by row; in numpy, `order="F"` ("F" for Fortran) switches to filling by column. With the switches on, each language reproduces the other's default:

**R**

```
A <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, byrow = TRUE)   # by row instead
A
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

**Python**

```
A = np.array([1, 2, 3, 4, 5, 6]).reshape((2, 3), order="F")   # by column instead
A
```

```
array([[1, 3, 5],
       [2, 4, 6]])
```

The dimensions are always rows first, columns second:

**R**

```
dim(A)     # rows, columns
```

```
[1] 2 3
```

```r
nrow(A)
```

```
[1] 2
```

```r
ncol(A)
```

```
[1] 3
```

**Python**

```python
A.shape      # (rows, columns)
```

```
(2, 3)
```

```python
A.shape[0]   # rows
```

```
2
```

```python
A.shape[1]   # columns
```

```
3
```

You can also build matrices by gluing vectors together: `cbind()` (numpy: `np.column_stack`) binds them as columns, `rbind()` (`np.vstack`) as rows. Remember the column one: it earns its keep in the challenge at the end of this chapter.

**R**

```r
cbind(c(1, 2), c(3, 4))   # two columns
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

```r
rbind(c(1, 2), c(3, 4))   # two rows
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
```

**Python**

```
np.column_stack([[1, 2], [3, 4]])   # two columns
```

```
array([[1, 3],
       [2, 4]])
```

```
np.vstack([[1, 2], [3, 4]])         # two rows
```

```
array([[1, 2],
       [3, 4]])
```

**Note for Python:** in practice the idiomatic numpy follows the filling order entirely: hand `np.array` a list of rows and the code looks like the matrix, no reshaping needed. This is considered to be more pythonic. Let us rebuild `A` that way (row-wise in R too, with the `rbind()` we just met), so that from here on both languages hold the *same* matrix:

**R**

```
A <- rbind(c(1, 2, 3), c(4, 5, 6))
A
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

**Python**

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])   # a list of rows: numpy's natural reading order
A
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

### 1.6.1 Indexing a matrix

Same square brackets as vectors, now with two positions: `[row, column]`. Leaving one slot empty (R) or putting a bare colon `:` in it (Python) means "all of them":

**R**

```r
A[1, 2]    # row 1, column 2
```

```
[1] 2
```

```r
A[2, ]     # all of row 2
```

```
[1] 4 5 6
```

```r
A[, 3]     # all of column 3
```

```
[1] 3 6
```

```r
A[, c(1, 3)]  # columns 1 and 3
```

```
     [,1] [,2]
[1,]    1    3
[2,]    4    6
```

**Python**

```python
A[0, 1]    # row 1, column 2 (0-based, as ever)
```

```
np.int64(2)
```

```python
A[1, :]    # all of row 2
```

```
array([4, 5, 6])
```

```python
A[:, 2]    # all of column 3
```

```
array([3, 6])
```

```
A[:, [0, 2]]  # columns 1 and 3
```

```
array([[1, 3],
       [4, 6]])
```

And `t()` (in numpy the `.T` attribute) transposes:

**R**

```
t(A)
```

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

**Python**

```
A.T
```

```
array([[1, 4],
       [2, 5],
       [3, 6]])
```

### 1.6.2 Matrix arithmetic: `*` is not what you think

Here is the classic trap, and it is the same trap in both languages. The ordinary arithmetic operators work on matrices exactly as they do on vectors, **elementwise**:

**R**

```
B <- matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE)
C <- matrix(c(10, 20, 30, 40), nrow = 2, byrow = TRUE)
B + C
```

```
     [,1] [,2]
[1,]   11   22
[2,]   33   44
```

```r
B * C     # elementwise! NOT the matrix product
```

```
     [,1] [,2]
[1,]   10   40
[2,]   90  160
```

**Python**

```python
B = np.array([[1, 2], [3, 4]])
C = np.array([[10, 20], [30, 40]])
B + C
```

```
array([[11, 22],
       [33, 44]])
```

```python
B * C     # elementwise! NOT the matrix product
```

```
array([[ 10,  40],
       [ 90, 160]])
```

The actual matrix product (rows times columns, as in $(BC)_{ij} = \sum_k B_{ik}C_{kj}$) has its own operator: `%*%` in R, `@` in Python:

**R**

```r
B %*% C
```

```
     [,1] [,2]
[1,]   70  100
[2,]  150  220
```

**Python**

```python
B @ C
```

```
array([[ 70, 100],
       [150, 220]])
```

The usual conformity rule applies: an $(n \times k)$ matrix can only `%*%` a $(k \times m)$ one. Writing `*` where you meant `%*%` (or `@`) is a rite of passage in either language; the resulting bug is nasty precisely because both versions run without error and return a matrix of the same size. Check a small case by hand when in doubt: that is, conveniently, the next exercise.

Exercise 1.7: Matrix products, by hand and by machine

Let

$$A = \begin{pmatrix} 2 & 0 \\ 1 & 3 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 4 \\ 2 & 5 \end{pmatrix}.$$

a. Compute $AB$ with pen and paper.
b. Build `A` and `B` (mind R's filling order!) and verify your answer with `%*%` (Python: `@`).
c. Compute `A * B` and explain, in one sentence, what your language did instead.
d. Compute $B^\top A$ using `t()` and `%*%` (`.T` and `@`). Is it equal to $(AB)^\top$? Should it be?

### 1.6.3 Inverses, determinants and linear systems

The function with the most misleading name in base R, `solve()`, computes matrix inverses:

**R**

```r
B_inv <- solve(B)   # the inverse of B
B_inv
```

```
     [,1] [,2]
[1,] -2.0  1.0
[2,]  1.5 -0.5
```

```r
round(B %*% B_inv, 10)   # B times its inverse: the identity (up to rounding)
```

```
     [,1] [,2]
[1,]    1    0
[2,]    0    1
```

**Python**

```python
B_inv = np.linalg.inv(B)   # the inverse of B
B_inv
```

```
array([[-2. ,  1. ],
       [ 1.5, -0.5]])
```

```
np.round(B @ B_inv, 10)    # B times its inverse: the identity (up to rounding)
```

```
array([[1., 0.],
       [0., 1.]])
```

**Note for Python:** numpy files all of linear algebra under `np.linalg`, and names things honestly: `np.linalg.inv` inverts, `np.linalg.solve` (coming below) solves systems, `np.linalg.det` takes determinants. R's `solve()` carries the misleading-name burden alone.

(The `round()` is there because floating-point arithmetic leaves microscopic dust like `2.2e-16` where exact zeros should be; more on that in Chapter 2.) The identity matrix itself, should you need one, is `diag(2)` in R, `np.eye(2)` in Python.

A matrix is invertible if and only if its determinant is non-zero, and `det()` (`np.linalg.det`) will tell you:

**R**

```
det(B)
```

```
[1] -2
```

```
S <- matrix(c(1, 2, 2, 4), nrow = 2)   # second column = 2 * first: singular
det(S)
```

```
[1] 0
```

**Python**

```
np.linalg.det(B)                # -2, plus a speck of that floating-point dust
```

```
np.float64(-2.0000000000000004)
```

```
S = np.array([[1, 2], [2, 4]])   # second column = 2 * first: singular
np.linalg.det(S)
```

```
np.float64(0.0)
```

Asking for the inverse of a singular matrix fails loudly in both languages (an *error*, not a warning: priorities in the right place):

**R**

```r
solve(S)
```

```
Error in `solve.default()`:
! Lapack routine dgesv: system is exactly singular: U[2,2] = 0
```

**Python**

```python
np.linalg.inv(S)
```

```
numpy.linalg.LinAlgError: Singular matrix
```

The name `solve()` starts making sense with its two-argument form, which solves the linear system $A\mathbf{x} = \mathbf{b}$. For

$$\begin{aligned} x + 2y &= 5 \\ 3x + 4y &= 6 \end{aligned} \qquad \text{i.e.} \qquad \underbrace{\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}}_{A} \mathbf{x} = \underbrace{\begin{pmatrix} 5 \\ 6 \end{pmatrix}}_{\mathbf{b}},$$

**R**

```r
A <- matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE)
b <- c(5, 6)
solve(A, b)
```

```
[1] -4.0  4.5
```

**Python**

```python
A = np.array([[1, 2], [3, 4]])
b = np.array([5, 6])
np.linalg.solve(A, b)
```

```
array([-4. ,  4.5])
```

Mathematically this is $\mathbf{x} = A^{-1}\mathbf{b}$, and `solve(A) %*% b` (Python: `np.linalg.inv(A) @ b`) gives the same answer, but the two-argument `solve(A, b)` / `np.linalg.solve(A, b)` never forms the inverse explicitly (it uses LU decomposition if you're curious), which is both faster and numerically more stable. Statistics is full of linear systems!

Exercise 1.8: Three equations, three unknowns

Consider the system

$$\begin{aligned} 2x + y - z &= 8 \\ -3x - y + 2z &= -11 \\ -2x + y + 2z &= -3. \end{aligned}$$

a. Build the coefficient matrix `A` and the right-hand side `b`.
b. Check that the system has a unique solution (which function tells you?).
c. Solve it, and verify your solution by computing `A %*% x_sol` (`A @ x_sol`) and comparing with `b`.

### 1.6.4 An eigenvalue teaser

One more linear-algebra tool, offered now as a teaser which will be useful later. A 30-second reminder first: a square matrix $A$ defines a map $\mathbf{v} \mapsto A\mathbf{v}$ that can rotate, stretch or compress, so the output $A\mathbf{v}$ generally points in a different direction from the input $\mathbf{v}$, with a different length. An **eigenvector** of $A$ is a special vector that $A$ maps to a multiple of itself:

$$A\mathbf{v} = \lambda\,\mathbf{v}, \qquad \mathbf{v} \neq \mathbf{0}.$$

$A\mathbf{v}$ lies on the same line as $\mathbf{v}$, merely stretched or shrunk (or flipped, when $\lambda < 0$), and the stretch factor $\lambda$ is its **eigenvalue**.

A symmetric $d \times d$ matrix, which is the kind statistics mostly cares about, has a full set of $d$ such directions, mutually perpendicular (orthogonal), with real eigenvalues. So intuitively, the eigenvectors are the axes along which the matrix acts as ordinary scalar multiplication. Better still, the $d$ eigen-directions $\mathbf{v}_1, \ldots, \mathbf{v}_d$ of a symmetric matrix form a basis: *any* vector $\mathbf{w}$ can be written as a combination $\mathbf{w} = c_1\mathbf{v}_1 + \cdots + c_d\mathbf{v}_d$, and applying $A$ piece by piece, each component just picks up its own stretch factor:

$$A\mathbf{w} = c_1\lambda_1\mathbf{v}_1 + \cdots + c_d\lambda_d\mathbf{v}_d.$$

The matrix's entire action, on any vector at all, is therefore captured by $d$ separate stretches. The size of each $\lambda$ says what happens along its axis: a large $|\lambda|$ amplifies, a small one flattens, and $\lambda = 0$ destroys the direction outright (that is precisely the singular matrix met earlier in this chapter).

Here is all of this in one picture, for the matrix $\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$ we will meet in a moment.

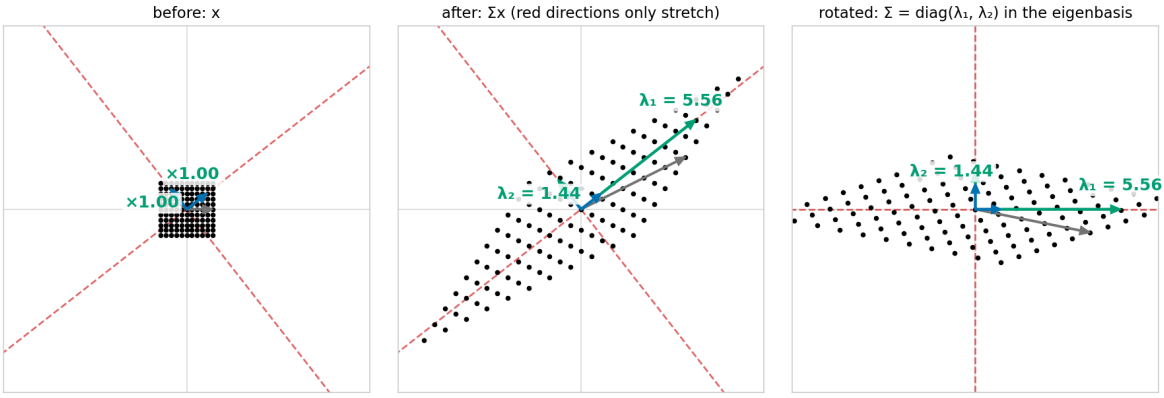



Figure 1.1: The map $\mathbf{x} \mapsto \Sigma\mathbf{x}$ acting on a grid of points (left: before; middle: after). Points on the two dashed red eigen-directions only slide along them; each eigenvector (blue) is stretched by its eigenvalue (green): $\lambda_1 \approx 5.56$ and $\lambda_2 \approx 1.44$. The grey vector, not an eigenvector, changes direction. Right: rotating until the eigenvectors are the axes, $\Sigma$ is nothing but a stretch along each axis. (The HTML version of these notes animates this.)

In statistics, the **covariance matrix** of a random vector packages the variances (diagonal) and covariances (off-diagonal) of its components. Its eigendecomposition can be quite informative:

**R**

```r
Sigma <- matrix(c(4, 2, 2, 3), nrow = 2)   # a 2x2 covariance matrix
eigen(Sigma)
```

```
eigen() decomposition
$values
[1] 5.561553 1.438447

$vectors
           [,1]       [,2]
[1,] -0.7882054  0.6154122
[2,] -0.6154122 -0.7882054
```

**Python**

```python
Sigma = np.array([[4, 2], [2, 3]])   # a 2x2 covariance matrix
evals, evecs = np.linalg.eig(Sigma)
evals
```

```
array([5.56155281, 1.43844719])
```

```
evecs
```

```
array([[ 0.78820544, -0.61541221],
       [ 0.61541221,  0.78820544]])
```

**Note for Python:** `np.linalg.eig` hands back a *pair* (eigenvalues, eigenvectors) rather than R's single object, and unlike R it makes no promise about their order (here it happens to match R's largest-first). For symmetric matrices, and covariance matrices always are, the specialised `np.linalg.eigh` is faster, more accurate, and sorts eigenvalues in *ascending* order, so the "top" eigenvalue comes last.

The eigenvector with the largest eigenvalue points in the direction along which the data varies most, and the eigenvalues split the total variance across those directions. This single idea is principal component analysis, the workhorse of dimension reduction. Eigenvalues will also quietly reappear in Chapter 5, where they govern how fast our optimisation algorithms converge. For now: `eigen()` (and `np.linalg.eig`) exists, and Challenge 1.2 lets you poke at it.

## 1.7 Challenges

Two optional problems to pull the week's vectors and matrices together.

**!** Challenge 1.1: Least squares, a sneak preview

A company records advertising spend $x$ (in £1000s) and sales $y$ (also £1000s) over eight campaigns:

**R**

```
x <- c(1.0, 1.9, 3.1, 3.8, 5.2, 6.0, 7.1, 8.2)
y <- c(2.4, 3.9, 6.3, 7.4, 10.4, 11.7, 14.1, 16.3)
```

**Python**

```
x = np.array([1.0, 1.9, 3.1, 3.8, 5.2, 6.0, 7.1, 8.2])
y = np.array([2.4, 3.9, 6.3, 7.4, 10.4, 11.7, 14.1, 16.3])
```

The best-fitting line $y = \beta_0 + \beta_1 x$ (in the least-squares sense) has coefficients

$$\hat{\beta} = (X^\top X)^{-1} X^\top \mathbf{y},$$

where $X$ is the matrix whose first column is all 1s and whose second column is $x$.

a. Build $X$ with `cbind()` (a neat trick: `cbind(1, x)` recycles the 1). In Python, where there is no recycling, build the column of ones explicitly: `np.column_stack([np.ones(len(x)), x])`.
b. Compute $\hat{\beta}$ with `t()`, `%*%` and `solve()` (Python: `.T`, `@` and `np.linalg.inv`, or better, `np.linalg.solve`).
c. Plot the data with `plot(x, y)` and overlay your line with `abline(a = beta[1], b = beta[2])`. (matplotlib has no `abline`: use `plt.scatter(x, y)`, then draw the line yourself with `plt.plot(x, beta[0] + beta[1] * x)`.) Admire.

You have just fitted your first regression, with your bare hands, using only week-1 material. In Chapter 5 we will fit models like this by *optimisation*, and see exactly where this magic formula comes from.

**!** Challenge 1.2: Dissecting a covariance matrix

Take the $3 \times 3$ covariance matrix

$$\Sigma = \begin{pmatrix} 2.0 & 0.8 & 0.4 \\ 0.8 & 1.5 & 0.6 \\ 0.4 & 0.6 & 1.0 \end{pmatrix}.$$

`eigen(Sigma)` returns an object with two pieces, which you can extract as `eigen(Sigma)$values` and `eigen(Sigma)$vectors` (more on the `$` in Chapter 4). In Python the two pieces arrive directly as a pair: `evals, evecs = np.linalg.eig(Sigma)`.

a. Verify numerically that the eigenvalues sum to the *total variance* $\mathrm{tr}(\Sigma)$, the sum of the diagonal (see `?diag`; Python: `np.diag`, or `np.trace` for the whole job).
b. Store the eigenvector matrix as `V` and compute `t(V) %*% V` (`V.T @ V`), rounded to 10 decimal places. What matrix do you get, and what does that say about the directions the eigenvectors describe?
c. What proportion of the total variance lies along the first eigenvector? In principal-component speak, that is the "variance explained by PC1".

## 1.8 What you can do now

- Use both R and Python as calculators, keep results in variables, and organise it all in commented scripts.
- Build vectors with `c()`, `seq()` and `rep()` (numpy: `np.array`, `np.arange`/`np.linspace`, `np.tile`/`np.repeat`), slice them by position (minding who counts from 1 and who from 0) and act on all elements at once with vectorised arithmetic; recycling (R) and broadcasting (numpy) warning bells installed.

- Round-trip your data to disk as csv files (`write.csv`/`read.csv`; pandas' `to_csv`/`read_csv`), and know where your session put them.
- Do real linear algebra: `%*%` (`@`), `t()` (`.T`), `det()`, `solve()` for inverses and systems (`np.linalg`'s `det`, `inv` and `solve`), and a first taste of `eigen()` (`np.linalg.eig`).

Next week, in Chapter 2, we teach the machine to make decisions: `TRUE` and `FALSE` (`True` and `False`) in earnest, `if` and `else`, loops, and with them our first proper algorithm, gradient descent, which will carry us all the way to Chapter 5.

# 2 Week 2: Logic, Loops and Iteration

Last week (Chapter 1) we learned about computer arithmetic. This week learn how to *make decisions* using logic and to *iterate a chunk of code*: the two ingredients that turn a calculator into an algorithm. By the end of the chapter we will have built our first genuinely useful one: **gradient descent**, the workhorse optimiser behind most of modern statistics and machine learning (Section 2.8). Everything else in this chapter is training for that final section.

## 2.1 Comparisons

A comparison takes two values and returns a *logical*: `TRUE` or `FALSE`. The operators are `==` (equal), `!=` (not equal), `<`, `>`, `<=` and `>=`.

**R**

```
2 == 2
```

```
[1] TRUE
```

```
"Hello" == "hello"   # case matters
```

```
[1] FALSE
```

```
3 != pi
```

```
[1] TRUE
```

```
2 < 3
```

```
[1] TRUE
```

```
7 >= 7
```

```
[1] TRUE
```

### Python

```
import math          # pi lives in the math module

2 == 2
```

```
True
```

```
"Hello" == "hello"   # case matters
```

```
False
```

```
3 != math.pi
```

```
True
```

```
2 < 3
```

```
True
```

```
7 >= 7
```

```
True
```

Note the double `=` in `==`: a single `=` is assignment.

Like arithmetic, comparisons are **vectorised**: comparing a vector against a scalar (or another vector: R recycles, numpy broadcasts; Chapter 1) gives a whole vector of logicals.

### R

```
x <- c(1, 4, 9, 16)
x > 5
```

```
[1] FALSE FALSE  TRUE  TRUE
```

```r
x == c(1, 2, 9, 20)
```

```
[1]  TRUE FALSE  TRUE FALSE
```

**Python**

```python
import numpy as np   # vectors live in numpy

x = np.array([1, 4, 9, 16])
x > 5
```

```
array([False, False,  True,  True])
```

```python
x == np.array([1, 2, 9, 20])
```

```
array([ True, False,  True, False])
```

One trap worth knowing now: comparing anything with `NA` (a missing value) gives `NA`, not `FALSE`: R refuses to claim it knows how a missing value compares. To test for missingness use `is.na()`.

**Note for Python:** Python has no true missing-value marker like R's `NA`. The closest stand-in is the floating-point "not a number", `np.nan`. But where R's `NA` *propagates* through comparisons (`NA == NA` is `NA`), `nan` simply compares `False` to everything, including itself. Test for it with `np.isnan()`, never with `==`. (Proper missing-data handling arrives with pandas in Chapter 4.)

**R**

```r
NA == NA
```

```
[1] NA
```

```r
is.na(c(1, NA, 3))
```

```
[1] FALSE  TRUE FALSE
```

**Python**

```python
np.nan == np.nan
```

```
False
```

```python
np.isnan(np.array([1, np.nan, 3]))
```

```
array([False,  True, False])
```

### 2.1.1 The floating-point trap

Here is the most famous line of code in numerical computing:

**R**

```r
0.1 + 0.2 == 0.3
```

```
[1] FALSE
```

**Python**

```python
0.1 + 0.2 == 0.3
```

```
False
```

R is not broken, and Python agrees, which should tell you something. This section is entirely *language-shared*: both languages store decimals in the same IEEE 754 double-precision *binary floating point*, and just as 1/3 has no finite decimal expansion, 0.1 has no finite binary one. What gets stored is the nearest representable number, correct to about 16 significant digits, and the tiny rounding errors surface when you compare:

**R**

```r
print(0.1 + 0.2, digits = 17)
```

```
[1] 0.30000000000000004
```

```r
print(0.3,       digits = 17)
```

```
[1] 0.29999999999999999
```

**Python**

```
print(f"{0.1 + 0.2:.17g}")
```

```
0.30000000000000004
```

```
print(f"{0.3:.17g}")
```

```
0.29999999999999999
```

The rule: **never test floating-point numbers for exact equality**. Ask instead whether they are equal *up to a small tolerance*, either by hand or with a ready-made helper: `all.equal()` in R, `math.isclose()` in Python (with `np.isclose()` / `np.allclose()` as the array versions):

**R**

```
abs((0.1 + 0.2) - 0.3) < 1e-8
```

```
[1] TRUE
```

```
all.equal(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

**Python**

```
abs((0.1 + 0.2) - 0.3) < 1e-8
```

```
True
```

```
math.isclose(0.1 + 0.2, 0.3)
```

```
True
```

Careful: when the numbers differ, `all.equal()` returns a *description of the difference* rather than `FALSE`, so inside a condition wrap it as `isTRUE(all.equal(a, b))`.

**R**

```r
all.equal(1, 1.5)          # not FALSE!
```

```
[1] "Mean relative difference: 0.5"
```

```r
isTRUE(all.equal(1, 1.5))  # this is what you test
```

```
[1] FALSE
```

**Python**

```python
math.isclose(1, 1.5)   # a plain False: nothing to wrap
```

```
False
```

Exercise 2.1: The pedantic computer

Predict the result of each comparison, then run them:

a. `0.1 + 0.3 == 0.4`
b. `0.1 + 0.2 + 0.3 == 0.6`
c. `sqrt(2)^2 == 2`
d. `1e5 / (1e5 + 1e-16) == 1`

(Python: item (c) is `math.sqrt(2)**2 == 2`; recall `**` is Python's power operator. The other three run as written.)
Explain what you see, and rewrite the `FALSE` comparisons so that they return `TRUE` for the right reason.

## 2.2 Logical operators

Logicals can be combined with `&` (AND), `|` (OR) and `!` (NOT):

- `a & b` is `TRUE` only when both are `TRUE`;
- `a | b` is `FALSE` only when both are `FALSE`;
- `!a` flips the value.

**Note for Python:** on single logicals the operators are *words*: `and`, `or`, `not` (and `^` doubles as xor on booleans). Python's `&` and `|` exist too, but as elementwise operators: they become important on numpy arrays in a moment.

### R

```
TRUE & FALSE
```

```
[1] FALSE
```

```
TRUE | FALSE
```

```
[1] TRUE
```

```
!TRUE
```

```
[1] FALSE
```

```
xor(TRUE, TRUE)   # exclusive OR: TRUE when exactly one is TRUE
```

```
[1] FALSE
```

### Python

```
True and False
```

```
False
```

```
True or False
```

```
True
```

```
not True
```

```
False
```

```
True ^ True   # exclusive OR: True when exactly one is True
```

```
False
```

**Note for R** In R, comparisons bind tighter than logical operators, and `&` binds tighter than `|`, so `x > 2 & x < 5` means what you hope it means. **Note for Python:** numpy's `&` and `|` bind *tighter* than comparisons, so that same line would be read as `x > (2 & x) < 5`, so parentheses around every comparison are mandatory: `(x > 2) & (x < 5)`. When in doubt, in either language, parenthesise.

All three operators are vectorised (R uses the same `&`, `|`, `!` on vectors; numpy's elementwise trio is `&`, `|`, `~`), which lets us combine conditions elementwise:

**R**

```
x <- c(1, 4, 9, 16)
x > 2 & x < 10
```

```
[1] FALSE  TRUE  TRUE FALSE
```

```
x < 2 | x > 10
```

```
[1]  TRUE FALSE FALSE  TRUE
```

**Python**

```
x = np.array([1, 4, 9, 16])
(x > 2) & (x < 10)
```

```
array([False,  True,  True, False])
```

```
(x < 2) | (x > 10)
```

```
array([ True, False, False,  True])
```

### 2.2.1 `&&` / `and` versus `&`

R also has doubled-up operators `&&` and `||`. They work only on *single* logicals and error on vectors:

**R**

```
c(TRUE, FALSE) && TRUE
```

```
Error in `c(TRUE, FALSE) && TRUE`:
! 'length = 2' in coercion to 'logical(1)'
```

**Python**

```
np.array([True, False]) and True
```

```
ValueError: The truth value of an array with more than one element is ambiguous. Use a.any()
```

They also *short-circuit*: `a && b` never evaluates `b` if `a` is already `FALSE`. That makes them the right choice inside `if()` conditions (coming up shortly), where you want exactly one `TRUE`/`FALSE` and you may want the second check skipped when the first fails, e.g. `n > 0 && x[n] > 5` never touches `x[n]` when `n` is 0. Rule of thumb: `&`/`|` for vectors, `&&`/`||` for `if` conditions.

**Note for Python:** there is no doubled-up operator to learn, and nothing missing, either. Python's `and`/`or` *are* the single-value, short-circuiting pair (`n > 0 and x[n - 1] > 5` skips the second check exactly like R's `&&`), while `&`/`|` are the elementwise pair. As the error above shows, `and` on a numpy array refuses to guess. The rule of thumb translates directly: `&`/`|` for arrays, `and`/`or` for `if` conditions.

### 2.2.2 `any()`, `all()` and `%in%` / `in`

Two functions collapse a logical vector down to a single verdict:

**R**

```
x <- c(1, 4, 9, 16)
any(x > 10)   # is at least one element > 10?
```

```
[1] TRUE
```

```r
all(x > 0)    # are they all positive?
```

```
[1] TRUE
```

**Python**

```python
x = np.array([1, 4, 9, 16])
np.any(x > 10)   # is at least one element > 10?
```

```
np.True_
```

```python
np.all(x > 0)    # are they all positive?
```

```
np.True_
```

And `%in%` checks membership (Python: the `in` operator for a single value, `np.isin()` for a whole array), which beats chaining many `==` together with `|`:

**R**

```r
5 %in% c(1, 3, 5, 7)
```

```
[1] TRUE
```

```r
c(2, 5, 8) %in% c(1, 3, 5, 7)   # vectorised on the left
```

```
[1] FALSE  TRUE FALSE
```

**Python**

```python
5 in [1, 3, 5, 7]
```

```
True
```

```python
np.isin([2, 5, 8], [1, 3, 5, 7])   # vectorised on the left
```

```
array([False,  True, False])
```

Exercise 2.2: Leap year logic

A year is a leap year if it is divisible by 4, *except* that century years (divisible by 100) are not, *unless* they are also divisible by 400. So 2024 and 2000 are leap years; 2023 and 1900 are not.
Store a year in a variable and write a **single logical expression** (no `if` needed) that is `TRUE` exactly when it is a leap year. The modulo operator `%%` (`%` in Python) gives remainders: `2024 %% 4 == 0`. Check your expression on 2024, 2023, 2000 and 1900.

## 2.3 Logical indexing

Last week we indexed vectors by *position*: `x[3]`, `x[c(1, 5)]`. The second (and in statistics, far more useful) way is to index with a **logical vector**: both R and numpy keep the elements where the index is `TRUE`. This is sometimes referred as a mask, as it "masks" the elements you want to keep.

**R**

```r
x <- c(2, 7, 1, 9, 4)
x[c(FALSE, TRUE, FALSE, TRUE, FALSE)]
```

```
[1] 7 9
```

**Python**

```python
x = np.array([2, 7, 1, 9, 4])
x[np.array([False, True, False, True, False])]
```

```
array([7, 9])
```

Nobody types logical vectors by hand, of course. The point is that *comparisons produce them*, so we can filter a vector by any condition:

**R**

```r
x[x > 3]            # elements greater than 3
```

```
[1] 7 9 4
```

```r
x[x %% 2 == 0]      # the even ones
```

```
[1] 2 4
```

```r
x[x > 3 & x < 9]    # combined conditions
```

```
[1] 7 4
```

**Python**

```python
x[x > 3]              # elements greater than 3
```

```
array([7, 9, 4])
```

```python
x[x % 2 == 0]         # the even ones
```

```
array([2, 4])
```

```python
x[(x > 3) & (x < 9)]  # combined conditions
```

```
array([7, 4])
```

Read `x[x > 3]` inside-out: `x > 3` builds the logical vector, and `x[...]` uses it as a sieve. A few companions:

**R**

```r
which(x > 3)    # the positions of the TRUEs
```

```
[1] 2 4 5
```

```r
sum(x > 3)      # counts them: TRUE is 1, FALSE is 0
```

```
[1] 3
```

```r
mean(x > 3)     # the proportion of TRUEs
```

```
[1] 0.6
```

**Python**

```python
np.where(x > 3)[0]   # the positions of the Trues
```

```
array([1, 3, 4])
```

```python
np.sum(x > 3)        # counts them: True is 1, False is 0
```

```
np.int64(3)
```

```python
np.mean(x > 3)       # the proportion of Trues
```

```
np.float64(0.6)
```

**Note for Python:** `np.where(condition)` returns a *tuple* of index arrays (one per dimension); the `[0]` extracts the one we want. And of course the positions are 0-based where R's are 1-based.

That last trick (the `mean()` of a logical vector is a proportion) is one of the most-used idioms in statistical computing. Here it is verifying the "68–95" rule for the normal distribution on simulated data (we set the seed so the "random" numbers are reproducible):

**R**

```r
set.seed(1234)
z <- rnorm(10000)        # 10,000 standard normal draws
mean(abs(z) < 1)         # theory says about 0.68
```

```
[1] 0.6896
```

```r
mean(abs(z) < 2)         # theory says about 0.95
```

```
[1] 0.9577
```

**Python**

```python
rng = np.random.default_rng(1234)
z = rng.standard_normal(10000)   # 10,000 standard normal draws
np.mean(np.abs(z) < 1)           # theory says about 0.68
```

```
np.float64(0.6795)
```

```python
np.mean(np.abs(z) < 2)           # theory says about 0.95
```

```
np.float64(0.9554)
```

**A note on seeds in two languages.** `set.seed(1234)` and `rng = np.random.default_rng(1234)` each make their own language reproducible, but R and numpy use *different* generators: the two columns above print different numbers, and always will. Never expect the R and Python streams to match: expect each to match *itself* on every re-run, and both to agree with the theory.

Logical indexing also works on the left of an assignment, replacing just the selected elements:

**R**

```r
x <- c(2, 7, 1, 9, 4)
x[x > 5] <- 0     # censor the large values
x
```

```
[1] 2 0 1 0 4
```

**Python**

```python
x = np.array([2, 7, 1, 9, 4])
x[x > 5] = 0      # censor the large values
x
```

```
array([2, 0, 1, 0, 4])
```

> Exercise 2.3: A century of leap years
>
> Take your leap-year expression from Exercise 2.2 and apply it to the vector `years <- 1900:2025` (it is vectorised for free; that's the beauty of it). (Python: `years = np.arange(1900, 2026)`, and remember the parentheses around each comparison.)

a. Extract the leap years with logical indexing.
b. How many are there? What proportion of all years is that?
c. Which century years in the range (divisible by 100) are leap years anyway?

## 2.4 Conditional statements

Logic becomes *control flow* with `if`: run a block of code only when a condition holds.

**R**

```r
x <- 10
if (x > 0) {
  print("x is positive")
}
```

```
[1] "x is positive"
```

**Python**

```python
x = 10
if x > 0:
    print("x is positive")
```

```
x is positive
```

In R the condition goes in `( )` and the code to run goes in `{ }`; in Python there are no parentheses or braces: a colon `:` ends the condition, and the body is delimited by *indentation* (consistent, four spaces by convention), which in Python is syntax, not cosmetics. In both languages the condition must evaluate to a **single** `TRUE` or `FALSE`. Add alternatives with `else if` (`elif` in Python) and a catch-all `else`: the *first* branch whose condition is `TRUE` runs, and the rest are skipped:

**R**

```r
x <- -3
if (x < 0) {
  print("x is negative")
} else if (x == 0) {
  print("x is zero")
```

```r
} else {
  print("x is positive")
}
```

```
[1] "x is negative"
```

**Python**

```python
x = -3
if x < 0:
    print("x is negative")
elif x == 0:
    print("x is zero")
else:
    print("x is positive")
```

```
x is negative
```

A classic example is *FizzBuzz*, once allegedly a job-interview filter (today sadly it's all AI… but some companies would still bring you in person for an interview, so you never know): print "Fizz" if a number is divisible by 3, "Buzz" if by 5, "FizzBuzz" if by both, and the number otherwise. Order matters: the both-case must come first, or it can never be reached:

**R**

```r
n <- 15
if (n %% 3 == 0 && n %% 5 == 0) {
  print("FizzBuzz")
} else if (n %% 3 == 0) {
  print("Fizz")
} else if (n %% 5 == 0) {
  print("Buzz")
} else {
  print(n)
}
```

```
[1] "FizzBuzz"
```

**Python**

```python
n = 15
if n % 3 == 0 and n % 5 == 0:
    print("FizzBuzz")
elif n % 3 == 0:
    print("Fizz")
elif n % 5 == 0:
    print("Buzz")
else:
    print(n)
```

```
FizzBuzz
```

Because `if` needs a single logical, feeding it a vector is an error:

**R**

```r
x <- c(1, 4, 9)
if (x > 2) print("big")
```

```
Error in `if (x > 2) ...`:
! the condition has length > 1
```

**Python**

```python
x = np.array([1, 4, 9])
if x > 2: print("big")
```

```
ValueError: The truth value of an array with more than one element is ambiguous. Use a.any()
```

Both languages refuse, each in its own words: R demands a condition of length one; numpy declares the truth value of an array *ambiguous* and points you to `.any()` / `.all()`.

### 2.4.1 Vectorised choice: `ifelse()` / `np.where()`

When you want an elementwise either/or across a whole vector, use `ifelse()` in R: it takes a logical vector and picks from two alternatives, per element, in one line. Python has no vectorised `ifelse()` of its own; `np.where(condition, yes, no)` is the numpy idiom that fills the role:

**R**

```r
marks <- c(35, 62, 58, 71, 24)
ifelse(marks >= 40, "pass", "fail")
```

```
[1] "fail" "pass" "pass" "pass" "fail"
```

**Python**

```python
marks = np.array([35, 62, 58, 71, 24])
np.where(marks >= 40, "pass", "fail")
```

```
array(['fail', 'pass', 'pass', 'pass', 'fail'], dtype='<U4')
```

Exercise 2.4: The p-value verdict machine

a. Store a p-value in `p` and write an `if` / `else if` / `else` chain (Python: `elif`) that prints `"significant at the 1% level"` if `p < 0.01`, `"significant at the 5% level"` if `p < 0.05`, and `"not significant"` otherwise. Test it on `p = 0.003`, `p = 0.02` and `p = 0.7`.

b. Now do it for the whole vector `p_values <- c(0.001, 0.02, 0.049, 0.051, 0.2)` in a single line, by nesting one `ifelse()` inside another (in Python, one `np.where()` inside another).

## 2.5 `for` loops

A `for` loop repeats a block of code once for each element of a sequence:

**R**

```r
for (i in 1:5) {
  print(i^2)
}
```

```
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
```

**Python**

```python
for i in range(1, 6):
    print(i**2)
```

```
1
4
9
16
25
```

The loop variable `i` takes each value in turn, and the block runs once per value. **Note for Python:** `range(1, 6)` produces 1, 2, 3, 4, 5; the stop value is *excluded*, the mirror image of R's inclusive `1:5`. Forgetting this off-by-one is the classic porting bug in both directions. You can loop over any vector, not just integers (in Python, over any *iterable*: lists, arrays, strings, `range`s):

**R**

```r
for (day in c("Mon", "Tue", "Wed")) {
  print(day)
}
```

```
[1] "Mon"
[1] "Tue"
[1] "Wed"
```

**Python**

```python
for day in ["Mon", "Tue", "Wed"]:
    print(day)
```

```
Mon
Tue
Wed
```

When you need positions rather than values, prefer `seq_along(x)` to `1:length(x)`. They agree on non-empty vectors, but on an empty vector `1:length(x)` is `1:0` (the sequence `c(1, 0)`), so the loop runs twice on nothing at all, usually breaking your code (I have seen this over and over in my career). `seq_along` produces an empty sequence and the loop correctly runs zero times.

**Note for Python:** the positions idiom is `range(len(x))`, the direct companion of `seq_along(x)`, just 0-based, and it has no empty-vector trap: `range(0)` is empty, so the loop correctly runs zero times. Since the positions start at 0, anything arithmetic done with them (like the divide-by-position below) needs an `i + 1`.

**R**

```r
x <- c(10, 20, 30)
for (i in seq_along(x)) {
  print(x[i] / i)
}
```

```
[1] 10
[1] 10
[1] 10
```

**Python**

```python
x = np.array([10, 20, 30])
for i in range(len(x)):
    print(x[i] / (i + 1))   # i runs 0, 1, 2, hence the + 1
```

```
10.0
10.0
10.0
```

Exercise 2.5: FizzBuzz, at last, in bulk

a. Wrap the FizzBuzz `if` chain from earlier in a `for` loop that plays the game for every number from 1 to 30.
b. Harder: reproduce the output *without* a loop. Create `x <- as.character(1:30)` (Python: `x = np.arange(1, 31).astype(str)`) and use logical indexing to overwrite the right entries with `"Fizz"`, `"Buzz"` and `"FizzBuzz"`. Think about the order of the replacements.

### 2.5.1 Loops with state

If everything a loop does could be done elementwise, we wouldn't need loops (more on this soon). Loops earn their keep when there is **state**: when step $i$ depends on the result of step $i-1$. The classic example is the Fibonacci sequence,

$$r_i = r_{i-1} + r_{i-2}, \qquad r_1 = r_2 = 1,$$

where each element is the sum of the previous two: no way to compute element 20 without computing 19 and 18 first. With a loop it is immediate: we create a vector, then fill it in order, using earlier entries to build later ones.

**R**

```r
n <- 20
fib <- numeric(n)   # a vector of n zeros, to be filled
fib[1] <- 1
fib[2] <- 1
for (i in 3:n) {
  fib[i] <- fib[i - 1] + fib[i - 2]
}
fib
```

```
 [1]    1    1    2    3    5    8   13   21   34   55   89  144  233  377  610
[16]  987 1597 2584 4181 6765
```

**Python**

```python
n = 20
fib = np.zeros(n)   # a vector of n zeros, to be filled
fib[0] = 1
fib[1] = 1
for i in range(2, n):
    fib[i] = fib[i - 1] + fib[i - 2]
fib
```

```
array([1.000e+00, 1.000e+00, 2.000e+00, 3.000e+00, 5.000e+00, 8.000e+00,
       1.300e+01, 2.100e+01, 3.400e+01, 5.500e+01, 8.900e+01, 1.440e+02,
       2.330e+02, 3.770e+02, 6.100e+02, 9.870e+02, 1.597e+03, 2.584e+03,
       4.181e+03, 6.765e+03])
```

If a loop confuses you, the universal debugging move is to put `print()` statements inside it (print `i`, print `fib[i]`) and watch the state evolve iteration by iteration.

### 2.5.2 Grow or preallocate?

In the code above we *preallocated*: `numeric(n)` (`np.zeros(n)`) created the full-length vector up front, and the loop only filled slots. The tempting alternative is to start empty and grow with `c()` (`np.append()`):

**R**

```r
squares <- c()
for (i in 1:10000) {
  squares <- c(squares, i^2)   # don't do this
}
```

**Python**

```python
squares = np.array([])
for i in range(1, 10001):
    squares = np.append(squares, i**2)   # don't do this
```

This works, but every `c()` call (and every `np.append()` call) builds a brand-new copy of the whole vector, so filling $n$ slots costs on the order of $n^2$ copying. Let's measure the difference: in R with the `microbenchmark` package, in Python with the standard-library `timeit` module. Both run each expression several times, so we judge by the distribution of timings rather than one noisy measurement:

**R**

```r
library(microbenchmark)
n <- 10000
microbenchmark(
  grow = {
    squares <- c()
    for (i in 1:n) squares <- c(squares, i^2)
  },
  preallocate = {
    squares <- numeric(n)
    for (i in 1:n) squares[i] <- i^2
  },
  times = 10
)
```

```
Unit: milliseconds
        expr       min       lq      mean    median        uq        max neval
        grow 73.664147 74.45107 87.919412 79.287671 81.292260 177.193853    10
 preallocate  1.071856  1.17008  1.430223  1.456823  1.644195   1.943397    10
```

**Python**

```
import timeit
n = 10000

def grow():
    squares = np.array([])
    for i in range(1, n + 1):
        squares = np.append(squares, i**2)

def preallocate():
    squares = np.zeros(n)
    for i in range(1, n + 1):
        squares[i - 1] = i**2

# best of 10 runs of each, in milliseconds
min(timeit.repeat(grow, number=1, repeat=10)) * 1000
```

```
19.12984000000506
```

```
min(timeit.repeat(preallocate, number=1, repeat=10)) * 1000
```

```
0.6584929997188738
```

**Note for Python:** `timeit` wants a *function* to call, hence the two `def`s. A full treatment of writing functions is Chapter 3's business; for now, `def name(): ...` simply packages the block. `timeit.repeat(f, number=1, repeat=10)` returns ten timings in seconds; the minimum is the honest estimate, and we scale it to milliseconds.

Same result, dozens of times apart! Since the copying cost is quadratic, the gap widens as `n` grows. The habit to build, in either language: **decide the size first, allocate, then fill.**

**Note for Python:** the quadratic growth here belongs to *numpy arrays*. Plain Python lists over-allocate as they grow, so `my_list.append(...)` is genuinely cheap, and building a list then converting with `np.array(my_list)` is idiomatic Python. For numerical work, preallocating the numpy array as above remains the habit to build.

## 2.6 `while`, `break` and `next`

A `for` loop needs to know in advance how many times to run. A `while` loop keeps going *as long as a condition holds*, ideal when the number of iterations is itself the unknown.

**R**

```r
i <- 1
while (i <= 5) {
  print(i)
  i <- i + 1
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

**Python**

```python
i = 1
while i <= 5:
    print(i)
    i = i + 1   # i += 1 is the common shorthand
```

```
1
2
3
4
5
```

The line `i <- i + 1` (Python: `i = i + 1`) is load-bearing: forget to update the state and the condition never becomes `FALSE`, so the loop runs forever. (If that happens, the Stop button or `Esc` in RStudio, or `Ctrl`+`C` at a console, is your friend. We've all been there.)

A more statistical example: how many draws from a standard normal until we see one above 2? The answer is random, so a `for` loop can't know its own length; a `while` loop is the natural fit.

**R**

```r
set.seed(1)
attempts <- 0
z <- 0
while (z <= 2) {
  z <- rnorm(1)          # one fresh draw
  attempts <- attempts + 1
}
attempts
```

```
[1] 61
```

### Python

```python
rng = np.random.default_rng(1)
attempts = 0
z = 0
while z <= 2:
    z = rng.standard_normal()   # one fresh draw
    attempts = attempts + 1
attempts
```

```
31
```

(Theory check: $P(Z > 2) \approx 0.023$, so on average we expect about $1/0.023 \approx 44$ attempts. The two columns disagree (61 draws in the R run, 31 in the Python run) because the two seeded streams differ; both counts are perfectly ordinary draws from the same geometric distribution with mean about 44.)

Two keywords fine-tune any loop: `next` (`continue` in Python) skips the rest of the current iteration, and `break` (the same word in both languages) exits the loop immediately.

### R

```r
for (i in 1:10) {
  if (i == 5) next    # skip 5
  if (i > 8)  break   # stop after 8
  print(i)
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 6
[1] 7
[1] 8
```

**Python**

```python
for i in range(1, 11):
    if i == 5: continue   # skip 5
    if i > 8:  break      # stop after 8
    print(i)
```

```
1
2
3
4
6
7
8
```

Use them sparingly: a loop that jumps around is harder to reason about, and often a clearer `while` condition can express the same logic.

> Exercise 2.6: Double your money
>
> You invest £1000 at 5% interest per year, compounded annually.
>
> a. Use a `while` loop to find how many whole years it takes for the balance to at least double.
> b. Sanity-check your answer against the closed form $\log(2)/\log(1.05)$.
> c. Now suppose you also add £500 at the end of each year. How many years to reach £10,000? (No closed form now; this is exactly the kind of question loops are for.)

## 2.7 When not to loop

Time for a confession: much of what beginners write loops for, R and numpy already do better with the vectorised operations of Chapter 1. Squaring a million numbers needs no loop:

`x^2` (`x**2`) *is* the loop, executed in fast compiled code rather than element-by-element in the interpreter:

**R**

```
x <- 1:1000000
microbenchmark(
  loop = {
    out <- numeric(length(x))
    for (i in seq_along(x)) out[i] <- x[i]^2
  },
  vectorised = x^2,
  times = 10
)
```

```
Unit: microseconds
       expr       min       lq     mean    median        uq       max neval
       loop 35437.747 36431.39 37282.14 36910.115 38462.377  39746.52    10
 vectorised   877.223   895.13 12497.98  2644.579  3413.924 103789.87    10
```

**Python**

```
x = np.arange(1, 1000001)

def loop():
    out = np.zeros(len(x))
    for i in range(len(x)):
        out[i] = x[i]**2

def vectorised():
    x**2

# best of 10 runs of each, in milliseconds
min(timeit.repeat(loop, number=1, repeat=10)) * 1000
```

```
155.77788499967937
```

```
min(timeit.repeat(vectorised, number=1, repeat=10)) * 1000
```

```
0.6596240000362741
```

The vectorised version is tens of times faster in the R run, and *hundreds* of times in the Python one, where every trip around an interpreted loop (plus the numpy machinery invoked on each single element) costs more. And that is against a *well-written*, preallocated loop; against a sloppy one the gap stretches further still, as Exercise 2.7 will show. The same holds for `sum(x)` versus an accumulator loop, `sqrt(x)`, logical indexing versus a loop of `if`s, and so on.

So, when do we actually loop? Ask one question: **does element $i$ of my output depend on element $i-1$?** If no (the computations are independent), think vectors first. If yes (there is state, as in Fibonacci, compound interest, or the simulation of any process that evolves over time), a loop is the honest tool, and with preallocation it will be perfectly fast.

Exercise 2.7: Vectorise this

A colleague sends you this code for standardising a vector (subtract the mean, divide by the standard deviation):

**R**

```r
set.seed(1)
x <- rnorm(10000)
z <- c()
for (i in 1:length(x)) {
  z <- c(z, (x[i] - mean(x)) / sd(x))
}
```

**Python**

```python
rng = np.random.default_rng(1)
x = rng.standard_normal(10000)
z = np.array([])
for i in range(len(x)):
    # ddof=1: R's sd() divides by n - 1
    z = np.append(z, (x[i] - np.mean(x)) / np.std(x, ddof=1))
```

a. It is slow for three separate reasons. Find all three.
b. Rewrite it as one vectorised line.
c. Use `microbenchmark(..., times = 5)` (Python: `timeit.repeat(..., repeat=5)`) to compare the two versions, and `all.equal()` (`np.allclose()`) to confirm they agree. (Why not `==`?)

## 2.8 A first taste of gradient descent

Time to put the toolbox to work. A huge share of statistics reduces to one task: **minimise a function**. Least squares minimises a sum of squared errors; maximum likelihood minimises a negative log-likelihood. For a few lucky models calculus hands us a formula, but for most modern ones there is no closed form, so we minimise *numerically*, with an algorithm. The simplest one worth knowing is gradient descent, and we can already build it with nothing but this week's loops.

Here is the simplest statistical loss there is. Given observations $y_1, \dots, y_n$, which single number $m$ summarises them best? Pick the one minimising the **quadratic loss**

$$L(m) = \frac{1}{n} \sum_{i=1}^{n} (y_i - m)^2.$$

(You may already know the answer is the sample mean $\bar{y}$: perfect, that gives us a way to check whether our algorithm works.)

### 2.8.1 The idea: follow the slope downhill

Imagine standing on the curve of $L$ at some guess $m$, in fog, unable to see where the minimum is. You can still feel the ground through your feet…



Fortunately, the derivative

$$L'(m) = -\frac{2}{n}\sum_{i=1}^{n}(y_i - m)$$

tells you the local slope. If $L'(m) < 0$ the function is decreasing, so the minimum lies to the *right*: step right. If $L'(m) > 0$, step left. Both cases are captured by one update: move *against* the slope:

$$m_{k+1} = m_k - \gamma\, L'(m_k),$$

where $\gamma > 0$ (the **step size**, or *learning rate*) scales how far we move. Notice the magnitude is used too: where the function is steep we take big strides; as the slope flattens near the minimum, the steps shrink on their own. Repeat until the steps become negligible: that is the whole algorithm. Here is the idea in a single picture, run for a few iterations on the loss we are about to minimise:

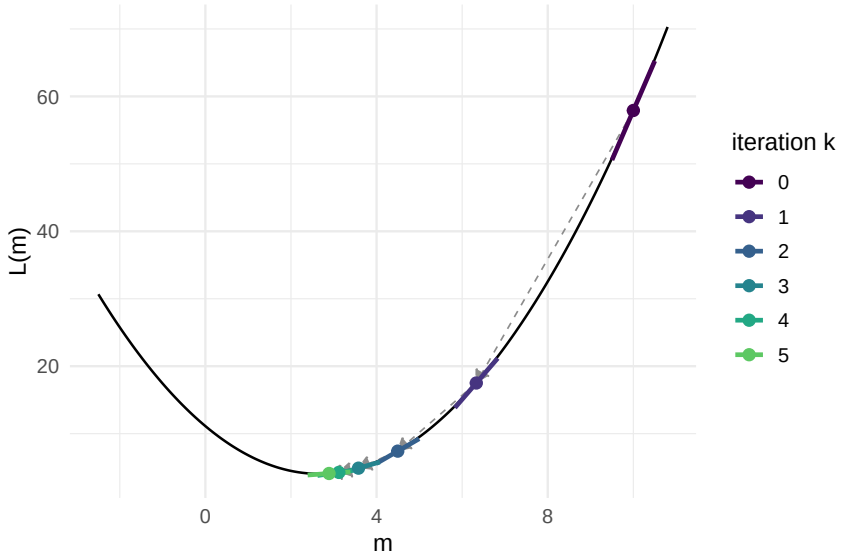



Figure 2.1: The slope idea on the quadratic loss $L(m)$, for the data of the R run (step size $\gamma = 0.25$, chosen to make a clean picture). Each coloured point is an iterate $m_k$, and the short segment through it shows the local slope there: the slope points uphill, so the update steps the other way (dashed arrows), and as the slope flattens towards the minimum the steps shrink on their own.

### 2.8.2 The algorithm in code

This is a computation with state ($m_{k+1}$ depends on $m_k$), so it's a job for a loop. We store the whole trajectory in a preallocated vector so we can inspect it afterwards, and we write the derivative *inline* as `-2 * mean(y - m)` (numpy: `-2 * np.mean(y - m)`):

**R**

```r
set.seed(7432)
y <- rnorm(200, mean = 3, sd = 2)   # our data

gamma <- 0.1      # step size
n_iter <- 60      # number of iterations
m_path <- numeric(n_iter)
m_path[1] <- 10   # a deliberately terrible starting guess

for (k in 2:n_iter) {
  slope <- -2 * mean(y - m_path[k - 1])          # L'(m) at the current point
  m_path[k] <- m_path[k - 1] - gamma * slope     # the descent step
}

m_path[n_iter]   # where we ended up
```

```
[1] 2.660242
```

```r
mean(y)          # where we should be
```

```
[1] 2.660228
```

**Python**

```python
rng = np.random.default_rng(7432)
y = rng.normal(loc=3, scale=2, size=200)   # our data

gamma = 0.1       # step size
n_iter = 60       # number of iterations
m_path = np.zeros(n_iter)
m_path[0] = 10    # a deliberately terrible starting guess

for k in range(1, n_iter):
    slope = -2 * np.mean(y - m_path[k - 1])      # L'(m) at the current point
```

```
    m_path[k] = m_path[k - 1] - gamma * slope    # the descent step

m_path[n_iter - 1]   # where we ended up
```

```
np.float64(2.929473307049999)
```

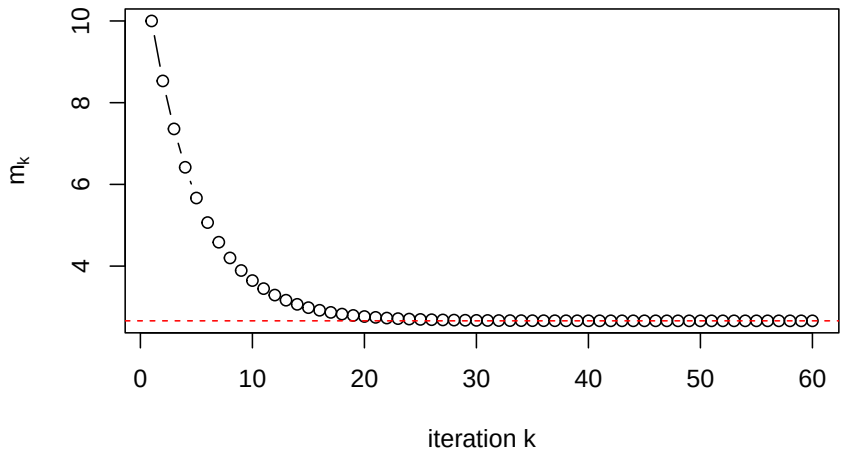
```
np.mean(y)           # where we should be
```

```
np.float64(2.929459762585767)
```

It works: sixty iterations of "step against the slope" land on the sample mean to several decimals. (The two columns simulate *different* datasets, because the streams never match, so they print different numbers: each lands on its own sample mean, 2.66 in the R run and 2.93 in the Python run, both consistent with the true mean of 3 given $n = 200$.) A quick plot of the trajectory (base `plot()` in R, `matplotlib` in Python) shows the approach: big early strides, then ever-finer adjustments (quick-and-dirty plots for now; we do plotting properly in Chapter 4):

**R**

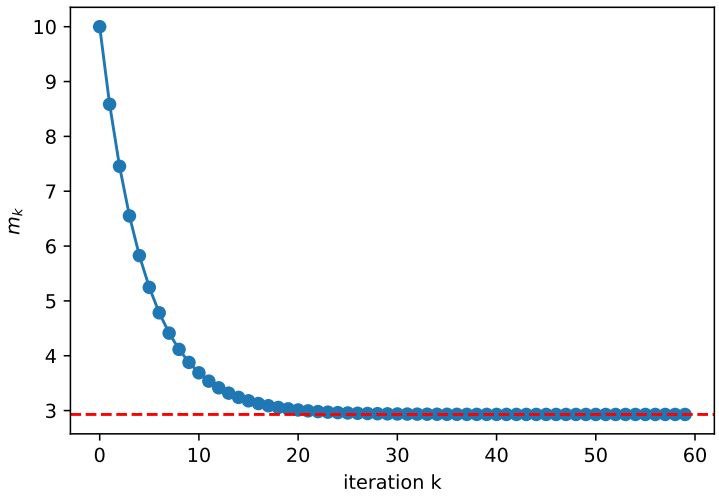
```
plot(m_path, type = "b", xlab = "iteration k", ylab = expression(m[k]))
abline(h = mean(y), lty = 2, col = "red")   # the target
```




**Python**

```
import matplotlib.pyplot as plt

plt.plot(m_path, "o-")                                  # "o-": points joined by lines
plt.axhline(np.mean(y), linestyle="--", color="red")    # the target
plt.xlabel("iteration k")
plt.ylabel("$m_k$")
plt.show()
```




### 2.8.3 The step size is a dial, not a detail

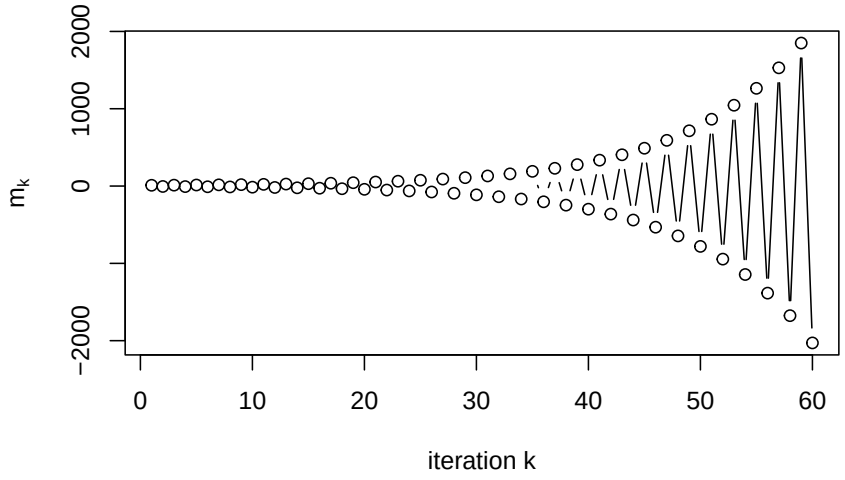
Everything above used $\gamma = 0.1$. That choice matters more than it looks:

- **Too small** ($\gamma = 0.005$, say): every step is a timid shuffle. The algorithm still converges, but might need thousands of iterations instead of sixty.
- **Too large**: each step overshoots the minimum and lands on the far side, further up than where it started. The iterates bounce from side to side with growing amplitude: the algorithm *diverges*:

**R**

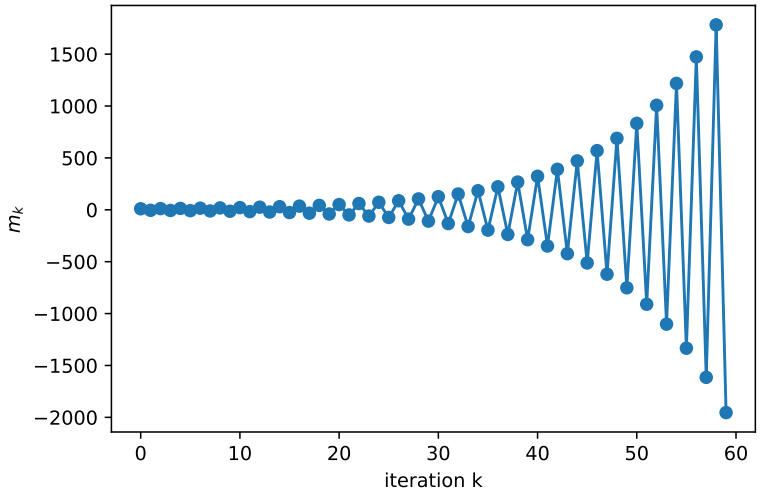
```
gamma <- 1.05                       # too ambitious
m_path <- numeric(n_iter)
m_path[1] <- 10
for (k in 2:n_iter) {
  slope <- -2 * mean(y - m_path[k - 1])
```

```r
  m_path[k] <- m_path[k - 1] - gamma * slope
}
plot(m_path, type = "b", xlab = "iteration k", ylab = expression(m[k]))
```




### Python

```python
gamma = 1.05                          # too ambitious
m_path = np.zeros(n_iter)
m_path[0] = 10
for k in range(1, n_iter):
    slope = -2 * np.mean(y - m_path[k - 1])
    m_path[k] = m_path[k - 1] - gamma * slope
plt.plot(m_path, "o-")
plt.xlabel("iteration k")
plt.ylabel("$m_k$")
plt.show()
```




Every quantity in that plot is further from the answer than the terrible initial guess. Choosing $\gamma$ well (and adapting it on the fly) is a central theme of Chapter 5; for now, Exercise 2.8 lets you map out the dial yourself.

### 2.8.4 Beyond the quadratic

Nothing in the recipe used the fact that $L$ was a quadratic, only that we can evaluate a derivative. Take the cubic

$$g(x) = x^3 + x^2 - x - 1, \qquad g'(x) = 3x^2 + 2x - 1,$$

which has a local minimum at $x = 1/3$ (and a local maximum at $x = -1$). Same loop, new inline derivative:

**R**

```
gamma <- 0.05
n_iter <- 60
x_path <- numeric(n_iter)
x_path[1] <- 1.5

for (k in 2:n_iter) {
  slope <- 3 * x_path[k - 1]^2 + 2 * x_path[k - 1] - 1   # g'(x), inline
  x_path[k] <- x_path[k - 1] - gamma * slope
}
```

```
x_path[n_iter]   # should be near 1/3
```

```
[1] 0.3333343
```

**Python**

```
gamma = 0.05
n_iter = 60
x_path = np.zeros(n_iter)
x_path[0] = 1.5

for k in range(1, n_iter):
    slope = 3 * x_path[k - 1]**2 + 2 * x_path[k - 1] - 1   # g'(x), inline
    x_path[k] = x_path[k - 1] - gamma * slope

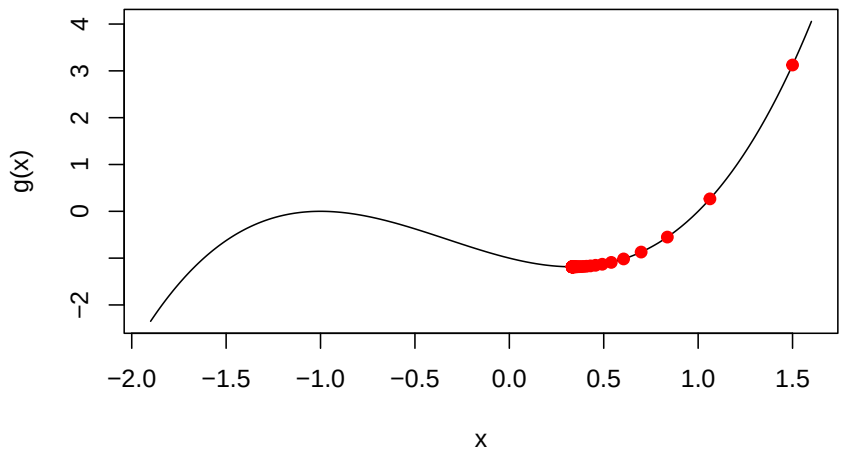
x_path[n_iter - 1]   # should be near 1/3
```

```
np.float64(0.3333343409933769)
```

(No randomness in this one: for once, the two columns agree to the last digit.) Plotting the iterates on the function itself shows the descent sliding into the valley and piling up at the minimum:

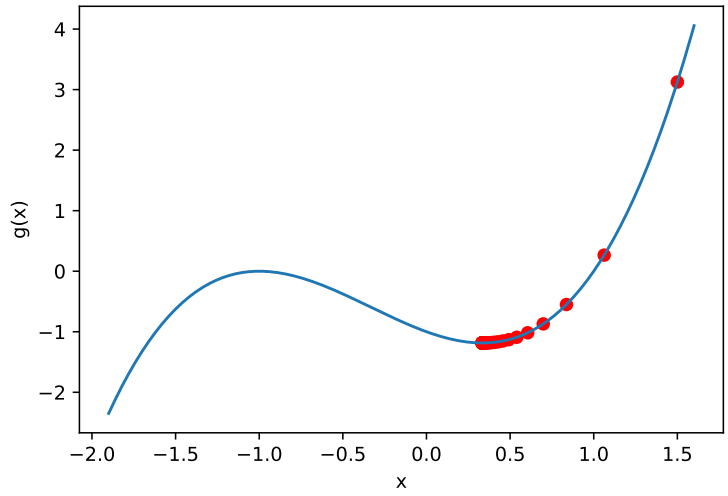
**R**

```
xs <- seq(-1.9, 1.6, by = 0.01)
plot(xs, xs^3 + xs^2 - xs - 1, type = "l",
     xlab = "x", ylab = "g(x)")
points(x_path, x_path^3 + x_path^2 - x_path - 1, pch = 19, col = "red")
```



**Python**

```
xs = np.arange(-1.9, 1.61, 0.01)   # arange excludes the endpoint, hence 1.61
plt.plot(xs, xs**3 + xs**2 - xs - 1)
plt.scatter(x_path, x_path**3 + x_path**2 - x_path - 1, color="red")
plt.xlabel("x")
plt.ylabel("g(x)")
plt.show()
```




One honest caveat: gradient descent only follows the slope it can feel, so it finds *local* minima. Start this example at $x_1 = -2$: downhill now points left, and since $g \to -\infty$ there, the iterates march off to $-\infty$ without ever looking back. Try it.

You may have noticed we copy-pasted the same loop three times, editing the derivative by hand each time. That itch has a cure: next week (Chapter 3) we wrap gradient descent into a reusable *function*, and by Chapter 5 it will be fitting real statistical models.

Exercise 2.8: Turning the step-size dial

Using the quadratic-loss setup (`set.seed(7432); y <- rnorm(200, mean = 3, sd = 2)`), run gradient descent from `m = 10` for 30 iterations, for **every** step size in `gammas <- c(0.005, 0.05, 0.2, 0.5, 0.8, 1.02)`. Store the trajectories in a matrix `paths` with one column per `gamma` (preallocate with `matrix(NA, nrow = 30, ncol = length(gammas))` and fill it with a nested loop). (Python: seed with `rng = np.random.default_rng(7432)`, simulate `y = rng.normal(3, 2, 200)`, preallocate with `np.full((30, len(gammas)), np.nan)`.)

a. Plot all trajectories at once with `matplot(paths, type = "l", lty = 1)` (Python: `plt.plot(paths)` draws one line per column, just like `matplot`). If one trajectory squashes the rest flat, plot it separately and re-plot the others without it.
b. Classify each `gamma`: smooth convergence, oscillating convergence, or divergence?
c. Something special happens at `gamma = 0.5`. What, and (looking at the update formula with $L'(m) = -2(\bar{y} - m)$) why?

## 2.9 Challenges

Two optional problems.

**!** Challenge 2.1: Gradient descent that knows when to stop

Running a fixed 60 iterations is crude: wasteful when converged by iteration 10, insufficient if convergence needs 200. Rebuild the quadratic-loss gradient descent with a `while` loop that stops when the step just taken is smaller than `tol = 1e-8` in absolute value, but add a safety cap of `max_iter = 5000` iterations so a diverging run cannot loop forever. Report the final `m` and the number of iterations used, for `gamma = 0.1`, `0.01` and `1.05`. For the last one, how does your code know it failed?

**!** Challenge 2.2: The Collatz conjecture

The Collatz conjecture: take a positive integer $n$; if even, halve it; if odd, replace it by $3n+1$. Repeat. The (unproven!) conjecture is that every starting value eventually reaches 1.
By Dkobak - Own work, CC0, Link

a. With a `while` loop, count the steps needed to reach 1 from $n = 27$. (You should get 111.)
b. Wrap that in a `for` loop over every starting value from 1 to 5000, storing the step counts in a preallocated vector. Which start takes longest, and how many steps does it need? A one-line plot of counts against starting value is worth a look.

That is the week: comparisons and logic to make decisions, loops to repeat them, and, stitched together, a first real algorithm. Next week (Chapter 3) we stop copy-pasting the gradient-descent loop and wrap it into a *function* we can hand any problem we like.

# 3 Week 3: Functions and Functional Programming

Last week ended with us descending a hill by repeating `x <- x - gamma * slope` (`x = x - gamma * slope` in Python) inside a `for` loop, with the derivative hard-coded inline (Section 2.8). It worked, but every new function to minimise meant rewriting the loop. This week we fix that properly. First we learn to write our own functions, and collect the repetition tool that had to wait for them: *recursion*, a function calling itself. Then we go one step further and treat functions themselves as data: passing them around, building them on the fly, mapping them over collections. By the end of the chapter our gradient descent will take *any* function you throw at it, which is exactly the tool Chapter 5 is built on.

## 3.1 Writing your own functions

We have been using functions since minute one: `sqrt()`, `length()`, `mean()`, `seq()`; or, on the Python side, `np.sqrt()`, `len()`, `np.mean()`, `np.arange()`. Each takes an input, does its job, and hands back an output. The real power move is writing your own. The rule of thumb: if you find yourself copy-pasting the same code a third time, stop and write a function. In coding, laziness done right is a virtue.

Let's write a function that converts pounds to grams, using $g = 453.5924 \cdot \text{lbs}$:

**R**

```r
lbs_to_grams <- function(lbs) {
  grams <- 453.5924 * lbs
  return(grams)
}
```

**Python**

```python
def lbs_to_grams(lbs):
    grams = 453.5924 * lbs
    return grams
```

Anatomy of the R snippet:

- `function(lbs)` creates a new function whose argument is called `lbs`;
- the body between `{ }` is the code that runs on every call;
- `return(grams)` says what comes out;
- the whole thing is assigned to the name `lbs_to_grams` with `<-`, exactly like any other variable. Keep that thought: it becomes important later in the chapter.

**Note for Python:** the anatomy differs in the details. `def lbs_to_grams(lbs):` both *names* the function and lists its arguments in one go, with no separate assignment step; the body is marked by indentation rather than `{ }` (four spaces, consistently); and `return grams` needs no parentheses.

Calling it works as you would expect:

**R**

```
lbs_to_grams(2)
```

```
[1] 907.1848
```

```
lbs_to_grams(lbs = 3)   # naming the argument is optional here
```

```
[1] 1360.777
```

**Python**

```
lbs_to_grams(2)
```

```
907.1848
```

```
lbs_to_grams(lbs=3)   # naming the argument is optional here
```

```
1360.7772
```

And because the body is just vectorised arithmetic (Chapter 1), it happily takes a whole vector:

**R**

```
lbs_to_grams(c(1, 2, 3))
```

```
[1]  453.5924  907.1848 1360.7772
```

**Python**

```
# importing numpy (the hidden setup did too; repeated imports are free)
import numpy as np

lbs_to_grams(np.array([1, 2, 3]))
```

```
array([ 453.5924,  907.1848, 1360.7772])
```

**Note for Python:** hand it a numpy array, not a plain list: `453.5924 * [1, 2, 3]` is an error (and `3 * [1, 2]` would *repeat* the list, not multiply it).

A few naming guidelines: lowercase, words separated by `_`, concise but meaningful, and avoid clobbering existing names like `mean` or `c` in R, or `len`, `sum` and `list` in Python (both languages will let you; your future self will not forgive you).

### 3.1.1 Arguments and defaults

Functions can take several arguments. To convert pounds to millilitres we also need the specific mass $\rho$ of the substance, since $ml = g/\rho$:

**R**

```
lbs_to_ml <- function(lbs, specific_mass) {
  ml <- lbs_to_grams(lbs) / specific_mass
  return(ml)
}

lbs_to_ml(2, 1)      # water
```

```
[1] 907.1848
```

```
lbs_to_ml(2, 0.92)   # oil
```

```
[1] 986.0704
```

**Python**

```
def lbs_to_ml(lbs, specific_mass):
    ml = lbs_to_grams(lbs) / specific_mass
    return ml


lbs_to_ml(2, 1)      # water
```

```
907.1848
```

```
lbs_to_ml(2, 0.92)   # oil
```

```
986.0704347826087
```

Note that `lbs_to_ml` calls the function we wrote earlier: functions building on functions is the normal way programs grow.

If 90% of our conversions are for water, typing `1` every time gets old. We can make the argument optional by giving it a **default value**:

**R**

```
lbs_to_ml <- function(lbs, specific_mass = 1) {
  ml <- lbs_to_grams(lbs) / specific_mass
  return(ml)
}

lbs_to_ml(2)                        # default: water
```

```
[1] 907.1848
```

```
lbs_to_ml(2, 0.92)                  # positional
```

```
[1] 986.0704
```

```
lbs_to_ml(2, specific_mass = 0.92)  # named -- same thing, more readable
```

```
[1] 986.0704
```

**Python**

```python
def lbs_to_ml(lbs, specific_mass=1):
    ml = lbs_to_grams(lbs) / specific_mass
    return ml


lbs_to_ml(2)                          # default: water
```

```
907.1848
```

```python
lbs_to_ml(2, 0.92)                    # positional
```

```
986.0704347826087
```

```python
lbs_to_ml(2, specific_mass=0.92)      # named -- same thing, more readable
```

```
986.0704347826087
```

Arguments are matched by position first, and by name when you spell the name out. With many arguments, naming the non-obvious ones makes code much easier to read: compare `rnorm(10, 5, 2)` with `rnorm(10, mean = 5, sd = 2)`. The same positional/keyword rules hold in Python: compare `rng.normal(5, 2, 10)` with `rng.normal(loc=5, scale=2, size=10)`.

**Note for Python:** default values are evaluated **once**, when the `def` statement runs, not afresh at each call as in R. With immutable defaults like `1` you will never notice the difference. With a mutable default you will: in `def add_entry(x, log=[])`, every call shares the *same* list, which quietly accumulates entries across calls. The standard idiom is a `None` default plus `if log is None: log = []` inside the body.

Exercise 3.1: Cooking for Remy

A small French rat with a strong culinary vision (title omitted for copyright reasons) has moved to the US, where recipes measure everything in *cups*. He has hired you to sort out the conversions.

a. Write `cups_to_ml(cups)` using $ml = 236.588 \cdot \text{cups}$.
b. Write `ml_to_grams(ml, specific_mass)` using $g = ml \cdot \rho$, where $\rho$ is the specific mass.
c. Write `cups_to_grams(cups, ingredient = "water")` which:
    i. converts cups to millilitres with `cups_to_ml`;
    ii. picks the specific mass with an `if`/`else if`/`else` chain (`elif` in Python) on

   `ingredient` (Chapter 2), using: water 1, flour 0.53, oil 0.92, oat milk 1.03;
   iii. returns the result of `ml_to_grams` with those values.

d. Test it on: 2 cups of water; half a cup of oil; 3 cups of oat milk; 2 cups of flour plus one cup of water.

Exercise 3.2: Standardise, with options

Write `standardise(x, center = TRUE, scale = TRUE)` which takes a numeric vector and:

- subtracts `mean(x)` (Python: `np.mean(x)`) if `center` is `TRUE`;
- divides by `sd(x)` (Python: `np.std(x, ddof=1)`) if `scale` is `TRUE`;
- returns the transformed vector.

Test all four combinations of `center` and `scale` on `x <- c(2, 4, 6, 8, 100)` (Python: `x = np.array([2, 4, 6, 8, 100])`). Then check the full standardisation with `mean(standardise(x))` and `sd(standardise(x))`. In the R run the mean will print as something like `1e-17` rather than exactly `0` (the numpy run may print an exact `0.0`: a lucky cancellation, not a guarantee). Recall the floating-point discussion in Chapter 2, and confirm with `all.equal(mean(standardise(x)), 0)` (Python: `np.isclose(...)`).

### 3.1.2 Return values

Three things worth knowing about what comes out of a function.

**Implicit return.** If R reaches the end of the body, the value of the *last expression* is returned automatically, no `return()` needed:

**R**

```
lbs_to_grams <- function(lbs) {
  453.5924 * lbs      # last expression: returned automatically
}
lbs_to_grams(2)
```

```
[1] 907.1848
```

**Python**

```python
def lbs_to_grams(lbs):
    return 453.5924 * lbs   # no implicit return in Python

lbs_to_grams(2)
```

```
907.1848
```

This is idiomatic R; most functions in the wild are written this way, and we will mostly drop `return()` from here on.

**Note for Python:** there is no implicit return. A function that reaches the end of its body without hitting `return` hands back `None`, silently. Keep writing `return`; the R habit of leaving the last expression bare does not travel.

**Early return.** `return()` (Python: `return`) exits the function *immediately*, which is handy for bailing out of bad inputs before they cause trouble:

**R**

```r
safe_log <- function(x) {
  if (any(x <= 0)) {
    return(NA)        # bail out early: log of a non-positive number
  }
  log(x)
}

safe_log(c(1, 10, 100))
```

```
[1] 0.000000 2.302585 4.605170
```

```r
safe_log(c(-1, 10, 100))
```

```
[1] NA
```

**Python**

```python
def safe_log(x):
    if np.any(x <= 0):
        return np.nan   # bail out early: log of a non-positive number
    return np.log(x)


safe_log(np.array([1, 10, 100]))
```

```
array([0.        , 2.30258509, 4.60517019])
```

```
safe_log(np.array([-1, 10, 100]))
```

```
nan
```

**Invisible return.** Occasionally a function should return a value without printing it when called at the console. Wrapping the result in `invisible()` does exactly that:

**R**

```
quiet_sqrt <- function(x) {
  invisible(sqrt(x))
}

quiet_sqrt(2)        # nothing printed...
y <- quiet_sqrt(2)   # ...but the value is really there
y
```

```
[1] 1.414214
```

**Python**

```
def quiet_sqrt(x):
    return np.sqrt(x)   # no invisible() here -- see the note

quiet_sqrt(2)        # this one DOES print in Python...
```

```
np.float64(1.4142135623730951)
```

```
y = quiet_sqrt(2)    # ...assignment never prints, in either language
y
```

```
np.float64(1.4142135623730951)
```

You have met this behaviour already: `library()` and most plotting functions return invisibly. That is all you need to know about it for now.

**Note for Python:** `invisible()` has no counterpart. At the console Python prints the value of every bare expression *unless it is* `None`, so the nearest thing to an invisible return is returning `None`, which is exactly what a function without a `return` does, but then there is no value to keep. Nothing to do here except know the difference.

## 3.2 Scoping: where variables live

Variables created inside a function are **local**: they live in a temporary environment created for that call, and vanish when the call ends.

**R**

```r
lbs_to_grams <- function(lbs) {
  grams <- 453.5924 * lbs
  grams
}

lbs_to_grams(2)
```

```
[1] 907.1848
```

```r
grams    # does not exist out here
```

```
Error:
! object 'grams' not found
```

**Python**

```python
def lbs_to_grams(lbs):
    grams = 453.5924 * lbs
    return grams


lbs_to_grams(2)
```

```
907.1848
```

```python
grams    # does not exist out here
```

```
NameError: name 'grams' is not defined
```

The reverse direction, however, *does* work: if R (or Python) cannot find a name inside the function, it looks in the environment where the function was *defined*: for a function typed at the top level of a script, that is the **global environment** (R's name; Python says *global scope*), where your script-level variables live. That sounds convenient. It is also a classic source of silent bugs. Watch:

**R**

```r
# an innocent global variable, defined at some point in a long script
lbs <- 3

# a typo: the argument is `lb`, but the body says `lbs`
bug_lbs_to_grams <- function(lb) {
  453.5924 * lbs
}

bug_lbs_to_grams(2)
```

```
[1] 1360.777
```

```r
bug_lbs_to_grams(100)   # same answer, whatever we feed it!
```

```
[1] 1360.777
```

**Python**

```python
# an innocent global variable, defined at some point in a long script
lbs = 3

# a typo: the argument is `lb`, but the body says `lbs`
def bug_lbs_to_grams(lb):
    return 453.5924 * lbs


bug_lbs_to_grams(2)
```

```
1360.7772
```

```python
bug_lbs_to_grams(100)   # same answer, whatever we feed it!
```

```
1360.7772
```

No error, no warning: R and Python alike quietly grabbed the global `lbs` and returned $453.5924 \times 3$ forever. The input is ignored completely. The moral is short and worth memorising: **everything a function needs should arrive through its arguments**. A function that only reads its arguments and only communicates through its return value is called *pure*, and pure functions are dramatically easier to test, reuse and debug; this idea is the backbone of the second half of this chapter.

**Note for Python:** the lookup rule has a name here: **LEGB** (Local, Enclosing, Global, Built-ins, searched in that order), which for our purposes is R's lexical scoping by another name. One genuine difference: *reading* a global from inside a function just works (as the bug shows), but *assigning* to one requires the `global` keyword: a plain `lbs = ...` inside a function always creates a local variable.

**Note for Python:** one more difference lives here, and it matters. R behaves *as if* every object passed to a function were copied: modify an argument inside a function and the caller's object is untouched (*copy-on-modify*). Python passes references instead: **rebinding** a name (`x = x + 1`) is local and safe, but **mutating** a mutable object in place (`x.append(3)`, or `x[0] = 99` on a list or numpy array) changes the caller's object too. A function that mutates its arguments is not pure; in Python, purity takes discipline that R's semantics give you for free.

**R**

```r
rm(lbs)   # tidy up: remove the leftover variable
```

**Python**

```python
del lbs   # tidy up: remove the leftover variable
```

> Exercise 3.3: Where does x live?
>
> Predict the output of each of the four final lines **before** running the code, then run it and check. For each one, explain which environment the value came from (or why there is an error).
> **R**

```r
x <- 1

f <- function() {
  x <- 10
  x * 2
}

g <- function() {
  x * 2
}

h <- function() {
  y <- x * 2
  y
}

f()
g()
x
y
```

**Python**

```python
x = 1

def f():
    x = 10
    return x * 2

def g():
    return x * 2

def h():
    y = x * 2
    return y

f()
g()
x
y
```

## 3.3 The `...` / `**kwargs` argument

Browse any R help page and you will meet a mysterious argument literally called `...` (pronounced "dot-dot-dot"); Python signatures spell the same idea `*args` / `**kwargs`. Both scoop up any extra arguments a caller supplies and let you pass them on to another function. One example is worth a paragraph of theory:

**R**

```r
summary_stats <- function(x, ...) {
  c(mean = mean(x, ...), sd = sd(x, ...))
}

heights <- c(1.62, 1.75, NA, 1.68)

summary_stats(heights)
```

```
mean   sd
  NA   NA
```

```r
summary_stats(heights, na.rm = TRUE)   # na.rm travels through ... to mean and sd
```

```
      mean         sd
1.68333333 0.06506407
```

**Python**

```python
def summary_stats(x, **kwargs):
    return {"mean": np.mean(x, **kwargs),
            "sd":   np.std(x, ddof=1, **kwargs)}

heights = np.array([1.62, 1.75, np.nan, 1.68])

summary_stats(heights)
```

```
{'mean': np.float64(nan), 'sd': np.float64(nan)}
```

```python
summary_stats(heights, where=~np.isnan(heights))   # `where` travels through **kwargs
```

```
{'mean': np.float64(1.6833333333333333), 'sd': np.float64(0.06506407098647707)}
```

We did not have to declare `na.rm` (or `where`, on the Python side) ourselves: whatever extras arrive get forwarded wholesale. Use it sparingly: it is wonderful for forwarding options, and terrible for hiding what a function actually depends on.

**Note for Python:** one star (`*args`) scoops extra *positional* arguments into a tuple, two stars (`**kwargs`) scoop extra *keyword* arguments into a dict, and `np.mean(x, **kwargs)` unpacks them again on the way out. (numpy has no `na.rm`; the `where` mask above is the closest kwarg-shaped idiom, and `np.nanmean`/`np.nanstd` exist for exactly this job.)

## 3.4 A brief look at recursion

Loops are not the only way to repeat. A **recursive** definition solves a problem in terms of smaller instances of itself, plus a *base case* to stop the descent. That is exactly how we first wrote Fibonacci mathematically in Section 2.5: $r_i = r_{i-1} + r_{i-2}$, with $r_1 = r_2 = 1$ as the base. Back then we had to translate the recurrence into a loop. Now that we can write functions, we can translate it *directly*, because a function is allowed to call itself:

**R**

```r
fib_rec <- function(n) {
  if (n <= 2) 1 else fib_rec(n - 1) + fib_rec(n - 2)
}

fib_rec(10)
```

```
[1] 55
```

**Python**

```python
def fib_rec(n):
    return 1 if n <= 2 else fib_rec(n - 1) + fib_rec(n - 2)


fib_rec(10)
```

```
55
```

The definition *is* the recurrence: `fib_rec` calls itself on smaller inputs until it hits the base case. (In R, `if` is an expression that returns a value, which is why the body fits on a single line; Python's counterpart is the *conditional expression* `value_if_true if condition else value_if_false`, where the value comes first.)

Elegant, and here catastrophically inefficient. Each call spawns *two* more calls, so the total number of calls grows exponentially in `n`, and the same values get recomputed over and over. Watch the run time grow as `n` creeps up (R's `elapsed` is the wall-clock time in seconds; the Python column hands the code to `timeit.timeit(..., number=1)` as a string, which runs it once and returns seconds directly, with `globals=globals()` letting the timed code see our function):

**R**

```
system.time(fib_rec(20))
```

```
   user  system elapsed
  0.002   0.000   0.002
```

```
system.time(fib_rec(30))
```

```
   user  system elapsed
  0.279   0.001   0.280
```

**Python**

```
import timeit

timeit.timeit("fib_rec(20)", globals=globals(), number=1)
```

```
0.0003767850002986961
```

```
timeit.timeit("fib_rec(30)", globals=globals(), number=1)
```

```
0.04834544499999538
```

Ten more terms, roughly a hundred times slower, while the humble preallocated loop of Chapter 2 filled 10,000 slots in milliseconds. Recursion also has a hard depth limit: every nested call sits on a stack waiting for its sub-calls, and R gives up beyond a few thousand levels. Python is stricter still, with a default cap of 1,000 frames; inspect or change it with `sys.getrecursionlimit()` / `sys.setrecursionlimit()`, though wanting to raise it is usually a hint that a loop would serve better. Forget the base case and you hit the limit immediately:

**R**

```r
broken <- function(n) broken(n - 1)   # no base case!
broken(1)
```

```
Error: evaluation nested too deeply: infinite recursion / options(expressions=)?
```

**Python**

```python
def broken(n):
    return broken(n - 1)   # no base case!

broken(1)
```

```
RecursionError: maximum recursion depth exceeded
```

Verdict: recursion is a beautiful way to *think*, and for divide-and-conquer algorithms it can be genuinely efficient, but for simple sequences, in either language, the loop wins.

Exercise 3.4: Recursive factorial

a. Write a recursive function `fact_rec` for $n! = n \times (n-1)!$ with base case $0! = 1$, mimicking `fib_rec` above.
b. Check `fact_rec(10)` against R's built-in `factorial(10)` (Python: `math.factorial(10)`).
c. Try `fact_rec(10000)`. What happens, and why? Would a loop version have the same problem?

## 3.5 Functions are values

Time for the second act. R is, at heart, a **functional programming** language: functions are *first-class* values. You have already seen the evidence: we create functions with `function()` and assign them with `<-`, like any number or vector. It follows that we can also store them, pass them to other functions as arguments, and even return them from functions. The same holds in Python: `def` and `lambda` both produce ordinary objects that can be assigned, stored and passed around.

Storing them takes a container more permissive than the atomic vector, so meet the **list**: built with `list()`, it holds elements of *any* type (numbers, strings, vectors of different lengths, other lists, and yes, functions). Name the elements when you build it, then extract one with `$name` or with double brackets `[[ ]]` (by position or by name). Data frames, the stars of Chapter 4, are secretly lists of columns, so the `$` will look familiar there.

**Note for Python:** Python splits the job of R's list between two containers: the **list** `[...]` (ordered, extracted by position) and the **dict** `{"name": value}` (extracted by name). Both hold elements of any type, functions included. Where R writes `funs$root` or `funs[["root"]]`, Python writes `funs["root"]`; and pandas data frames (Chapter 4) are dict-like collections of columns in just the way R data frames are lists of columns.

**R**

```
square <- function(x) x^2            # one-expression body: no braces needed

funs <- list(sq = square, root = sqrt)   # functions in a list, why not
funs$sq(4)
```

```
[1] 16
```

```
funs$root(4)
```

```
[1] 2
```

```
funs[["root"]](9)   # double brackets: extract by name or position
```

```
[1] 3
```

**Python**

```
def square(x):                       # one-expression body: still needs def...
    return x**2                      # ...or a lambda, coming right up

funs = {"sq": square, "root": np.sqrt}   # functions in a dict, why not
funs["sq"](4)
```

```
16
```

```
funs["root"](4)
```

```
np.float64(2.0)
```

```r
funs["root"](9)   # dicts extract by name only -- no positional [[ ]]
```

```
np.float64(3.0)
```

### 3.5.1 Lambdas

For short, throwaway functions, both languages offer a **lambda** shorthand: R (since version 4.1) spells it `\( )`, which is exactly equivalent to `function( )`, and Python spells it with the keyword `lambda`:

**R**

```r
square <- \(x) x^2        # same as: function(x) x^2
add    <- \(x, y) x + y

square(4)
```

```
[1] 16
```

```r
add(1, 2)
```

```
[1] 3
```

**Python**

```python
square = lambda x: x**2        # same as: def square(x): return x**2
add    = lambda x, y: x + y

square(4)
```

```
16
```

```python
add(1, 2)
```

```
3
```

We will use `\(x)` for one-liners and anonymous functions, and keep the long form `function(x)` for anything with a multi-line body. They are the same thing; the backslash just saves seven keystrokes.

**Note for Python:** a `lambda` body must be a **single expression**: no statements, no assignments, no multi-line logic; the expression's value is returned automatically. R's `\(x)` carries no such restriction (it is literally `function(x)` respelled). The moment a Python function needs more than one expression, switch to `def`. Style guides also frown on *assigning* a lambda to a name as we just did (`def` is the official way), but for throwaway one-liners in these notes we allow ourselves the shorthand.

### 3.5.2 Function factories and currying

Here is where first-class functions start paying off: a function can *return a function*. Such a function is called a **function factory**:

**R**

```
make_power <- function(exponent) {
  \(x) x^exponent       # the returned function remembers `exponent`
}

cube         <- make_power(3)
to_the_tenth <- make_power(10)

cube(2)
```

```
[1] 8
```

```
to_the_tenth(2)
```

```
[1] 1024
```

**Python**

```
def make_power(exponent):
    return lambda x: x**exponent   # the returned function remembers `exponent`


cube         = make_power(3)
to_the_tenth = make_power(10)

cube(2)
```

```
8
```

```
to_the_tenth(2)
```

```
1024
```

Note the small miracle: `cube` still knows that `exponent` is 3, long after the call to `make_power` finished. The returned function carries its birth environment with it (the technical name is a *closure*), and this works identically in both languages.

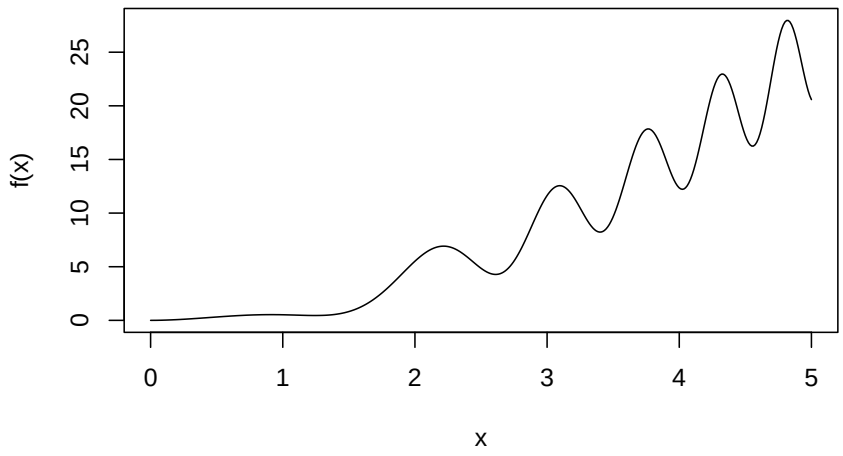
Turning a function of several arguments into a chain of single-argument functions like this is called **currying** (after the logician Haskell Curry; the food connection is, sadly, coincidental). It shines when you fix some arguments once and vary the others a lot. A situation you will meet constantly in numerical work: evaluating a function on grids of different resolutions. Take this deliberately awkward function,

$$f(x) = x^2 + x\sin(x - 1.5\,x^2),$$

**R**

```
f <- \(x) x^2 + x * sin(x - 1.5 * x^2)

# a quick look -- we'll do plotting properly in week 4
xs <- seq(0, 5, length.out = 400)
plot(xs, f(xs), type = "l", xlab = "x", ylab = "f(x)")
```

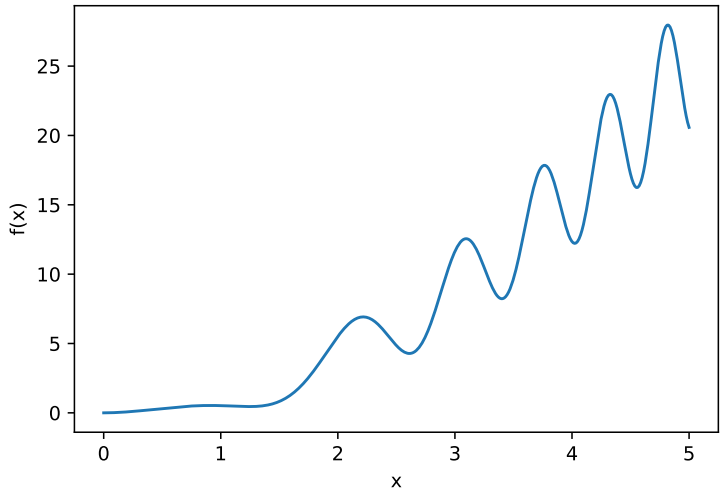



**Python**

```python
import matplotlib.pyplot as plt

f = lambda x: x**2 + x * np.sin(x - 1.5 * x**2)

# a quick look -- we'll do plotting properly in week 4
xs = np.linspace(0, 5, 400)
plt.plot(xs, f(xs))
plt.xlabel("x")
plt.ylabel("f(x)")
plt.show()
```




and suppose we want its maximum over $[0, u]$ for various $u$, numerically, on grids of different resolutions. A grid factory currying `seq()` (Python: `np.linspace()`) does this neatly:

**R**

```r
grid_factory <- function(len) {
  \(up) seq(from = 0, to = up, length.out = len)
}

coarse <- grid_factory(10)     # 10-point grids
fine   <- grid_factory(1000)   # 1000-point grids

max(f(coarse(4)))   # coarse grid: misses the wiggles
```

```
[1] 12.53
```

```
max(f(fine(4)))     # fine grid: much closer to the true maximum
```

```
[1] 17.84983
```

**Python**

```
def grid_factory(length):     # `len` is a Python built-in: pick another name
    return lambda up: np.linspace(0, up, length)


coarse = grid_factory(10)     # 10-point grids
fine   = grid_factory(1000)   # 1000-point grids

max(f(coarse(4)))   # coarse grid: misses the wiggles
```

```
np.float64(12.52999905034901)
```

```
max(f(fine(4)))     # fine grid: much closer to the true maximum
```

```
np.float64(17.84983180085103)
```

The complementary move is a function that *takes* functions as arguments. Pass the function's name (`f`), not a call (`f(...)`):

**R**

```
# evaluate a function on a grid up to `up`, return the max
max_over_grid <- function(f, grid, up) {
  max(f(grid(up)))
}

f2 <- \(x) x^3 - x * cos(x - 1.5 * x^2)

max_over_grid(f,  fine, 4)
```

```
[1] 17.84983
```

```r
max_over_grid(f2, fine, 4)
```

```
[1] 62.36767
```

**Python**

```python
# evaluate a function on a grid up to `up`, return the max
def max_over_grid(f, grid, up):
    return max(f(grid(up)))


f2 = lambda x: x**3 - x * np.cos(x - 1.5 * x**2)

max_over_grid(f,  fine, 4)
```

```
np.float64(17.84983180085103)
```

```python
max_over_grid(f2, fine, 4)
```

```
np.float64(62.36767175274643)
```

One generic `max_over_grid`, any function, any grid resolution. This pattern (machinery that accepts the mathematical object as an argument) is exactly how we will package gradient descent at the end of the chapter.

Exercise 3.5: A density factory

The density of a $N(\mu, \sigma^2)$ distribution is

$$\varphi_{\mu,\sigma}(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

a. Write a factory `make_dnorm(mu, sigma)` that returns the function $\varphi_{\mu,\sigma}$ (a lambda in `x`), implementing the formula yourself (`pi` is built in; Python: `np.pi`).
b. Create `standard <- make_dnorm(0, 1)` and `wide <- make_dnorm(0, 3)`, and evaluate both at $x = 0$ and $x = 1$.
c. Check your factory against R's `dnorm(x, mean, sd)` (Python: `scipy.stats.norm.pdf(x, mu, sigma)`) with `all.equal()` (Python: `np.allclose()`) on a vector of test points.
d. Quick base-plot sanity check: plot `standard` over $[-6, 6]$ with `type = "l"`, then

overlay `wide` using `lines()` (Python: two `plt.plot` calls on the same figure).

### 3.5.3 map: one function, every element

Vectorised arithmetic (Chapter 1) covers a lot of ground, but it works on atomic vectors (numpy arrays, on the Python side). The moment your data lives in a list (samples of different sizes, say), `mean(samples)` is meaningless and a `for` loop feels like heavy machinery for such a small job. The functional answer is `map`: apply a function to every element of a collection, collect the results. It lives in the `purrr` package (part of the tidyverse, which we meet properly in Chapter 4); Python has its counterparts built in:

**R**

```
library(purrr)

set.seed(7432)   # randomness ahead: fix the seed so results are reproducible
samples <- list(
  small  = rnorm(5),
  medium = rnorm(50),
  large  = rnorm(5000)
)

map(samples, mean)       # returns a list
```

```
$small
[1] 0.2437877

$medium
[1] -0.2232069

$large
[1] -0.01679829
```

```
map_dbl(samples, mean)   # returns a named numeric vector of doubles
```

```
      small      medium       large
 0.24378768 -0.22320689 -0.01679829
```

```r
map_dbl(samples, sd)
```

```
    small    medium     large
0.6906119 0.8322211 1.0034644
```

### Python

```python
# no library needed: map() and comprehensions are built in

rng = np.random.default_rng(7432)   # randomness ahead: fix the seed so results are reproduci
samples = {
    "small":  rng.normal(size=5),
    "medium": rng.normal(size=50),
    "large":  rng.normal(size=5000),
}

list(map(np.mean, samples.values()))                # map(): a plain list, names lost
```

```
[np.float64(-0.15104809319671958), np.float64(-0.008465847720283817), np.float64(-
0.014895157721422986)]
```

```python
{name: np.mean(x) for name, x in samples.items()}    # dict comprehension: names kept
```

```
{'small': np.float64(-0.15104809319671958), 'medium': np.float64(-0.008465847720283817), 'la
0.014895157721422986)}
```

```python
{name: np.std(x, ddof=1) for name, x in samples.items()}   # ddof=1 matches R's sd
```

```
{'small': np.float64(0.811246783136126), 'medium': np.float64(0.9253342476731854), 'large': n
```

`map()` always returns a list, because *a priori* it cannot know what your function outputs. When you know the common type, the suffixed versions `map_dbl`, `map_int`, `map_chr`, `map_lgl` return an atomic vector of that type, and fail loudly if any result does not fit, which is exactly the kind of failure you want. (One current exception: `map_chr` handed a number coerces it with a deprecation warning rather than an error; purrr would rather you called `as.character()` yourself.)

**Note for Python:** `map(f, xs)` is built in and *lazy*: it returns an iterator that computes nothing until asked, hence the `list(...)` wrapper to see the results at once. The more

idiomatic tool is the **comprehension**: `[f(x) for x in xs]` for lists, `{k: f(v) for k, v in d.items()}` for dicts (our named lists). There are no type-suffixed variants and no type checking: a comprehension happily builds a list of mixed types, so the discipline `map_dbl` enforces is on you.

A word on the randomness: `set.seed(7432)` and `np.random.default_rng(7432)` each fix their own language's stream, so re-running reproduces the same numbers *within* a language, but the two generators are unrelated, so the R and Python columns print different samples. Nothing is wrong.

Lambdas slot in perfectly as the function argument:

**R**

```r
# sample-mean of a N(k, 1) sample, for several k
map_dbl(1:5, \(k) mean(rnorm(1000, mean = k)))
```

```
[1] 0.9730188 1.9949919 3.0825153 3.9797734 5.0297364
```

**Python**

```python
# sample-mean of a N(k, 1) sample, for several k
[np.mean(rng.normal(loc=k, size=1000)) for k in range(1, 6)]
```

```
[np.float64(0.9917050759103179), np.float64(2.041836403752563), np.float64(2.985753453443778
```

When the function needs **two** varying inputs, use `map2`, which walks two collections in parallel:

**R**

```r
# trimmed (also known as truncated) means: a different trimming fraction for each sample
trims <- c(0, 0.1, 0.2)
map2_dbl(samples, trims, \(x, t) mean(x, trim = t))
```

```
     small     medium      large
 0.2437877 -0.2610742 -0.0181686
```

**Python**

```python
from scipy.stats import trim_mean

# trimmed means: a different trimming fraction for each sample
trims = [0, 0.1, 0.2]
[trim_mean(x, t) for x, t in zip(samples.values(), trims)]
```

```
[np.float64(-0.15104809319671958), np.float64(-0.0811993214607887), np.float64(-
0.018583434160877932)]
```

There is also `pmap` for three-or-more inputs, same idea. One line of caution: the collections passed to `map2` should have the same length.

**Note for Python:** `zip` pairs the collections up (and `map()` itself takes several iterables: `map(f, xs, ys)`); both stop silently at the *shortest* input rather than complaining, so check your lengths yourself.

**Base R has this too.** The `apply` family predates purrr and you will see it everywhere: `lapply(x, f)` is essentially `map`, `sapply(x, f)` is `map` that then *simplifies* to a vector when it can (convenient interactively, mildly hair-raising in scripts, since the output type depends on the data), `vapply` is the type-safe version, and `apply(M, margin, f)` sweeps a function over the rows (`margin = 1`) or columns (`margin = 2`) of a matrix. We use purrr in these notes for its consistent naming and type-stable returns, but the two families are interchangeable for everything we do:

**R**

```r
sapply(samples, mean)   # base R equivalent of map_dbl(samples, mean)
```

```
      small      medium       large
 0.24378768 -0.22320689 -0.01679829
```

**Python**

```python
# Python has no purrr-vs-base split: comprehensions are the one idiom
{name: np.mean(x) for name, x in samples.items()}
```

```
{'small': np.float64(-0.15104809319671958), 'medium': np.float64(-0.008465847720283817), 'la
0.014895157721422986)}
```

Exercise 3.6: Mapping the lab marks

Three lab groups sat a test:

**R**

```r
marks <- list(
  lab1 = c(72, 58, 91, 43, 65),
  lab2 = c(80, 61, 75, 49, 70, 66),
  lab3 = c(55, 95, 78, 62)
)
```

**Python**

```python
marks = {
    "lab1": [72, 58, 91, 43, 65],
    "lab2": [80, 61, 75, 49, 70, 66],
    "lab3": [55, 95, 78, 62],
}
```

a. Use `map_dbl` (Python: a dict comprehension) to get each group's average mark.
b. Use `map_int` (or `map_dbl`) with `length` (Python: `len`) to get each group's size.
c. Use `map_dbl` with a lambda to compute each group's mark *range* (max minus min).
d. Use `map` with `sort` (Python: `sorted`). Why would `map_dbl` be the wrong tool here (what *shape* does each result have)?
e. Which group did best on average? Answer in code with `which.max` (Python: `max(..., key=...)`).

Exercise 3.7: Currying the trimmed (truncated) mean

Reaction times (seconds) for four subjects, each with the occasional attention lapse:

**R**

```r
reaction_times <- list(
  subj1 = c(0.31, 0.28, 0.35, 0.30, 4.20),
  subj2 = c(0.25, 0.29, 0.26, 3.80, 0.27),
  subj3 = c(0.33, 0.34, 0.31, 0.32, 0.30),
  subj4 = c(0.29, 5.10, 0.28, 0.30, 0.26)
)
```

**Python**

```
reaction_times = {
    "subj1": [0.31, 0.28, 0.35, 0.30, 4.20],
    "subj2": [0.25, 0.29, 0.26, 3.80, 0.27],
    "subj3": [0.33, 0.34, 0.31, 0.32, 0.30],
    "subj4": [0.29, 5.10, 0.28, 0.30, 0.26],
}
```

a. Write a factory `make_trimmed_mean(trim)` that returns a function computing `mean(x, trim = trim)` (Python: `scipy.stats.trim_mean(x, trim)`).
b. Use `map_dbl` (Python: a comprehension) with `make_trimmed_mean(0)` and then with `make_trimmed_mean(0.2)` on `reaction_times`.
c. Compare the two sets of results. Which subjects change most, and why is the trimmed version a better summary here?

### 3.5.4 `Reduce` / `functools.reduce`: fold a collection to one value

Where `map` applies a function elementwise, `Reduce` (base R; purrr spells it `reduce`, Python keeps it in `functools.reduce`) *folds* a whole collection down to a single value with a two-argument function: combine the first two elements, combine the result with the third, and so on.

**R**

```
attended <- list(
  week1 = c("Ada", "Bea", "Carl", "Dan", "Eve"),
  week2 = c("Ada", "Carl", "Dan", "Eve"),
  week3 = c("Ada", "Carl", "Eve", "Fay")
)

# who has attended every single week? intersect two at a time:
Reduce(intersect, attended)
```

```
[1] "Ada"  "Carl" "Eve"
```

```
Reduce(`+`, 1:5)                       # ((((1 + 2) + 3) + 4) + 5): the sum
```

```
[1] 15
```

```
Reduce(`+`, 1:5, accumulate = TRUE)    # keep the partial results: cumsum
```

```
[1]  1  3  6 10 15
```

**Python**

```python
from functools import reduce
from operator import add
from itertools import accumulate

attended = {
    "week1": ["Ada", "Bea", "Carl", "Dan", "Eve"],
    "week2": ["Ada", "Carl", "Dan", "Eve"],
    "week3": ["Ada", "Carl", "Eve", "Fay"],
}

# who has attended every single week? intersect two at a time:
reduce(lambda a, b: [name for name in a if name in b], attended.values())
```

```
['Ada', 'Carl', 'Eve']
```

```python
reduce(add, range(1, 6))          # ((((1 + 2) + 3) + 4) + 5): the sum
```

```
15
```

```python
list(accumulate(range(1, 6)))     # keep the partial results: cumsum
```

```
[1, 3, 6, 10, 15]
```

The backticks around `` `+` `` let us pass an operator where a function is expected: operators *are* functions in R. That is the whole idea; `Reduce` is niche but irreplaceable when it fits.

**Note for Python:** the pieces live in three standard-library modules: `functools.reduce` is the fold, the `operator` module wraps the operators as functions (`operator.add` is Python's `` `+` ``), and running folds come from `itertools.accumulate` rather than an `accumulate =` flag. There is no built-in `intersect` for lists, hence the lambda.

Exercise 3.8: Reduce, reuse, recycle

a. Compute 10! with a single `Reduce` call (no loops, no `factorial`). (Python: `functools.reduce` with `operator.mul`.)
b. Given `x <- c(3, 1, 4, 1, 5, 9, 2, 6)`, compute the *running maximum* with

`Reduce` and `accumulate = TRUE` (Python: `itertools.accumulate` with `max`), and check it against the built-in `cummax(x)` (Python: `np.maximum.accumulate(x)`).

c. With the `attended` list from above, use `Reduce` and `union` to list everyone who attended *at least once*, and check it against `unique(unlist(attended))` (Python: write the union step yourself, and check with `np.unique` on the concatenated lists, noting that `np.unique` also sorts).

## 3.6 Packaging gradient descent

Time to cash in. In Section 2.8 we minimised a function by iterating

$$x_{n+1} = x_n - \gamma\, f'(x_n),$$

with the derivative typed straight into the loop body. Every new target function meant surgery on the loop. Now we know better: the derivative can simply be an *argument*.

**R**

```r
grad_descent <- function(grad, x0, gamma, n) {
  x <- x0
  for (i in 1:n) {
    x <- x - gamma * grad(x)   # the week 2 update, verbatim
  }
  x
}
```

**Python**

```python
def grad_descent(grad, x0, gamma, n):
    x = x0
    for i in range(n):
        x = x - gamma * grad(x)   # the week 2 update, verbatim
    return x
```

That is the whole algorithm: pure (no globals touched), reusable, half a dozen lines. Notice what is *not* among the arguments: the objective $f$ itself. The update rule only ever evaluates the gradient, so the gradient is all we pass; an argument the body never uses would only be noise. The objective is still useful to *us*, to check the answer (see the plot below), and we simply call it ourselves when we need it. The signature `grad_descent(grad, x0, gamma, n)` is identical in both languages by design, and Chapter 5 keeps it.

Minimising $f(x) = (x-2)^2$, whose gradient is $f'(x) = 2(x-2)$, is now a matter of one lambda:

**R**

```r
grad_f <- \(x) 2 * (x - 2)

grad_descent(grad_f, x0 = 10, gamma = 0.1, n = 100)
```

```
[1] 2
```

**Python**

```python
grad_f = lambda x: 2 * (x - 2)

grad_descent(grad_f, x0=10, gamma=0.1, n=100)
```

```
2.000000001629629
```

Swapping in a completely different problem costs nothing. Here is the double-well function $g(x) = x^4/4 - x^2/2$, with $g'(x) = x^3 - x$ and *two* local minima at $\pm 1$:

**R**

```r
g      <- \(x) x^4 / 4 - x^2 / 2
grad_g <- \(x) x^3 - x

grad_descent(grad_g, x0 =  2, gamma = 0.1, n = 200)
```

```
[1] 1
```

```r
grad_descent(grad_g, x0 = -2, gamma = 0.1, n = 200)   # other basin
```

```
[1] -1
```

**Python**

```python
g      = lambda x: x**4 / 4 - x**2 / 2
grad_g = lambda x: x**3 - x

grad_descent(grad_g, x0=2, gamma=0.1, n=200)
```

```
1.0000000000000004
```

```
grad_descent(grad_g, x0=-2, gamma=0.1, n=200)   # other basin
```

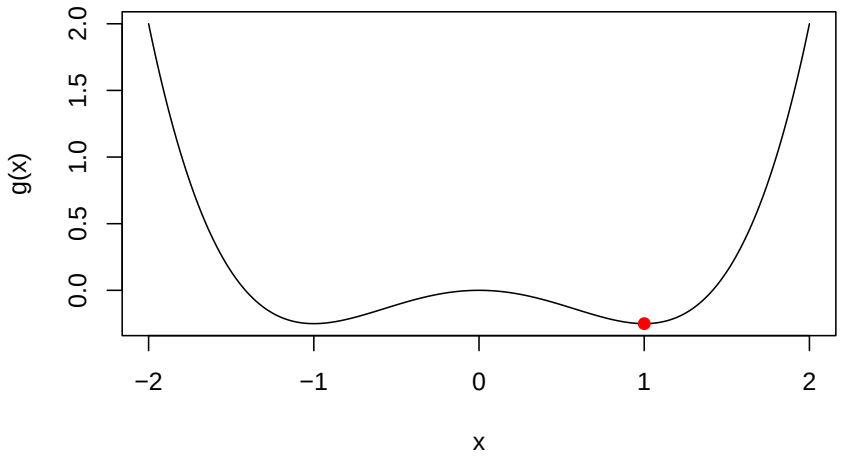
```
-1.0000000000000004
```

Gradient descent rolls downhill from wherever you drop it, so it finds the *local* minimum of its starting basin, a theme we return to at length in Chapter 5. A quick base-plot sanity check that the answer really sits at the bottom:

**R**

```
xhat <- grad_descent(grad_g, x0 = 2, gamma = 0.1, n = 200)

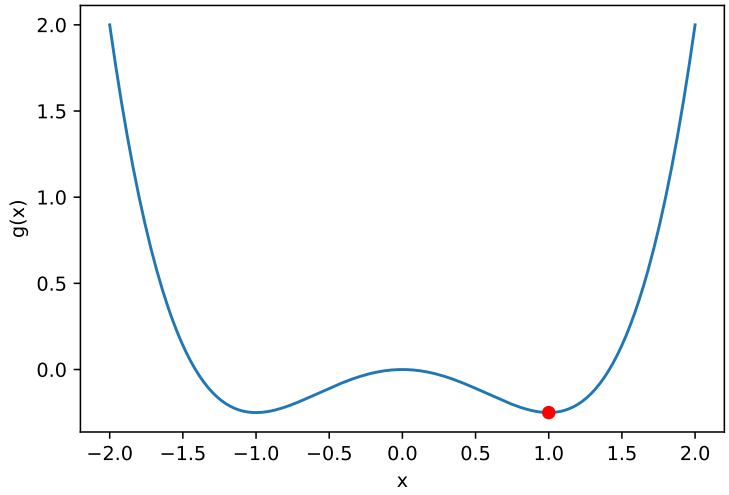
xs <- seq(-2, 2, length.out = 400)
plot(xs, g(xs), type = "l", xlab = "x", ylab = "g(x)")
points(xhat, g(xhat), col = "red", pch = 19)   # our minimiser
```




**Python**

```
xhat = grad_descent(grad_g, x0=2, gamma=0.1, n=200)

xs = np.linspace(-2, 2, 400)
plt.plot(xs, g(xs))
plt.xlabel("x")
plt.ylabel("g(x)")
plt.plot(xhat, g(xhat), "ro")   # our minimiser
plt.show()
```




One last cautionary demonstration: the step size $\gamma$ is not a detail:

**R**

```r
grad_descent(grad_f, x0 = 10, gamma = 1.05, n = 50)   # too greedy: diverges
```

```
[1] 941.1268
```

**Python**

```python
grad_descent(grad_f, x0=10, gamma=1.05, n=50)   # too greedy: diverges
```

```
941.1268230375671
```

Each step overshoots the minimum by more than it gained, and the iterates diverge (no randomness here, so for once the two columns agree to the last digit). Why this happens, how to choose $\gamma$, and what to do when the problem is multivariate and the data are big: that is precisely the programme of Chapter 5, where `grad_descent` and its descendants do statistics for real.

> Exercise 3.9: A new client for grad_descent
>
> Consider $h(x) = e^x + x^2$.
>
> a. Derive $h'(x)$ by hand, and write both as lambdas.

b. Run `grad_descent(grad_h, x0 = 1, gamma = 0.1, n = 100)`.
c. Sanity-check the result with a base plot (Python: matplotlib) of $h$ over $[-2, 1]$, marking your minimiser with `points()` (Python: `plt.plot(..., "ro")`).
d. Compare your answer with R's built-in one-dimensional optimiser: `optimize(h, interval = c(-2, 1))` (Python: `scipy.optimize.minimize_scalar(h, bounds = (-2, 1))`).
e. Re-run with `gamma = 1` and with `gamma = 0.001` (same `n`). Describe what goes wrong in each case.

## 3.7 Challenges

Two optional problems: one teaches `grad_descent` to stop by itself, one generalises a classic.

**!** Challenge 3.1: Stop when you're done

Our `grad_descent` runs for exactly `n` steps, even if it converged after ten. Write an improved `grad_descent_tol(grad, x0, gamma, max_n, tol = 1e-8)` that:

a. iterates at most `max_n` times, but **stops early** when the step becomes tiny: $|x_{\text{new}} - x_{\text{old}}| <$ `tol`;
b. records every iterate along the way (including `x0`);
c. returns a **list** (Python: a dict) with three elements: `minimiser` (the final $x$), `path` (the vector of all iterates), and `iterations` (how many updates were performed).

Run it on $f(x) = (x - 2)^2$ with `x0 = 10`, `gamma = 0.1`, `max_n = 1000`. How many iterations did it actually need? Finish with a base plot of `f(path)` against iteration number; add `log = "y"` to the plot call (Python: `plt.yscale("log")`) and admire the straight line (linear convergence, literally).

**!** Challenge 3.2: Generic FizzBuzz

The classic FizzBuzz problem: for the numbers $1, \ldots, n$, print "Fizz" for multiples of 3, "Buzz" for multiples of 5, "FizzBuzz" for multiples of both, and the number itself otherwise. You are a mathematician, so you will of course generalise it.
Write `generic_fizzbuzz(x, divisors, phrases)` taking a vector of numbers, a vector of divisors, and a vector of phrases (same length as `divisors`). Each number maps to the concatenation of the phrases of all divisors that divide it, or to the number itself (as a character) if none do. The classic game is then:
**R**

```r
generic_fizzbuzz(1:15, c(3, 5), c("Fizz", "Buzz"))
```

**Python**

```python
generic_fizzbuzz(range(1, 16), [3, 5], ["Fizz", "Buzz"])
```

and your test case:
**R**

```r
generic_fizzbuzz(1:20, c(2, 3, 5, 7), c("Skibbidi", "Fizz", "Buzz", "Yo"))
```

**Python**

```python
generic_fizzbuzz(range(1, 21), [2, 3, 5, 7], ["Skibbidi", "Fizz", "Buzz", "Yo"])
```

Hint: `map_chr` over `x`, and inside the lambda, logical indexing of `phrases` plus `paste0(..., collapse = "")` gets you there in very few lines. Python route: a list comprehension over `x`, and inside, `"".join` over `zip(phrases, divisors)` plays the role of the logical indexing plus `paste0`; and an empty string is falsy, which handles the fallback.

## 3.8 Looking ahead

We now hold both halves of the toolkit this module runs on: writing clean, pure functions, and wiring them together functionally: factories to build them, `map` to spread them over data, and higher-order functions like `grad_descent` that take the mathematics as an argument. In Chapter 4 we meet the data structures and plotting tools that real datasets deserve, and in Chapter 5 everything converges (pun fully intended) on optimisation for statistics. There we will see how useful this functional style is in practice.

# 4 Week 4: Data Frames and Plotting

Three weeks of machinery: vectors and matrices (Chapter 1), loops (Chapter 2), functions (Chapter 3). This week we point all of it at what statisticians actually spend their days on: tables of data. We will learn how to store data in *data frames*, manipulate them with `dplyr` (in Python: `pandas`), and, at last, plot them properly with `ggplot2` (in Python: `plotnine`). The quick-and-dirty `plot()` and `plt.plot()` calls from the previous weeks retire this week.

## 4.1 The tidyverse (and friends in Python)

Modern data work in R happens largely inside the **tidyverse**, a collection of packages that share a common design: `dplyr` for manipulation, `readr` for import/export, `tibble` for nicer data frames, `ggplot2` for graphics, and `purrr`, which you already met in Chapter 3.

**Note for Python:** Python spreads the same jobs over separate packages: `pandas` covers what `dplyr`, `readr` and `tibble` do, and `plotnine` is a deliberate port of `ggplot2` (same grammar, nearly the same code). Together with `numpy` they are this week's toolkit, installed and imported alongside the R column below.

Install them once (already done on university machines). The R line runs in the R console; the Python line is a shell command, so it runs in a terminal, not in the Python console:

**R**

```
install.packages("tidyverse")
```

**Python**

```
python -m pip install pandas plotnine
```

then load them at the top of every script:

**R**

```
library(tidyverse)
```

**Python**

```
# several packages instead of one: import them together, at the top
import numpy as np
import pandas as pd
from plotnine import (
    ggplot, aes, geom_point, geom_smooth, geom_histogram, geom_boxplot,
    geom_line, geom_hline, facet_wrap, labs, theme_minimal
)
```

Installing happens once per computer; loading happens once per session: a distinction everyone mixes up exactly once.

## 4.2 Data frames

A **data frame** is a table: each column is a named vector, and different columns may hold different types (numeric, character, logical). This is the crucial difference from the matrices of Chapter 1, where every entry had to share one type. In statistics-speak, columns are **variables** and rows are **observations**.

### 4.2.1 Building one

Here is a small dataset of plant-based milks and their nutritional content per 250 ml glass:

| type | calories | protein | fibre | carbs | sugars | fat |
|---|---|---|---|---|---|---|
| Almond | 60 | 1 | 1 | 8 | 7 | 2.5 |
| Soy | 100 | 7 | 2 | 10 | NA | 4 |
| Oat | 120 | 3 | NA | 20 | 19 | 2.5 |
| Rice | 130 | 1 | NA | 25 | NA | 2.5 |

Note the `NA`s: real data is rarely complete, and R has a dedicated value for "not available" (in Python, `pandas` marks missing numbers with the floating-point `NaN`, written `np.nan`). We build the data frame column by column with `data.frame()` (in Python, from a dictionary mapping column names to lists):

**R**

```r
# each argument is one column (a vector); all must have the same length
milk <- data.frame(
  type     = c("Almond", "Soy", "Oat", "Rice"),
  calories = c(60, 100, 120, 130),
  protein  = c(1, 7, 3, 1),
  fibre    = c(1, 2, NA, NA),
  carbs    = c(8, 10, 20, 25),
  sugars   = c(7, NA, 19, NA),
  fat      = c(2.5, 4, 2.5, 2.5)
)
milk
```

```
    type calories protein fibre carbs sugars fat
1 Almond       60       1     1     8      7 2.5
2    Soy      100       7     2    10     NA 4.0
3    Oat      120       3    NA    20     19 2.5
4   Rice      130       1    NA    25     NA 2.5
```

**Python**

```python
# each entry is one column (a list); all must have the same length
milk = pd.DataFrame({
    "type":     ["Almond", "Soy", "Oat", "Rice"],
    "calories": [60, 100, 120, 130],
    "protein":  [1, 7, 3, 1],
    "fibre":    [1, 2, np.nan, np.nan],
    "carbs":    [8, 10, 20, 25],
    "sugars":   [7, np.nan, 19, np.nan],
    "fat":      [2.5, 4, 2.5, 2.5]
})
milk
```

```
     type  calories  protein  fibre  carbs  sugars  fat
0  Almond        60        1    1.0      8     7.0  2.5
1     Soy       100        7    2.0     10     NaN  4.0
2     Oat       120        3    NaN     20    19.0  2.5
3    Rice       130        1    NaN     25     NaN  2.5
```

### 4.2.2 Columns, rows and cells

Single columns come out with the dollar syntax in R and single square brackets in Python, returning a plain vector (in pandas: a `Series`):

**R**

```
milk$calories
```

```
[1]  60 100 120 130
```

**Python**

```
milk["calories"]
```

```
0     60
1    100
2    120
3    130
Name: calories, dtype: int64
```

In R, square brackets with a column name return instead a *smaller data frame*. pandas does the same when you double the brackets:

**R**

```
milk["calories"]           # still a data frame
```

```
  calories
1       60
2      100
3      120
4      130
```

```
milk[c("type", "protein")] # two columns, still a data frame
```

```
    type protein
1 Almond       1
2    Soy       7
3    Oat       3
4   Rice       1
```

**Python**

```
milk[["calories"]]          # double brackets: still a data frame
```

```
   calories
0        60
1       100
2       120
3       130
```

```
milk[["type", "protein"]]   # two columns, still a data frame
```

```
     type  protein
0  Almond        1
1     Soy        7
2     Oat        3
3    Rice        1
```

**Note for Python:** the single/double bracket split is worth committing to memory: `milk["calories"]` is a `Series` (the vector-like object), `milk[["calories"]]` is a one-column `DataFrame`. The attribute shorthand `milk.calories` also works, but only for names that are valid Python identifiers. Brackets are the safe habit.

For rows (and row/column combinations) we index in two dimensions, exactly as we did with matrices: `df[rows, columns]`, with an empty slot meaning "everything":

**R**

```
milk[2, ]                       # second row, all columns
```

```
  type calories protein fibre carbs sugars fat
2  Soy      100       7     2    10     NA   4
```

```
milk[1:2, c("type", "calories")] # first two rows, two columns
```

```
    type calories
1 Almond       60
2    Soy      100
```

**Python**

```
milk.iloc[[1]]                        # second row: position 1 (Python counts from 0!)
```

```
  type  calories  protein  fibre  carbs  sugars  fat
1  Soy       100        7    2.0     10     NaN  4.0
```

```
milk.loc[0:1, ["type", "calories"]]  # first two rows, two columns
```

```
     type  calories
0  Almond        60
1     Soy       100
```

**Note for Python:** pandas splits two-dimensional indexing over two *indexers*: `.iloc` takes integer **positions** (0-based, end excluded: the rules of Chapter 1), while `.loc` takes **labels** (here the row labels happen to be the integers 0, 1, 2, …) and includes *both* ends, which is why `0:1` above returns two rows. Also, `milk.iloc[1]` without the inner brackets squeezes the row into a `Series`; the doubled brackets keep it a one-row `DataFrame`.

The comparison operators of Chapter 2 work here too. A logical vector selects rows:

**R**

```
# rows where calories exceed 100
milk[milk$calories > 100, ]
```

```
  type calories protein fibre carbs sugars fat
3  Oat      120       3    NA    20     19 2.5
4 Rice      130       1    NA    25     NA 2.5
```

**Python**

```
# rows where calories exceed 100
milk[milk["calories"] > 100]
```

```
   type  calories  protein  fibre  carbs  sugars  fat
2   Oat       120        3    NaN     20    19.0  2.5
3  Rice       130        1    NaN     25     NaN  2.5
```

New columns are created by assigning to a name that does not exist yet:

**R**

```r
# what fraction of the carbs is sugar?
milk$sugar_frac <- milk$sugars / milk$carbs
milk
```

```
    type calories protein fibre carbs sugars fat sugar_frac
1 Almond       60       1     1     8      7 2.5      0.875
2    Soy      100       7     2    10     NA 4.0         NA
3    Oat      120       3    NA    20     19 2.5      0.950
4   Rice      130       1    NA    25     NA 2.5         NA
```

**Python**

```python
# what fraction of the carbs is sugar?
milk["sugar_frac"] = milk["sugars"] / milk["carbs"]
milk
```

```
     type  calories  protein  fibre  carbs  sugars  fat  sugar_frac
0  Almond        60        1    1.0      8     7.0  2.5       0.875
1     Soy       100        7    2.0     10     NaN  4.0         NaN
2     Oat       120        3    NaN     20    19.0  2.5       0.950
3    Rice       130        1    NaN     25     NaN  2.5         NaN
```

### 4.2.3 Getting to know a data frame

For a data frame you did not build yourself, four functions tell you almost everything:

**R**

```r
nrow(milk)     # number of rows (ncol() and dim() also exist)
```

```
[1] 4
```

```r
str(milk)      # structure: column names, types, first values
```

```
'data.frame':   4 obs. of  8 variables:
 $ type      : chr  "Almond" "Soy" "Oat" "Rice"
 $ calories  : num  60 100 120 130
 $ protein   : num  1 7 3 1
 $ fibre     : num  1 2 NA NA
```

```
 $ carbs     : num  8 10 20 25
 $ sugars    : num  7 NA 19 NA
 $ fat       : num  2.5 4 2.5 2.5
 $ sugar_frac: num  0.875 NA 0.95 NA
```

```
summary(milk)  # summary statistics per column, NAs included
```

```
        type      calories        protein      fibre          carbs
 Length   :4   Min.   : 60.0   Min.   :1   Min.   :1.00   Min.   : 8.00
 N.unique :4   1st Qu.: 90.0   1st Qu.:1   1st Qu.:1.25   1st Qu.: 9.50
 N.blank  :0   Median :110.0   Median :2   Median :1.50   Median :15.00
 Min.nchar:3   Mean   :102.5   Mean   :3   Mean   :1.50   Mean   :15.75
 Max.nchar:6   3rd Qu.:122.5   3rd Qu.:4   3rd Qu.:1.75   3rd Qu.:21.25
               Max.   :130.0   Max.   :7   Max.   :2.00   Max.   :25.00
                                           NAs    :2
     sugars         fat          sugar_frac
 Min.   : 7   Min.   :2.500   Min.   :0.8750
 1st Qu.:10   1st Qu.:2.500   1st Qu.:0.8938
 Median :13   Median :2.500   Median :0.9125
 Mean   :13   Mean   :2.875   Mean   :0.9125
 3rd Qu.:16   3rd Qu.:2.875   3rd Qu.:0.9313
 Max.   :19   Max.   :4.000   Max.   :0.9500
 NAs    :2                    NAs    :2
```

```
head(milk, 2)  # first few rows (useful for big data)
```

```
    type calories protein fibre carbs sugars fat sugar_frac
1 Almond       60       1     1     8      7 2.5      0.875
2    Soy      100       7     2    10     NA 4.0         NA
```

### Python

```
len(milk)        # number of rows (milk.shape gives rows and columns)
```

```
4
```

```
milk.info()      # structure: column names, types, non-missing counts
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   type        4 non-null      str
 1   calories    4 non-null      int64
 2   protein     4 non-null      int64
 3   fibre       2 non-null      float64
 4   carbs       4 non-null      int64
 5   sugars      2 non-null      float64
 6   fat         4 non-null      float64
 7   sugar_frac  2 non-null      float64
dtypes: float64(4), int64(3), str(1)
memory usage: 388.0 bytes
```

```
milk.describe()  # summary statistics per numeric column
```

```
         calories   protein     fibre      carbs     sugars    fat  sugar_frac
count    4.000000  4.000000  2.000000   4.000000   2.000000  4.000    2.000000
mean   102.500000  3.000000  1.500000  15.750000  13.000000  2.875    0.912500
std     30.956959  2.828427  0.707107   8.098354   8.485281  0.750    0.053033
min     60.000000  1.000000  1.000000   8.000000   7.000000  2.500    0.875000
25%     90.000000  1.000000  1.250000   9.500000  10.000000  2.500    0.893750
50%    110.000000  2.000000  1.500000  15.000000  13.000000  2.500    0.912500
75%    122.500000  4.000000  1.750000  21.250000  16.000000  2.875    0.931250
max    130.000000  7.000000  2.000000  25.000000  19.000000  4.000    0.950000
```

```
milk.head(2)     # first few rows (useful for big data)
```

```
     type  calories  protein  fibre  carbs  sugars  fat  sugar_frac
0  Almond        60        1    1.0      8     7.0  2.5       0.875
1     Soy       100        7    2.0     10     NaN  4.0         NaN
```

`summary()` counts the `NA`s for us, and those `NA`s are contagious. Any computation touching a missing value returns `NA`, unless you explicitly ask R to drop them:

**R**

```r
mean(milk$sugars)                # NA: R refuses to guess
```

```
[1] NA
```

```r
mean(milk$sugars, na.rm = TRUE) # mean of the observed values
```

```
[1] 13
```

**Python**

```python
milk["sugars"].mean()                # pandas skips NaN by default!
```

```
np.float64(13.0)
```

```python
milk["sugars"].mean(skipna=False)    # the R-style refusal is opt-in
```

```
np.float64(nan)
```

**Note for Python:** the two languages picked *opposite* defaults: R propagates `NA` until you say `na.rm = TRUE`; pandas silently skips `NaN` until you say `skipna=False`. Neither is wrong, but know which one you are getting: a summary that quietly ignores missing values can hide a data problem R would have shouted about.

Exercise 4.1: A data frame of distributions

Build a data frame `dists` describing five distributions, with columns:

- `name`: `"Bernoulli"`, `"Poisson"`, `"Exponential"`, `"Gaussian"`, `"Uniform"`;
- `n_params`: the number of parameters of each: 1, 1, 1, 2, 2;
- `discrete`: `TRUE, TRUE, FALSE, FALSE, FALSE`.

Then:

a. Extract the `name` column as a vector.
b. Using logical indexing, extract the rows of the discrete distributions. (Hint: `discrete` is already a logical vector, so no comparison is needed.)
c. Add a logical column `symmetric` using the dollar syntax (Python: bracket assignment). Only the Gaussian and the Uniform are symmetric.
d. Check your work with `str()` and `dim()` (Python: `.info()` and `.shape`).

## 4.3 Tibbles (and the pandas index)

The tidyverse ships a modernised data frame called a **tibble**. It *is* a data frame (everything above applies) with two behavioural upgrades. First, printing: a tibble shows ten rows, only the columns that fit, and the type of each. A classic data frame would happily dump every last row of a 50,000-row dataset into your console. Second, no partial matching. Watch this data.frame quirk:

**R**

```r
milk$cal   # base data frame: silently matches "calories"!
```

```
[1]  60 100 120 130
```

**Python**

```python
milk["cal"]   # pandas never guesses: unknown column, immediate error
```

```
KeyError: 'cal'
```

That is a bug waiting to happen: mistype a column name and R may "helpfully" return a different column. A tibble warns you and returns `NULL`:

**R**

```r
as_tibble(milk)$cal
```

```
Warning: Unknown or uninitialised column: `cal`.
```

```
NULL
```

**Python**

```python
print(milk.get("cal"))   # the forgiving lookup: None (Python's NULL)
```

```
None
```

**Note for Python:** there is no tibble to reach for because there is no quirk to fix: pandas has exactly **one** table type, the `DataFrame` you have been using all along, and it never partial-matches (as the `KeyError` above shows; `.get()` is the polite version). The genuinely new thing to watch is the **index**: every `DataFrame` carries row labels (the unnamed left column in the printouts), which stick to their rows through filtering and sorting. It is pandas' (heavily used) version of R's (rarely used) row names; when it gets in your way, `.reset_index()` demotes it to an ordinary column, and `index=False` keeps it out of your CSV files. Both will appear below.

You create tibbles directly with `tibble()` (same syntax as `data.frame()`, with the bonus that later columns can use earlier ones), or convert with `as_tibble()`. From now on we work with tibbles, and we need a bigger dataset to play with: `mtcars`, a built-in table of 32 cars from a 1974 issue of *Motor Trend* (in Python it ships with plotnine, as `plotnine.data.mtcars`). The variables we will use most: `mpg` (miles per gallon; higher means more fuel-efficient), `cyl` (cylinders), `hp` (horsepower), `wt` (weight, 1000 lbs), `qsec` (quarter-mile time in seconds), `am` (transmission: 0 automatic, 1 manual).

**R**

```r
head(mtcars, 3)
```

```
               mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4     21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710    22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
```

**Python**

```python
from plotnine.data import mtcars
mtcars.head(3)
```

```
            name   mpg  cyl   disp   hp  drat     wt   qsec  vs  am  gear  carb
0      Mazda RX4  21.0    6  160.0  110  3.90  2.620  16.46   0   1     4     4
1  Mazda RX4 Wag  21.0    6  160.0  110  3.90  2.875  17.02   0   1     4     4
2     Datsun 710  22.8    4  108.0   93  3.85  2.320  18.61   1   1     4     1
```

One wrinkle: `mtcars` stores the car names as *row names*, a base-R feature tibbles deliberately drop. `as_tibble()` can rescue them into a proper column:

**R**

```r
cars <- as_tibble(mtcars, rownames = "model")
cars
```

```
# A tibble: 32 x 12
   model          mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
   <chr>        <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
 1 Mazda RX4     21       6  160    110  3.9   2.62  16.5     0     1     4     4
 2 Mazda RX4 ~   21       6  160    110  3.9   2.88  17.0     0     1     4     4
 3 Datsun 710    22.8     4  108     93  3.85  2.32  18.6     1     1     4     1
 4 Hornet 4 D~   21.4     6  258    110  3.08  3.22  19.4     1     0     3     1
 5 Hornet Spo~   18.7     8  360    175  3.15  3.44  17.0     0     0     3     2
 6 Valiant       18.1     6  225    105  2.76  3.46  20.2     1     0     3     1
 7 Duster 360    14.3     8  360    245  3.21  3.57  15.8     0     0     3     4
 8 Merc 240D     24.4     4  147.    62  3.69  3.19  20       1     0     4     2
 9 Merc 230      22.8     4  141.    95  3.92  3.15  22.9     1     0     4     2
10 Merc 280      19.2     6  168.   123  3.92  3.44  18.3     1     0     4     4
# i 22 more rows
```

### Python

```python
# plotnine's copy already keeps the names in a column; just rename it
cars = mtcars.rename(columns={"name": "model"})
cars
```

```
                 model   mpg  cyl   disp   hp  ...   qsec  vs  am  gear  carb
0            Mazda RX4  21.0    6  160.0  110  ...  16.46   0   1     4     4
1        Mazda RX4 Wag  21.0    6  160.0  110  ...  17.02   0   1     4     4
2           Datsun 710  22.8    4  108.0   93  ...  18.61   1   1     4     1
3       Hornet 4 Drive  21.4    6  258.0  110  ...  19.44   1   0     3     1
4    Hornet Sportabout  18.7    8  360.0  175  ...  17.02   0   0     3     2
5              Valiant  18.1    6  225.0  105  ...  20.22   1   0     3     1
6           Duster 360  14.3    8  360.0  245  ...  15.84   0   0     3     4
7            Merc 240D  24.4    4  146.7   62  ...  20.00   1   0     4     2
8             Merc 230  22.8    4  140.8   95  ...  22.90   1   0     4     2
9             Merc 280  19.2    6  167.6  123  ...  18.30   1   0     4     4
10            Merc 280C 17.8    6  167.6  123  ...  18.90   1   0     4     4
11           Merc 450SE 16.4    8  275.8  180  ...  17.40   0   0     3     3
12           Merc 450SL 17.3    8  275.8  180  ...  17.60   0   0     3     3
13          Merc 450SLC 15.2    8  275.8  180  ...  18.00   0   0     3     3
14   Cadillac Fleetwood 10.4    8  472.0  205  ...  17.98   0   0     3     4
15  Lincoln Continental 10.4    8  460.0  215  ...  17.82   0   0     3     4
```

```
16    Chrysler Imperial  14.7    8  440.0  230  ...  17.42   0   0     3     4
17             Fiat 128  32.4    4   78.7   66  ...  19.47   1   1     4     1
18          Honda Civic  30.4    4   75.7   52  ...  18.52   1   1     4     2
19       Toyota Corolla  33.9    4   71.1   65  ...  19.90   1   1     4     1
20        Toyota Corona  21.5    4  120.1   97  ...  20.01   1   0     3     1
21     Dodge Challenger  15.5    8  318.0  150  ...  16.87   0   0     3     2
22          AMC Javelin  15.2    8  304.0  150  ...  17.30   0   0     3     2
23           Camaro Z28  13.3    8  350.0  245  ...  15.41   0   0     3     4
24     Pontiac Firebird  19.2    8  400.0  175  ...  17.05   0   0     3     2
25            Fiat X1-9  27.3    4   79.0   66  ...  18.90   1   1     4     1
26        Porsche 914-2  26.0    4  120.3   91  ...  16.70   0   1     5     2
27         Lotus Europa  30.4    4   95.1  113  ...  16.90   1   1     5     2
28       Ford Pantera L  15.8    8  351.0  264  ...  14.50   0   1     5     4
29         Ferrari Dino  19.7    6  145.0  175  ...  15.50   0   1     5     6
30        Maserati Bora  15.0    8  301.0  335  ...  14.60   0   1     5     8
31           Volvo 142E  21.4    4  121.0  109  ...  18.60   1   1     4     2

[32 rows x 12 columns]
```

**Note for Python:** pandas' index (see the note above) is exactly the row-names idea, and `mtcars.set_index("name")` would recreate R's original layout. But, as in R, the tidy habit is to keep data in ordinary columns.

## 4.4 Reading and writing data

Data usually lives in files, and the lingua franca of data files is the CSV (comma-separated values). The `readr` package (in the tidyverse) provides `write_csv()` and `read_csv()`:

**R**

```r
# write the milk data next to this script...
write_csv(milk, "week4_milk.csv")

# ...and read it back
milk2 <- read_csv("week4_milk.csv")
```

```
Rows: 4 Columns: 8
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (1): type
dbl (7): calories, protein, fibre, carbs, sugars, fat, sugar_frac
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
milk2
```

```
# A tibble: 4 x 8
  type   calories protein fibre carbs sugars   fat sugar_frac
  <chr>     <dbl>   <dbl> <dbl> <dbl>  <dbl> <dbl>      <dbl>
1 Almond       60       1     1     8      7   2.5      0.875
2 Soy         100       7     2    10     NA   4       NA
3 Oat         120       3    NA    20     19   2.5      0.95
4 Rice        130       1    NA    25     NA   2.5     NA
```

### Python

```
# write the milk data next to this script...
milk.to_csv("week4_milk_py.csv", index=False)

# ...and read it back
milk2 = pd.read_csv("week4_milk_py.csv")
milk2
```

```
     type  calories  protein  fibre  carbs  sugars  fat  sugar_frac
0  Almond        60        1    1.0      8     7.0  2.5       0.875
1     Soy       100        7    2.0     10     NaN  4.0         NaN
2     Oat       120        3    NaN     20    19.0  2.5       0.950
3    Rice       130        1    NaN     25     NaN  2.5         NaN
```

`read_csv()` reads the file, guesses each column's type (it prints its guesses: read them, they catch import bugs early), and returns a tibble. Base R has `read.csv()` (dot, not underscore), which does the same job into a plain data frame, more slowly on large files. Recall from Chapter 1 that a bare filename like `"week4_milk.csv"` is relative to your working directory: if the file cannot be found, `getwd()` (Python: `os.getcwd()`) is the first thing to check.

**Note for Python:** two things in the chunk above. First, the filename: throughout these notes the Python column writes `_py` files (`week4_milk_py.csv`) so the two languages never overwrite each other's output. Second, `index=False`: without it pandas saves the index as an anonymous extra column, which comes back to haunt you at reading time. `pd.read_csv()` guesses column types just like `readr`, but silently; run `.info()` on an import when it matters.

Exercise 4.2: A CSV round trip

Using the `dists` data frame from Exercise 4.1:

a. Write it to `"week4_dists.csv"` with `write_csv()` (Python: to `"week4_dists_py.csv"` with `.to_csv(..., index=False)`).
b. Find the file and open it with a text editor (or click it in the RStudio Files pane and choose *View File*). What does the raw format look like?
c. Read it back with `read_csv()` (Python: `pd.read_csv()`) and compare `str()` (Python: `.info()`) of the original and the copy. Did the logical column survive the trip?
d. Read it once more with base `read.csv()`. What differs? (R only: Python has just the one CSV reader.)

## 4.5 The pipe |> (and method chaining)

Data analysis is a sequence of steps, and code reads best when it is written in the order the steps happen. The **pipe** operator `|>` takes the value on its left and passes it as the *first argument* of the function on its right: programming's version of function composition $(f_2 \circ f_1)(x)$, which we met in Chapter 3. So to compute $\ln(1 + 2 + \cdots + 10)$:

**R**

```
x <- 1:10
x |> sum() |> log()   # same as log(sum(x))
```

```
[1] 4.007333
```

**Python**

```
x = np.arange(1, 11)
np.log(x.sum())       # no pipe in Python: nest the calls, or chain methods
```

```
np.float64(4.007333185232471)
```

Read it left to right: "take `x`, sum it, take the log". Further arguments go in their usual place, after the piped-in first one:

**R**

```r
x |> sum() |> log(base = 10)   # same as log(sum(x), base = 10)
```

```
[1] 1.740363
```

**Python**

```python
np.log10(x.sum())     # numpy's dedicated base-10 logarithm
```

```
np.float64(1.7403626894942439)
```

In the wild you will also meet `%>%`, the older pipe from the `magrittr` package. For our purposes it does the same thing, but write the native `|>`.

The pipe matters this week because every `dplyr` verb takes a data frame as its first argument and returns a data frame: they are *built* to be piped.

**Note for Python:** Python has no pipe operator; the idiom that plays its role is **method chaining**. Every pandas verb below is a *method* that returns a new `DataFrame`, so a pipeline reads left to right as `cars.query(...).assign(...).sort_values(...)`: dots instead of pipes. Wrap a chain in parentheses to split it over several lines, one verb per line; you will see that layout constantly from here on.

## 4.6 Data manipulation with dplyr (and pandas)

`dplyr` provides one verb per manipulation you would want to do to a table:

| Verb | Action | pandas counterpart |
|---|---|---|
| `filter()` | keep a subset of the **rows** | `.query()` / boolean mask |
| `select()` | keep a subset of the **columns** | `[[...]]` double brackets |
| `mutate()` | add new columns computed from existing ones | `.assign()` |
| `arrange()` | reorder the rows | `.sort_values()` |
| `summarise()` | collapse rows down to summary statistics | `.agg()` |

Inside a verb you refer to columns by bare name (no `cars$` prefix needed); `dplyr` knows to look inside the data frame first. (One side effect of loading the tidyverse: its `filter()` masks the base function of the same name. If `filter()` ever behaves strangely, be explicit and call `dplyr::filter()`.)

**Note for Python:** pandas gives you the bare-name comfort only inside `.query()`, whose condition is a *string* evaluated inside the data frame. Elsewhere columns are spelled `cars["mpg"]`. Inside a chain, where the intermediate data frame has no name, `.assign()` accepts functions of it (`lambda df: ...`). Both appear below.

### 4.6.1 filter and select

**R**

```
# rows: frugal cars only
cars |> filter(mpg > 25)
```

```
# A tibble: 6 x 12
  model          mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
  <chr>        <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Fiat 128      32.4     4  78.7    66  4.08  2.2   19.5     1     1     4     1
2 Honda Civic   30.4     4  75.7    52  4.93  1.62  18.5     1     1     4     2
3 Toyota Coro~  33.9     4  71.1    65  4.22  1.84  19.9     1     1     4     1
4 Fiat X1-9     27.3     4  79      66  4.08  1.94  18.9     1     1     4     1
5 Porsche 914~  26       4 120.     91  4.43  2.14  16.7     0     1     5     2
6 Lotus Europa  30.4     4  95.1   113  3.77  1.51  16.9     1     1     5     2
```

```
# conditions combine with the logical operators of week 2
cars |> filter(cyl == 4 & hp > 90)
```

```
# A tibble: 6 x 12
  model          mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
  <chr>        <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Datsun 710    22.8     4 108      93  3.85  2.32  18.6     1     1     4     1
2 Merc 230      22.8     4 141.     95  3.92  3.15  22.9     1     0     4     2
3 Toyota Coro~  21.5     4 120.     97  3.7   2.46  20.0     1     0     3     1
4 Porsche 914~  26       4 120.     91  4.43  2.14  16.7     0     1     5     2
5 Lotus Europa  30.4     4  95.1   113  3.77  1.51  16.9     1     1     5     2
6 Volvo 142E    21.4     4 121     109  4.11  2.78  18.6     1     1     4     2
```

**Python**

```
# rows: frugal cars only
cars.query("mpg > 25")
```

```
             model   mpg  cyl   disp   hp  ...   qsec  vs  am  gear  carb
17        Fiat 128  32.4    4   78.7   66  ...  19.47   1   1     4     1
18     Honda Civic  30.4    4   75.7   52  ...  18.52   1   1     4     2
19  Toyota Corolla  33.9    4   71.1   65  ...  19.90   1   1     4     1
25       Fiat X1-9  27.3    4   79.0   66  ...  18.90   1   1     4     1
26   Porsche 914-2  26.0    4  120.3   91  ...  16.70   0   1     5     2
27    Lotus Europa  30.4    4   95.1  113  ...  16.90   1   1     5     2

[6 rows x 12 columns]
```

```python
# conditions combine with the logical operators of week 2
cars.query("cyl == 4 & hp > 90")
```

```
            model   mpg  cyl   disp   hp  ...   qsec  vs  am  gear  carb
2      Datsun 710  22.8    4  108.0   93  ...  18.61   1   1     4     1
8        Merc 230  22.8    4  140.8   95  ...  22.90   1   0     4     2
20  Toyota Corona  21.5    4  120.1   97  ...  20.01   1   0     3     1
26  Porsche 914-2  26.0    4  120.3   91  ...  16.70   0   1     5     2
27   Lotus Europa  30.4    4   95.1  113  ...  16.90   1   1     5     2
31     Volvo 142E  21.4    4  121.0  109  ...  18.60   1   1     4     2

[6 rows x 12 columns]
```

**R**

```r
# columns: just the ones we care about
cars |> select(model, mpg, hp) |> head(3)
```

```
# A tibble: 3 x 3
  model           mpg    hp
  <chr>         <dbl> <dbl>
1 Mazda RX4      21     110
2 Mazda RX4 Wag  21     110
3 Datsun 710     22.8    93
```

```r
# or everything except some
cars |> select(-vs, -am) |> head(3)
```

```
# A tibble: 3 x 10
  model           mpg   cyl  disp    hp  drat    wt  qsec  gear  carb
```

```
  <chr>          <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4       21       6   160   110  3.9   2.62  16.5     4     4
2 Mazda RX4 Wag   21       6   160   110  3.9   2.88  17.0     4     4
3 Datsun 710      22.8     4   108    93  3.85  2.32  18.6     4     1
```

**Python**

```
# columns: just the ones we care about
cars[["model", "mpg", "hp"]].head(3)
```

```
           model   mpg   hp
0      Mazda RX4  21.0  110
1  Mazda RX4 Wag  21.0  110
2     Datsun 710  22.8   93
```

```
# or everything except some
cars.drop(columns=["vs", "am"]).head(3)
```

```
           model   mpg  cyl   disp   hp  drat     wt   qsec  gear  carb
0      Mazda RX4  21.0    6  160.0  110  3.90  2.620  16.46     4     4
1  Mazda RX4 Wag  21.0    6  160.0  110  3.90  2.875  17.02     4     4
2     Datsun 710  22.8    4  108.0   93  3.85  2.320  18.61     4     1
```

### 4.6.2 mutate and arrange

`mutate()` adds columns. Let's add fuel efficiency in sensible units, and, while we are at it, turn the 0/1 codes into labelled categorical variables. R's type for categorical data is the **factor**: a vector with a fixed set of levels and readable labels, which `summary()` and `ggplot2` treat as labelled groups (discrete colour scales, one box or line per level) rather than as numbers on a continuous axis.

**R**

```
cars <- cars |>
  mutate(
    kpl = mpg * 0.425,                                  # kilometres per litre
    am  = factor(am, labels = c("automatic", "manual")),
    vs  = factor(vs, labels = c("V-shaped", "straight"))
  )
```

**Python**

```
cars = cars.assign(
    kpl = cars["mpg"] * 0.425,                          # kilometres per litre
    am  = pd.Categorical(cars["am"].map({0: "automatic", 1: "manual"})),
    vs  = pd.Categorical(cars["vs"].map({0: "V-shaped", 1: "straight"}))
)
```

**Note for Python:** the factor's pandas counterpart is the **categorical** dtype: `.map()` swaps the 0/1 codes for readable labels, and `pd.Categorical()` marks the result as categorical, which plotnine then treats exactly as ggplot2 treats factors: discrete colour scales, one box or line per level.

`arrange()` sorts rows, ascending by default, descending inside `desc()` (pandas: `ascending=False`). And because every verb pipes into the next, a whole analysis becomes one readable chain:

**R**

```
# the most frugal 4-cylinder cars
cars |>
  filter(cyl == 4) |>
  arrange(desc(kpl)) |>
  select(model, kpl, hp) |>
  head(3)
```

```
# A tibble: 3 x 3
  model            kpl    hp
  <chr>          <dbl> <dbl>
1 Toyota Corolla  14.4    65
2 Fiat 128        13.8    66
3 Honda Civic     12.9    52
```

**Python**

```
# the most frugal 4-cylinder cars
(
  cars
  .query("cyl == 4")
  .sort_values("kpl", ascending=False)
  [["model", "kpl", "hp"]]
  .head(3)
)
```

```
             model      kpl  hp
19  Toyota Corolla  14.4075  65
17        Fiat 128  13.7700  66
18     Honda Civic  12.9200  52
```

Exercise 4.3: dplyr drills

Working on `cars` (the `mtcars` table with the labelled categorical columns from above):

a. Find all manual cars that do more than 25 miles per gallon.
b. Add a column `efficiency = mpg / hp`, miles per gallon per horsepower.
c. Show the `model`, `mpg`, `hp` and `efficiency` of the five most efficient cars.
d. Now do (b) and (c) as a single pipeline, starting from `cars` and using each verb at most once.

### 4.6.3 group_by and summarise

`summarise()` collapses a data frame to summary statistics: `n()` counts the rows, and any vector-to-number function (`mean`, `sd`, `median`, `min`, …) works:

**R**

```r
cars |> summarise(n = n(), mean_mpg = mean(mpg), sd_mpg = sd(mpg))
```

```
# A tibble: 1 x 3
      n mean_mpg sd_mpg
  <int>    <dbl>  <dbl>
1    32     20.1   6.03
```

**Python**

```python
# ungrouped: pick the column, name the summaries
cars["mpg"].agg(n = "size", mean_mpg = "mean", sd_mpg = "std")
```

```
n           32.000000
mean_mpg    20.090625
sd_mpg       6.026948
Name: mpg, dtype: float64
```

On its own this is a glorified `mean()`. The power move is `group_by()`: it splits the data frame into groups by the values of one or more columns, so that `summarise()` produces *one row per group*. Group means and group spreads are the first step of nearly every statistical analysis, and here they cost one pipeline:

**R**

```
cars |>
  group_by(am) |>
  summarise(n = n(), mean_mpg = mean(mpg), sd_mpg = sd(mpg))
```

```
# A tibble: 2 x 4
  am            n mean_mpg sd_mpg
  <fct>     <int>    <dbl>  <dbl>
1 automatic    19     17.1   3.83
2 manual       13     24.4   6.17
```

**Python**

```
(
  cars
  .groupby("am", observed=True)
  .agg(n        = ("mpg", "size"),
       mean_mpg = ("mpg", "mean"),
       sd_mpg   = ("mpg", "std"))
  .reset_index()
)
```

```
          am   n   mean_mpg    sd_mpg
0  automatic  19  17.147368  3.833966
1     manual  13  24.392308  6.166504
```

Manual cars in this dataset do, on average, about 7 more miles per gallon, though with 1974 technology, the causal story is mostly "small cheap cars had manual gearboxes". Grouping by two variables gives one row per combination:

**R**

```
cars |>
  group_by(am, cyl) |>
  summarise(n = n(), mean_mpg = mean(mpg))
```

```
# A tibble: 6 x 4
# Groups:   am [2]
  am           cyl     n mean_mpg
  <fct>      <dbl> <int>    <dbl>
1 automatic      4     3     22.9
2 automatic      6     4     19.1
3 automatic      8    12     15.0
4 manual         4     8     28.1
5 manual         6     3     20.6
6 manual         8     2     15.4
```

**Python**

```
(
  cars
  .groupby(["am", "cyl"], observed=True)
  .agg(n = ("mpg", "size"), mean_mpg = ("mpg", "mean"))
  .reset_index()
)
```

```
          am  cyl   n   mean_mpg
0  automatic    4   3  22.900000
1  automatic    6   4  19.125000
2  automatic    8  12  15.050000
3     manual    4   8  28.075000
4     manual    6   3  20.566667
5     manual    8   2  15.400000
```

**Note for Python:** each `.agg()` pair reads `new_column = (column, function)`: the counterpart of `summarise()`'s `new_column = f(column)`. `groupby()` moves the grouping values into the index, so we finish with `.reset_index()` to get ordinary columns back; and `observed=True` keeps only group combinations that actually occur, which is what `summarise()` does (it also silences a pandas warning about its changing default).

Exercise 4.4: Group statistics on mtcars

a. Group `cars` by number of cylinders and compute, per group: the number of cars, the mean and standard deviation of `mpg`, and the mean of `hp`.
b. Group by transmission *and* cylinders, computing group sizes and mean `mpg`.
c. Compare (b) with the transmission-only summary in the notes: within each cylinder class, is the manual advantage still around 7 mpg? What does that tell you about comparing group means without controlling for other variables?

### 4.6.4 Joining data frames

Real analyses often need to combine two tables that share a common column (a *key*). Suppose we find supermarket prices for some of our milks:

**R**

```r
prices <- tibble(
  type  = c("Almond", "Oat", "Soy"),
  price = c(1.80, 1.50, 1.40)   # £ per litre
)

# match rows of `milk` with rows of `prices` by the shared column `type`
left_join(milk, prices, by = "type")
```

```
    type calories protein fibre carbs sugars fat sugar_frac price
1 Almond       60       1     1     8      7 2.5      0.875   1.8
2    Soy      100       7     2    10     NA 4.0         NA   1.4
3    Oat      120       3    NA    20     19 2.5      0.950   1.5
4   Rice      130       1    NA    25     NA 2.5         NA    NA
```

**Python**

```python
prices = pd.DataFrame({
    "type":  ["Almond", "Oat", "Soy"],
    "price": [1.80, 1.50, 1.40]   # £ per litre
})

# match rows of `milk` with rows of `prices` by the shared column `type`
milk.merge(prices, on="type", how="left")
```

```
     type  calories  protein  fibre  carbs  sugars  fat  sugar_frac  price
0  Almond        60        1    1.0      8     7.0  2.5       0.875    1.8
1     Soy       100        7    2.0     10     NaN  4.0         NaN    1.4
2     Oat       120        3    NaN     20    19.0  2.5       0.950    1.5
3    Rice       130        1    NaN     25     NaN  2.5         NaN    NaN
```

`left_join()` keeps every row of its first argument and attaches the matching information from the second; rows with no match (poor Rice) get `NA` (`NaN` in pandas). There is a whole family (`full_join()`, `inner_join()`, `right_join()`) differing only in which unmatched rows survive. (pandas folds the whole family into `.merge()`'s `how` argument: `"left"`, `"outer"`, `"inner"`, `"right"`.)

## 4.7 Plotting with ggplot2 (and plotnine)

`ggplot2` implements the **grammar of graphics**: instead of memorising one function per chart type, you describe a plot with three ingredients:

- the **data**: a data frame;
- the **aesthetic mapping** `aes()`: which variables map to which visual channels (x-position, y-position, colour, fill, shape);
- the **geometries** `geom_*()`: how the mapped data is drawn (points, lines, bars, boxes).

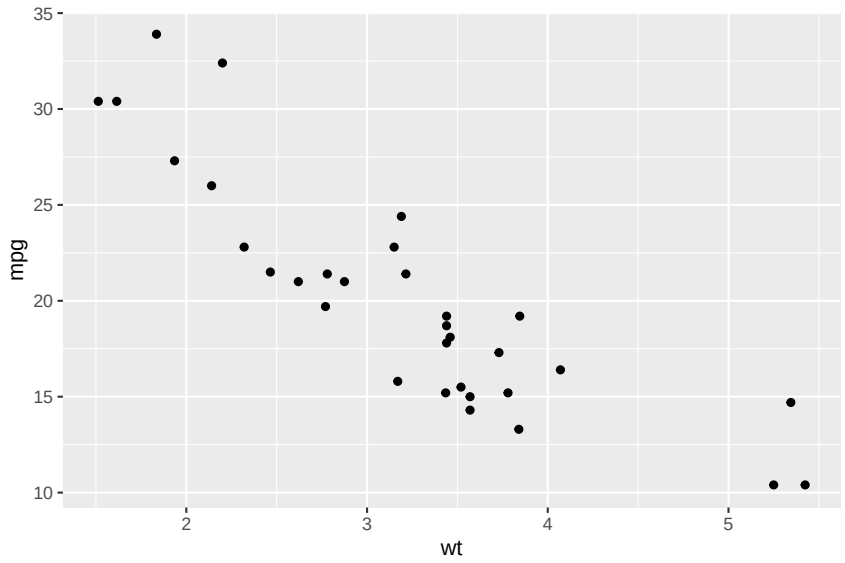
Layers are glued together with `+` (a historical quirk: ggplot2 predates the pipe). Swap the geometry and the same data-to-aesthetics mapping becomes a different plot; that is the whole grammar. Easier done than said:

**Note for Python:** `plotnine` is a faithful port of `ggplot2`: same grammar, same function names, same `+`. Two mechanical differences only: column names inside `aes()` are quoted strings (`aes(x="wt", y="mpg")`) and a plot spread over several lines needs wrapping parentheses (plus `.show()` to display it from a script). Both spellings of `colour`/`color` work, in either language.

### 4.7.1 Scatter plots

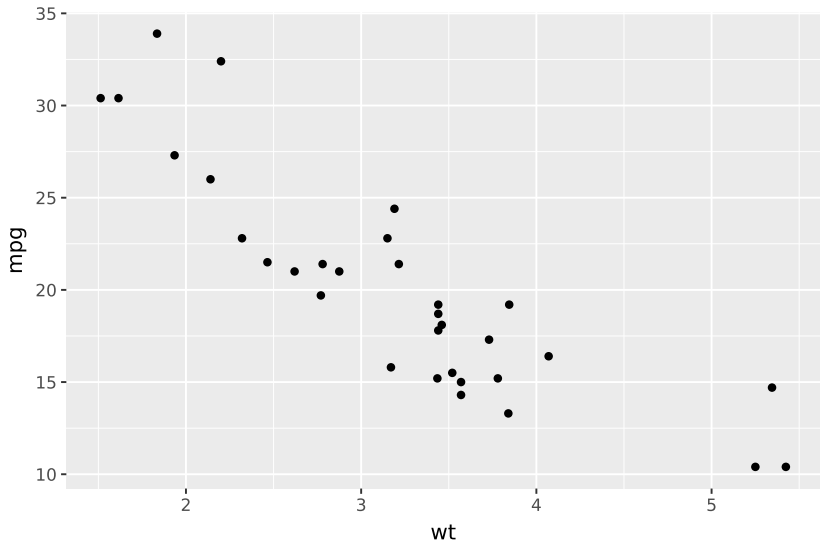
**R**

```
ggplot(cars, aes(x = wt, y = mpg)) +
  geom_point()
```



**Python**

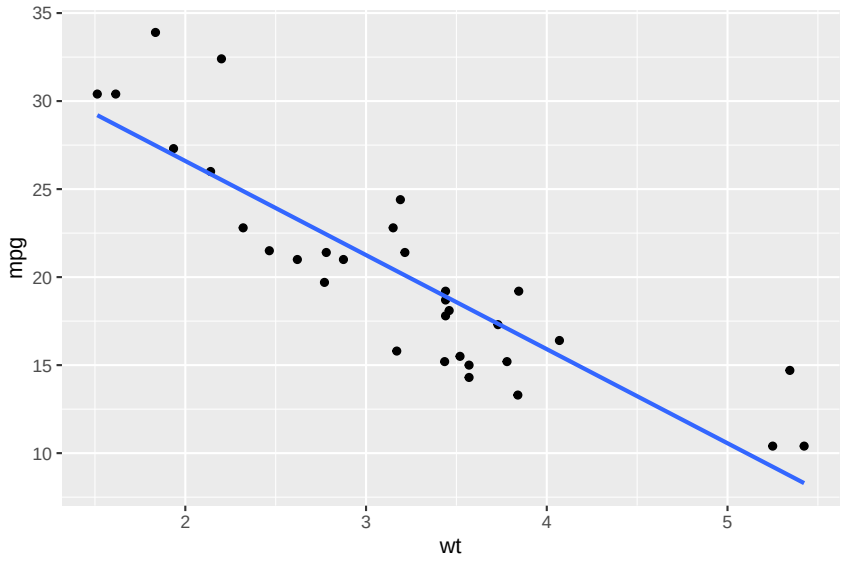
```
(
  ggplot(cars, aes(x="wt", y="mpg")) +
  geom_point()
).show()
```




Dissecting it: `ggplot(cars, ...)` starts a plot from the `cars` data; `aes(x = wt, y = mpg)` maps weight to the x-axis and fuel consumption to the y-axis; `+ geom_point()` draws one point per row. Heavier cars burn more fuel, a relationship crying out for a line through it:

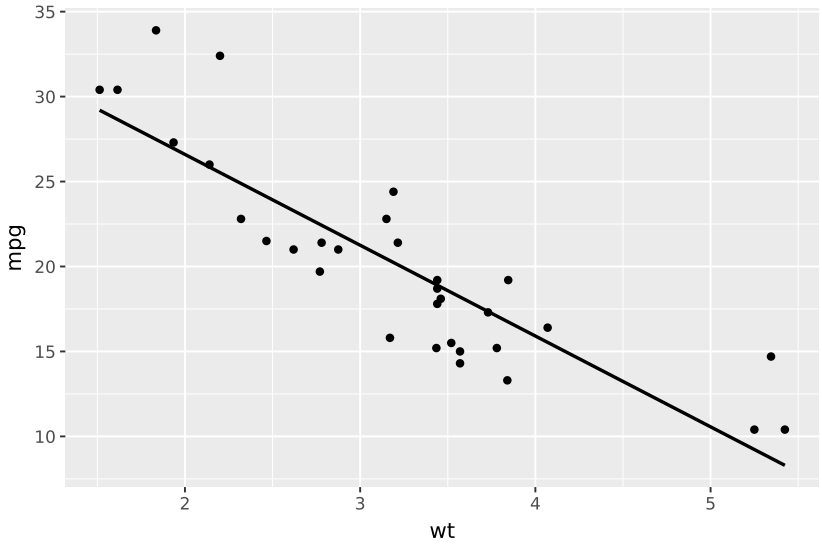
**R**

```
ggplot(cars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```




**Python**

```
(
  ggplot(cars, aes(x="wt", y="mpg")) +
  geom_point() +
  geom_smooth(method="lm", se=False)
).show()
```



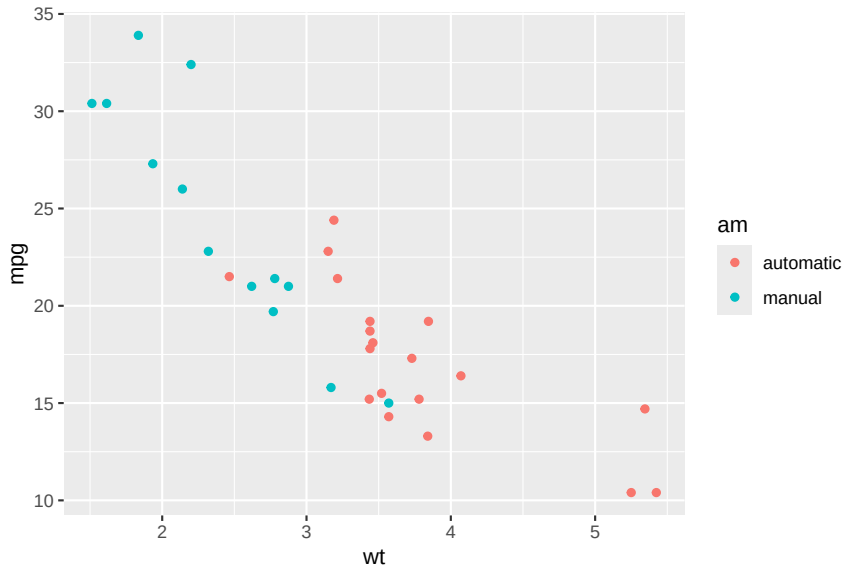

`geom_smooth(method = "lm")` overlays the least-squares regression line: the line minimising the sum of squared vertical distances to the points, the same fit you would get from R's

`lm(mpg ~ wt, data = cars)` (in Python, from `statsmodels`, which is exactly what plotnine's `method="lm"` calls under the hood). Fitting models by minimising a loss is precisely the business of Chapter 5.

To bring in a third variable, map it to a third aesthetic:

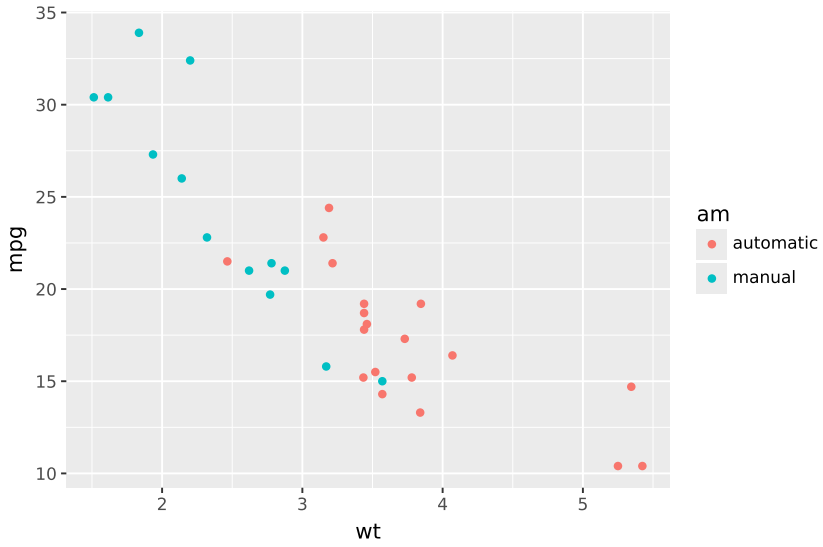
**R**

```
ggplot(cars, aes(x = wt, y = mpg, colour = am)) +
  geom_point()
```




**Python**

```
(
  ggplot(cars, aes(x="wt", y="mpg", colour="am")) +
  geom_point()
).show()
```



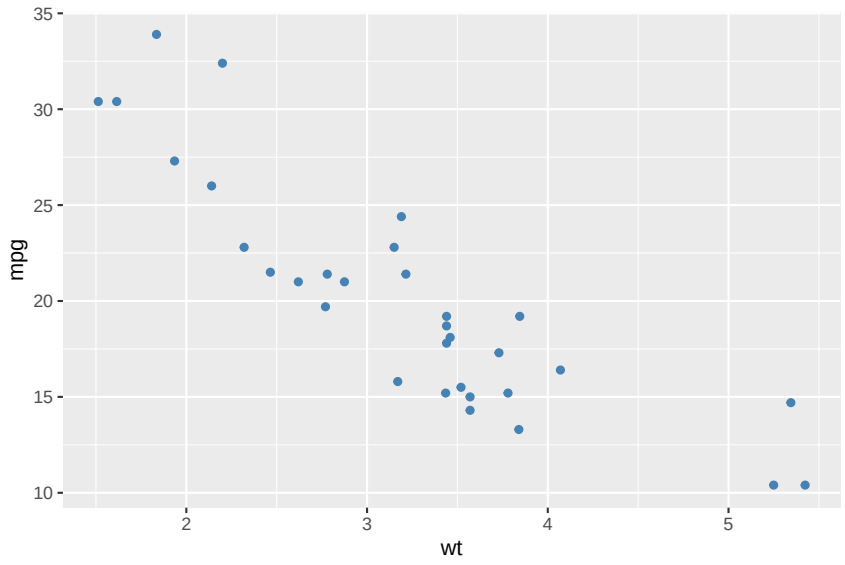

One extra word in `aes()` bought us a coloured grouping *and* a legend.

### 4.7.2 Mapping vs setting aesthetics

There are two ways to control colour, and mixing them up is the most common ggplot bug in existence. **Mapping** (inside `aes()`) ties an aesthetic to a *variable*; **setting** (outside `aes()`, inside the geom) fixes it to a *constant*:

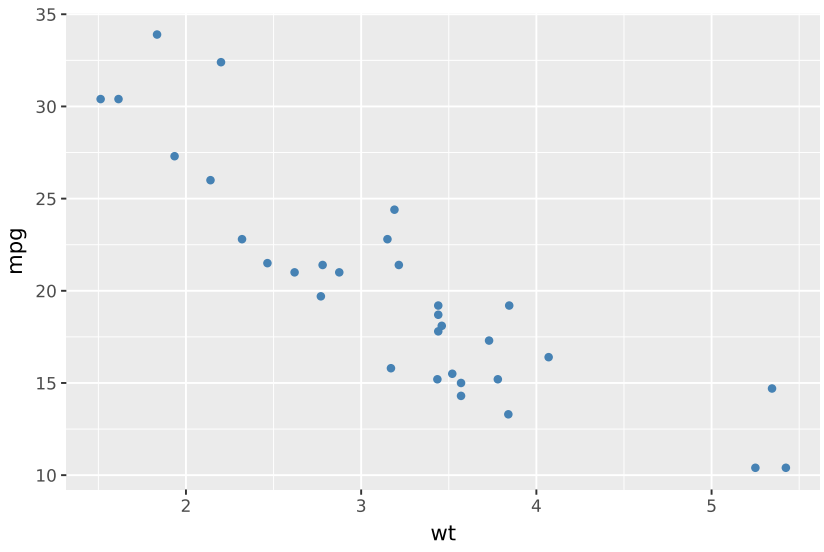
**R**

```
# setting: every point steelblue, no legend (a constant, so OUTSIDE aes())
ggplot(cars, aes(x = wt, y = mpg)) +
  geom_point(colour = "steelblue")
```




**Python**

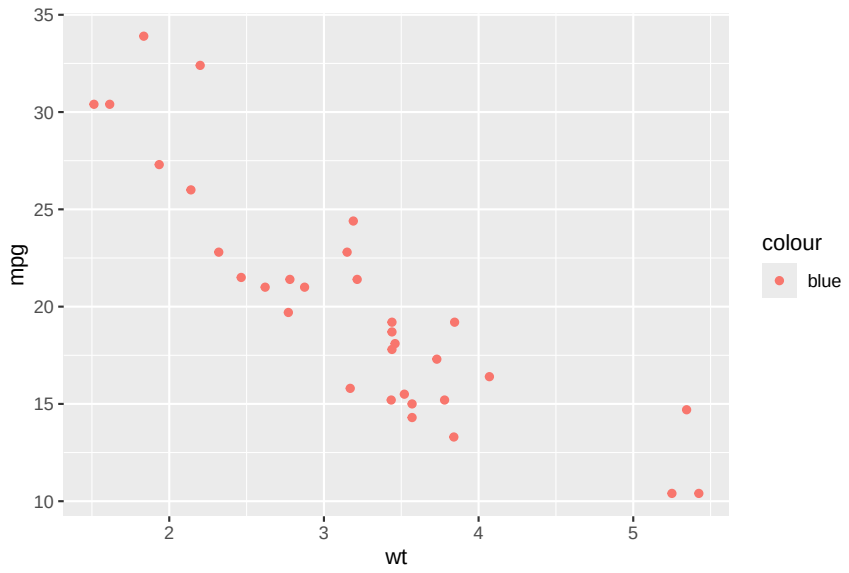
```
# setting: every point steelblue, no legend (a constant, so OUTSIDE aes())
(
  ggplot(cars, aes(x="wt", y="mpg")) +
  geom_point(colour="steelblue")
).show()
```



Put a constant *inside* `aes()` and ggplot does something legal but absurd: it invents a variable whose every value is the string `"blue"`, and dutifully assigns it the first default colour, which is red:

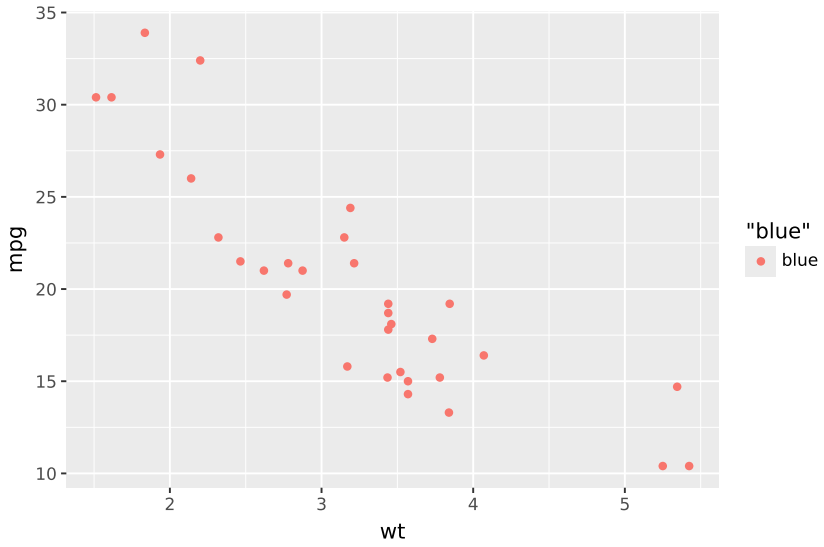
**R**

```
# the classic mistake: "blue" is not a variable
ggplot(cars, aes(x = wt, y = mpg, colour = "blue")) +
  geom_point()
```




**Python**

```
# the classic mistake: "blue" is not a variable
(
  ggplot(cars, aes(x="wt", y="mpg", colour='"blue"')) +
  geom_point()
).show()
```




If your plot sports a legend titled `"blue"` full of red points, you now know why.

**Note for Python:** mind the doubled quotes in `aes(colour='"blue"')`. plotnine evaluates `aes()` strings as expressions, so the inner quotes are what make `"blue"` a constant string: a plain `aes(colour="blue")` would go looking for a *column* called `blue` and raise an error instead of drawing red points. Different failure, same moral: constants do not belong in `aes()`.

Exercise 4.5: Reproduce this scatter plot

Build, from `cars`, the following plot (described in words: your job is to translate it into one ggplot call):

- a scatter plot with gross horsepower `hp` on the x-axis and quarter-mile time `qsec` on the y-axis;
- points coloured by transmission (`am`);
- one least-squares line per transmission group, without the confidence band.

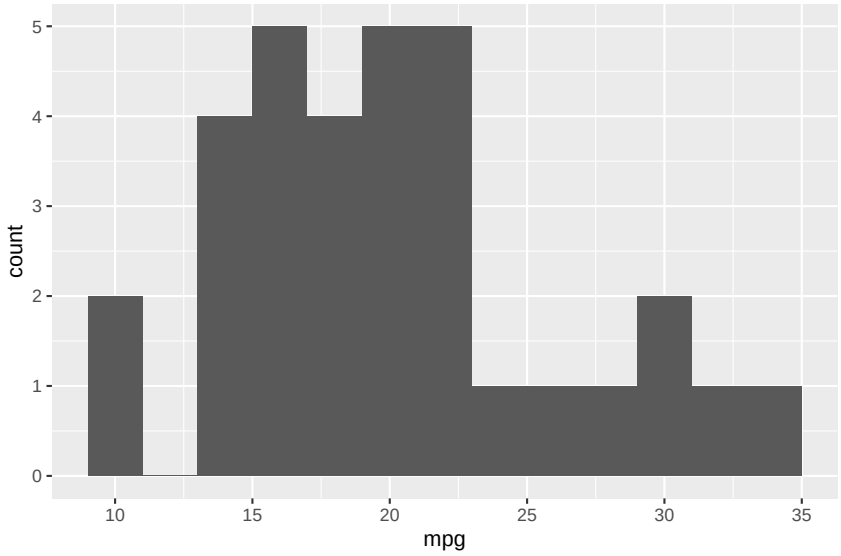
You should find that mapping `colour` automatically makes `geom_smooth()` fit one line per group. Does more horsepower mean a faster quarter mile? Does the relationship differ between transmissions?

### 4.7.3 Histograms and boxplots

The distribution of a single variable needs only an `x` mapping; `geom_histogram()` bins it and draws bar heights:

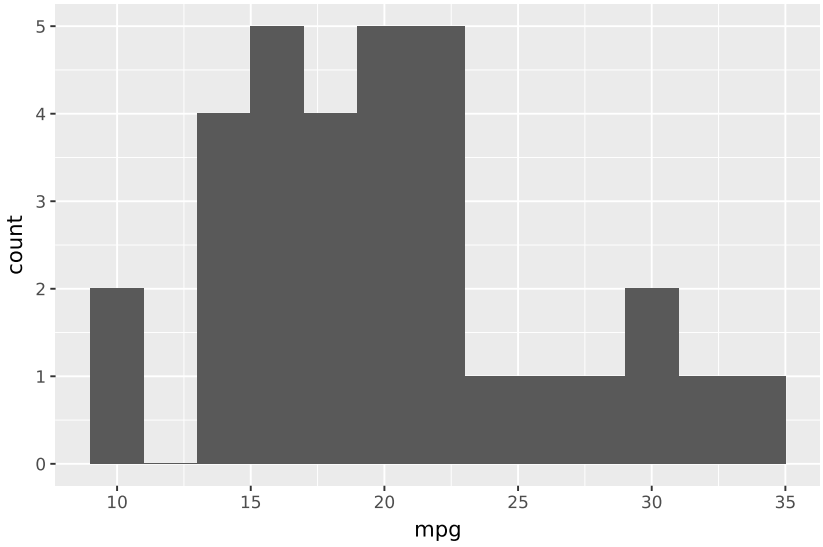
### R

```
ggplot(cars, aes(x = mpg)) +
  geom_histogram(binwidth = 2)
```




### Python

```
(
  ggplot(cars, aes(x="mpg")) +
  geom_histogram(binwidth=2)
).show()
```

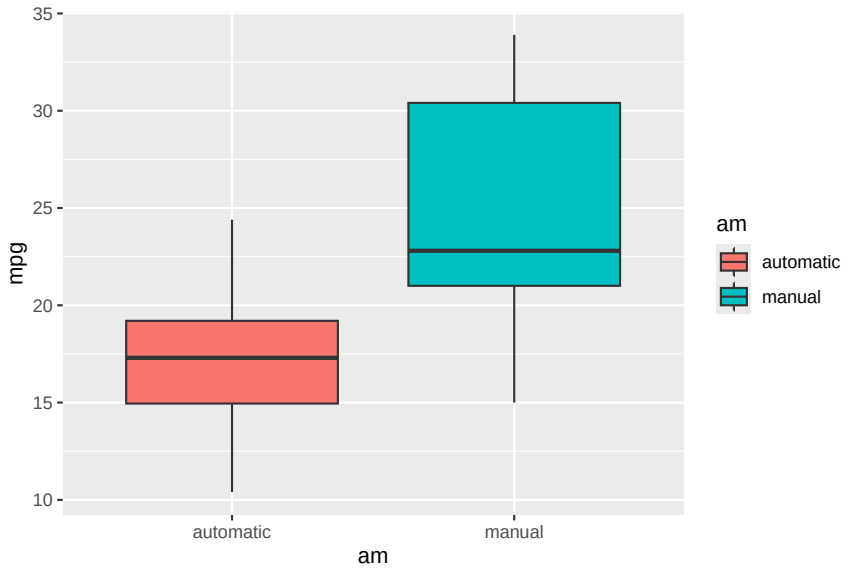



The `binwidth` is your choice, and an important one: too narrow is noise, too wide smooths away every feature. Always try a few values.

For comparing a distribution *across groups*, the boxplot is the tool: a box from the first to the third quartile with the median inside, whiskers spanning the rest of the data, and any outliers as lone points. Map the group to `x` and the variable to `y`; boxes and bars are filled shapes, so their interior colour is the `fill` aesthetic (`colour` is just the outline):

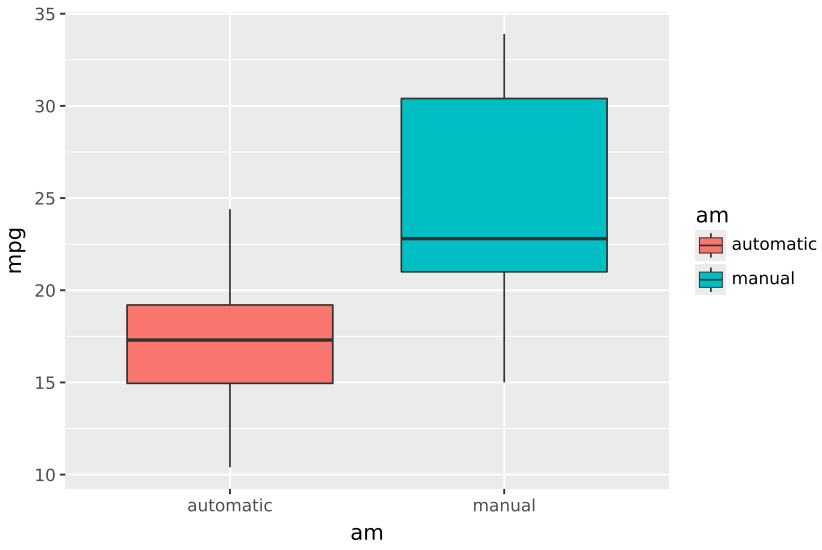
**R**

```
ggplot(cars, aes(x = am, y = mpg, fill = am)) +
  geom_boxplot()
```




### Python

```
(
  ggplot(cars, aes(x="am", y="mpg", fill="am")) +
  geom_boxplot()
).show()
```




The medians echo our `group_by()` summary (manual cars sit visibly higher), but the boxplot also shows the overlap and spread that a table of means quietly hides.

Exercise 4.6: Petal boxplots

The built-in `iris` data frame records sepal and petal measurements for 150 flowers of three species. (Python: `iris` ships with R, not Python. Download `iris.csv` from Moodle into your working directory and load it with `iris = pd.read_csv("iris.csv")`; the R-style column names such as `Petal.Length` carry over, and plotnine accepts them.)

a. Draw boxplots of `Petal.Length` by `Species`, with each box filled in a different colour.
b. Which species is clearly separated from the others? Would a single overall histogram of `Petal.Length` have shown you this?
c. Change `fill` from a mapping to a setting: fill *all* boxes `"orchid"`. Where does `fill = "orchid"` have to go?

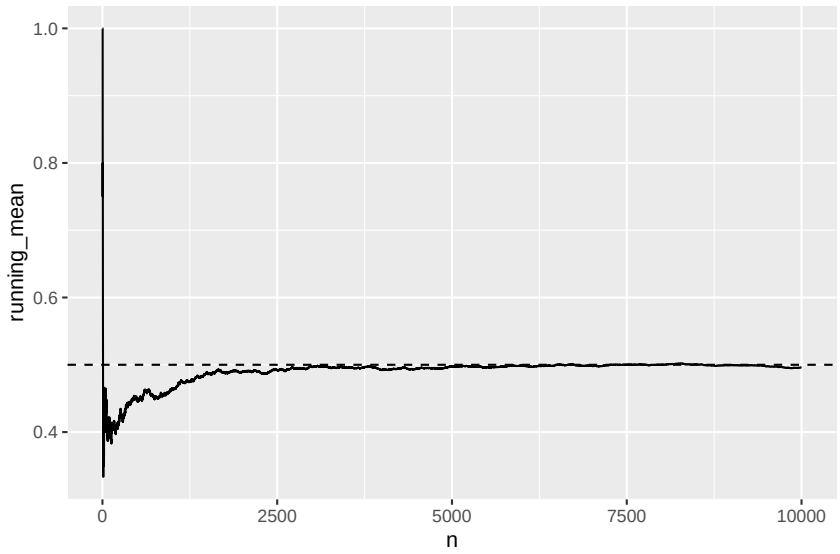
### 4.7.4 Line plots

`geom_line()` connects observations in x-order: the right geometry when x is time, iteration count, or anything sequential. Let's watch the law of large numbers in action: simulate 10000 fair-coin flips (`rbinom(10000, size = 1, prob = 0.5)` in R, `rng.binomial(1, 0.5, size=10000)` in Python; both draw 10000 independent 0/1 values) and plot the running proportion of heads. Since randomness is involved, we set the seed so that each "random" figure is exactly reproducible, though not the *same* figure: R's and numpy's generators produce different streams, so the two columns show two different, equally fair, coin sequences:

**R**

```
set.seed(7432)
lln <- tibble(
  n            = 1:10000,
  running_mean = cumsum(rbinom(10000, size = 1, prob = 0.5)) / n
)

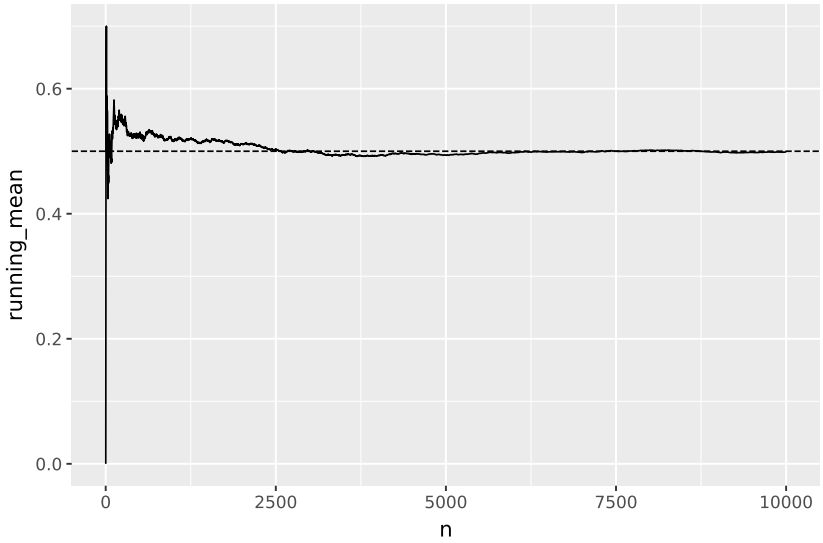
ggplot(lln, aes(x = n, y = running_mean)) +
  geom_line() +
  geom_hline(yintercept = 0.5, linetype = "dashed")
```




### Python

```
rng = np.random.default_rng(7432)
n = np.arange(1, 10001)
lln = pd.DataFrame({
    "n":            n,
    "running_mean": np.cumsum(rng.binomial(1, 0.5, size=10000)) / n
})

(
  ggplot(lln, aes(x="n", y="running_mean")) +
  geom_line() +
  geom_hline(yintercept=0.5, linetype="dashed")
).show()
```




In both runs the wild early swings settle onto the dashed line at $1/2$: the sample mean converging to the expectation, slowly, as the fluctuations only shrink like $1/\sqrt{n}$. (`geom_hline()` draws a horizontal reference line; note how it stacks onto the plot as just another layer.) In Chapter 5 the very same plot template (value against iteration) will track optimisation algorithms as they converge.

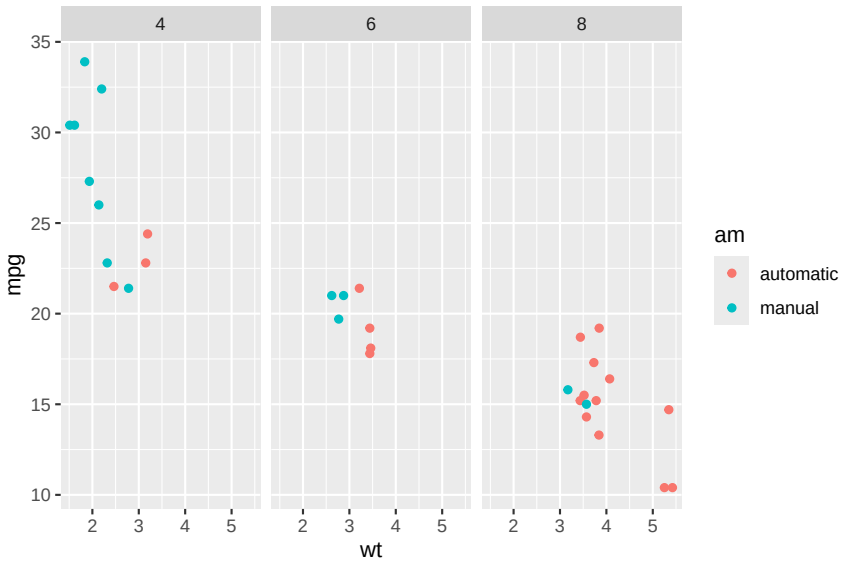
**Note for Python:** watch the argument names: they swap roles across the languages. R's `rbinom(10000, size = 1, prob = 0.5)` asks for 10000 draws of `size = 1` trial each; numpy's `rng.binomial(1, 0.5, size=10000)` says the same thing, but now `size` counts the *draws*. Copying arguments across languages by name, without reading, is how "10000 flips of one coin" silently becomes "one flip of 10000 coins".

### 4.7.5 Facets

Colour handles a third variable; a fourth would get crowded. **Faceting** splits one plot into a panel per value of a variable, with shared axes so panels compare fairly:

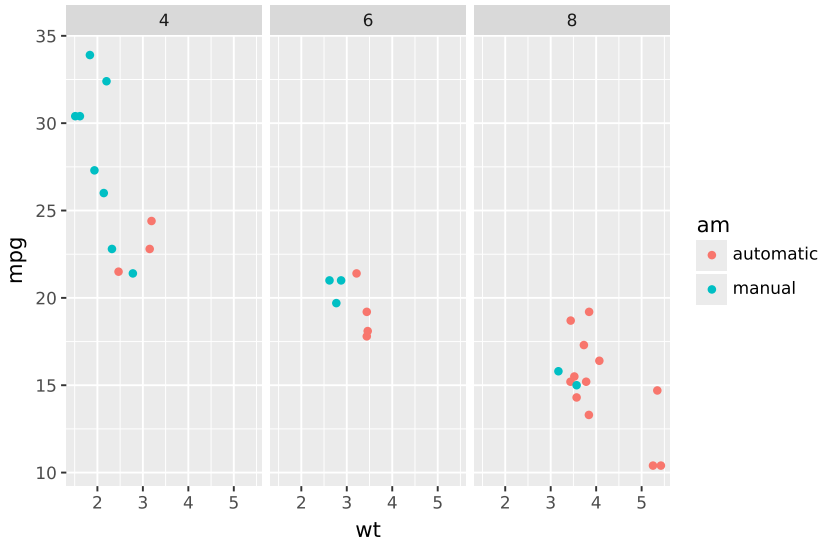
**R**

```
ggplot(cars, aes(x = wt, y = mpg, colour = am)) +
  geom_point() +
  facet_wrap(~ cyl)
```




### Python

```
(
  ggplot(cars, aes(x="wt", y="mpg", colour="am")) +
  geom_point() +
  facet_wrap("~cyl")
).show()
```




`facet_wrap(~ cyl)` means "one panel per value of `cyl`" (the `~` is R's formula syntax; you will meet it again in regression). Arrange the panels with `nrow` or `ncol`, e.g. `facet_wrap(~ cyl,`

`ncol = 1`) stacks them vertically, which is ideal for comparing distributions along a shared x-axis.

**Note for Python:** plotnine takes the formula as a string (`facet_wrap("~cyl")`, or just `facet_wrap("cyl")`), with the same `nrow`/`ncol` arguments.

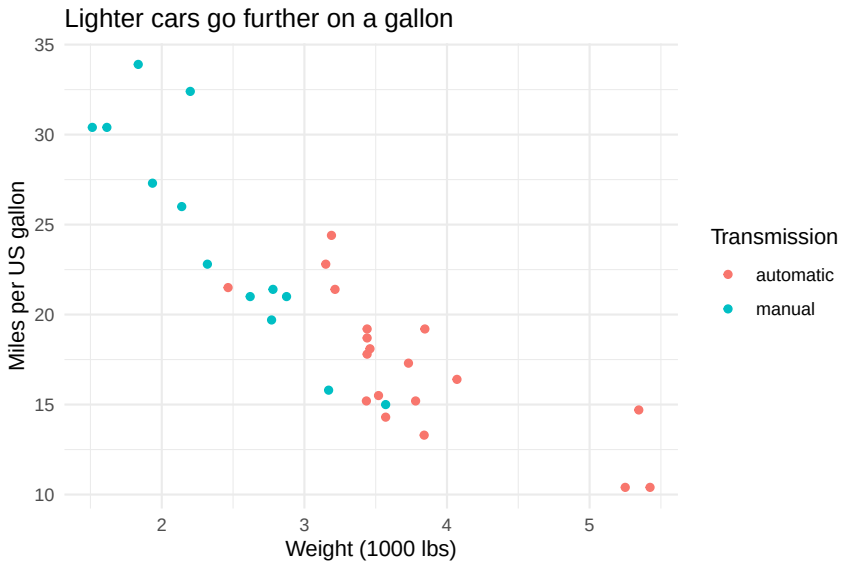
> Exercise 4.7: Facet the histograms
>
> Back to `iris`: draw histograms of `Petal.Length` (pick a sensible `binwidth`), faceted by `Species`, with the three panels stacked in a single column. Why does stacking them vertically (rather than side by side) make the species easier to compare? How does this figure relate to the boxplots of Exercise 4.6?

### 4.7.6 Labels, themes and saving

Everything so far was exploratory: quick plots for you. A figure for a report needs proper labels (nobody downstream knows what `wt` means) and a cleaner look. `labs()` names every mapped aesthetic plus the title, and `theme_minimal()` swaps the grey default for something publication-friendly (we will use it from now on):

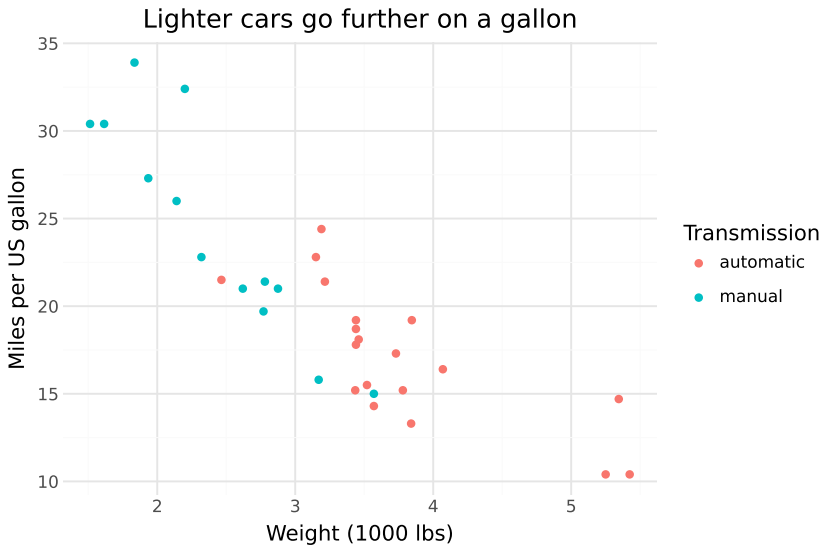
**R**

```r
p <- ggplot(cars, aes(x = wt, y = mpg, colour = am)) +
  geom_point() +
  labs(
    title  = "Lighter cars go further on a gallon",
    x      = "Weight (1000 lbs)",
    y      = "Miles per US gallon",
    colour = "Transmission"
  ) +
  theme_minimal()
p
```




### Python

```
p = (
  ggplot(cars, aes(x="wt", y="mpg", colour="am")) +
  geom_point() +
  labs(
      title  = "Lighter cars go further on a gallon",
      x      = "Weight (1000 lbs)",
      y      = "Miles per US gallon",
      colour = "Transmission"
  ) +
  theme_minimal()
)
p.show()
```




Note we stored the plot in a variable: plots are ordinary objects in both languages, so you can print them, add more layers later (`p + facet_wrap(~ cyl)`), or save them to a file with `ggsave()` (in plotnine, the plot's `.save()` method):

**R**

```r
ggsave("week4_mpg_wt.png", p, width = 6, height = 4)  # inches
```

**Python**

```python
p.save("week4_mpg_wt_py.png", width=6, height=4, verbose=False)  # inches
```

Change the extension to get a different format (`.pdf` scales beautifully in documents). If a figure looks wrong in your report, resize it here, in code, never by dragging its corner in a word processor.

## 4.8 Challenges

**!** Challenge 4.1: The full pipeline

Time to run the whole week end to end: file → table → wrangle → summarise → figure → file. First create the raw data file by running:

**R**

```r
write_csv(as_tibble(mtcars, rownames = "model"), "week4_cars.csv")
```

**Python**

```python
mtcars.rename(columns={"name": "model"}).to_csv("week4_cars_py.csv", index=False)
```

Now, starting from that CSV (pretend someone emailed it to you):

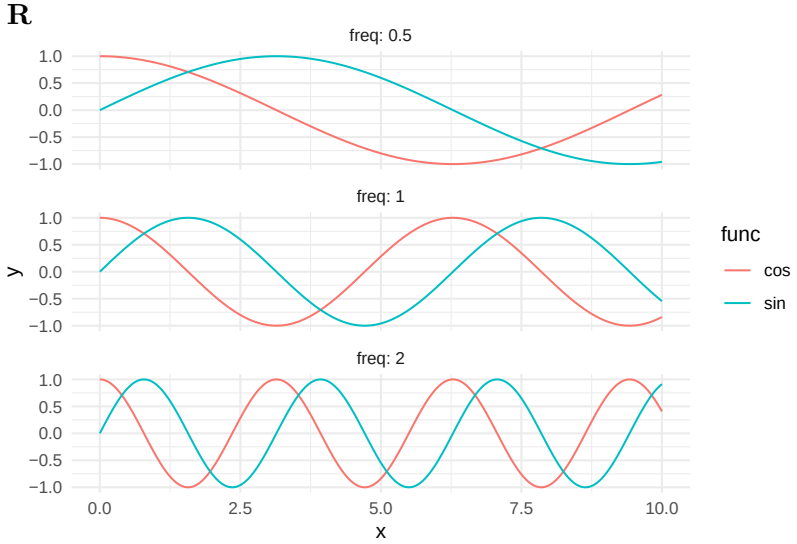
a. Read `week4_cars.csv` (Python: `week4_cars_py.csv`) with `read_csv()` (Python: `pd.read_csv()`).
b. In one `mutate()` (Python: one `.assign()`): convert `am` to a factor (Python: categorical) labelled `"automatic"`/`"manual"`, `vs` to one labelled `"V-shaped"`/`"straight"`, and add `efficiency = mpg / hp`.
c. Produce a per-`cyl` summary table: number of cars, mean and standard deviation of `mpg`.
d. Draw a scatter plot of `hp` against `mpg`, coloured by transmission, faceted by `cyl`, with a least-squares line per transmission (no confidence band), full labels, and `theme_minimal()`.
e. Save the figure to `week4_analysis.png` (Python: `week4_analysis_py.png`) with `ggsave()` (Python: `.save()`).

Bonus points if (a)–(b) happen in a single pipeline.

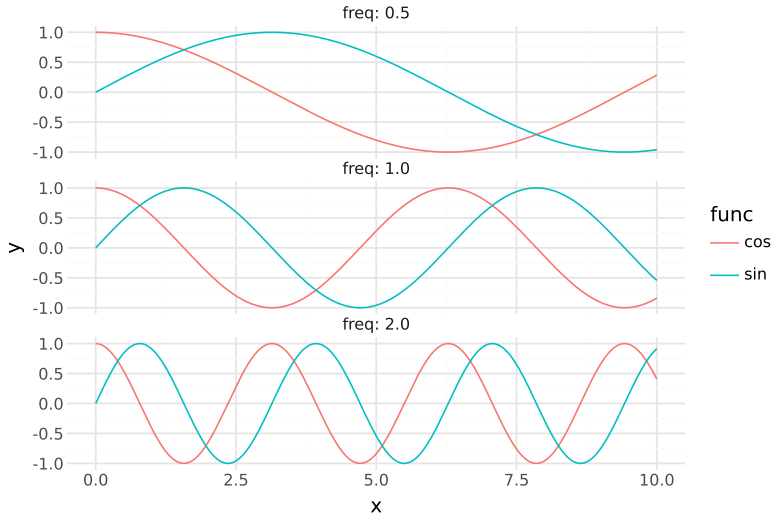
**!** Challenge 4.2: Waves, functionally

A plotting challenge that resurrects Chapter 3. Reproduce this figure:

**R**



**Python**




That is six curves ($\sin(fx)$ and $\cos(fx)$ for frequencies $f \in \{0.5, 1, 2\}$ over $x \in [0, 10]$) in one data frame with columns `x`, `y`, `func` and `freq`, plotted with colour mapped to the function and one facet per frequency (`labeller = label_both` prints `freq: 0.5` etc. in the panel strips).

The rules: no copy-pasting six blocks of data-frame code, and no `for` loops. Write a lambda that builds the data frame for one frequency, `map()` it over the three frequencies (Python: a list comprehension does the same job), and stitch the results with `bind_rows()` (Python: `pd.concat()`), both of which happily accept a list of data frames.

That completes the toolbox: data structures, control flow, functions, and now data wrangling and graphics. Next week (Chapter 5) we put everything to work!

# 5 Week 5: Introduction to Optimisation for Statistics

In Chapter 2 we met gradient descent in one dimension, in Chapter 3 we wrapped it into a reusable function, and in Chapter 4 we learnt to draw pictures of what our code is doing. This week we put those tools to work on the problem that underlies an enormous amount of modern statistics: **Fitting a model is minimising a function.**

Maximum likelihood, least squares, penalised regression, neural networks: behind each of them sits an optimisation problem, and behind most of the algorithms that solve those problems sits some descendant of gradient descent. By the end of this chapter you will have met the whole family: plain gradient descent, its stochastic cousin, preconditioning, and the quasi-Newton method BFGS that the built-in optimisers of both languages (R's `optim()` and Python's `scipy.optimize.minimize()`) use in practice.

## 5.1 Gradient descent, again

Quick recap. We want to minimise a differentiable function $f : \mathbb{R}^d \to \mathbb{R}$. The gradient $\nabla f(\mathbf{x})$ points in the direction of steepest *increase*, so we repeatedly take a small step the other way:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \gamma \nabla f(\mathbf{x}_n),$$

where $\gamma > 0$ is the **step size** (machine learning people say *learning rate*). In Chapter 3 we packaged this into `grad_descent(grad, x0, gamma, n)`. Algorithm 5.1 states the method formally; the code adds only the bookkeeping that records the trajectory (instead of returning only the final point, it records the whole trajectory, because this week we want to *watch* the algorithm work).

Algorithm 5.1: Gradient descent

**Input:** the gradient $\nabla f$ of an objective $f : \mathbb{R}^d \to \mathbb{R}$, start point $\mathbf{x}_0$, step size $\gamma > 0$, number of steps $n_{\text{steps}}$ (the code's `n`).
**Output:** the trajectory $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{n_{\text{steps}}}$; the final iterate approximates the minimiser.

1. **for** $t = 1, 2, \dots, n_{\text{steps}}$:

a. $\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \gamma \nabla f(\mathbf{x}_{t-1})$

2. **return** $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{n_{\text{steps}}}$

**R**

```r
grad_descent <- function(grad, x0, gamma, n) {
  # one row per iterate, one column per coordinate
  path <- matrix(NA_real_, nrow = n + 1, ncol = length(x0))
  path[1, ] <- x0
  for (i in 1:n) {
    path[i + 1, ] <- path[i, ] - gamma * grad(path[i, ])
  }
  # return a data frame ready for ggplot: one row per iterate
  out <- as.data.frame(path)
  names(out) <- paste0("x", seq_along(x0))
  out$iter <- 0:n
  out
}
```

**Python**

```python
import numpy as np
import pandas as pd

def grad_descent(grad, x0, gamma, n):
    # one row per iterate, one column per coordinate
    path = np.full((n + 1, len(x0)), np.nan)
    path[0] = x0
    for i in range(n):
        path[i + 1] = path[i] - gamma * grad(path[i])
    # return a data frame ready for plotting: one row per iterate
    out = pd.DataFrame(path, columns=[f"x{j + 1}" for j in range(len(x0))])
    out["iter"] = range(n + 1)
    return out
```

**Note for Python:** the R loop runs `i = 1, ..., n` and fills rows `2, ..., n + 1`; the Python loop runs `i = 0, ..., n - 1` and fills rows `1, ..., n`: the same trajectory, one index base apart (Chapter 1's indexing note at work).

Note the functional style from Chapter 3: `grad` is an *argument*. The same dozen lines will minimise a toy parabola now and a logistic regression likelihood later, without changing a character.

Let us try it on a friendly two-dimensional bowl,

$$f(x_1, x_2) = (x_1 - 1)^2 + 2\,(x_2 + 1)^2, \qquad \nabla f(x_1, x_2) = \begin{pmatrix} 2(x_1 - 1) \\ 4(x_2 + 1) \end{pmatrix},$$

whose minimum sits at $(1, -1)$.

**R**

```
f_bowl <- \(x) (x[1] - 1)^2 + 2 * (x[2] + 1)^2
grad_bowl <- \(x) c(2 * (x[1] - 1), 4 * (x[2] + 1))

path_bowl <- grad_descent(grad_bowl, x0 = c(-2, 2), gamma = 0.1, n = 60)
round(tail(path_bowl, 3), 5)
```

```
        x1 x2 iter
59 0.99999 -1   58
60 0.99999 -1   59
61 1.00000 -1   60
```

**Python**

```
f_bowl = lambda x: (x[0] - 1)**2 + 2 * (x[1] + 1)**2
grad_bowl = lambda x: np.array([2 * (x[0] - 1), 4 * (x[1] + 1)])

path_bowl = grad_descent(grad_bowl, x0=np.array([-2, 2]), gamma=0.1, n=60)
path_bowl.tail(3).round(5)
```

```
         x1   x2  iter
58  0.99999 -1.0    58
59  0.99999 -1.0    59
60  1.00000 -1.0    60
```

Sixty steps and we are sitting on the minimum. To *see* the descent we plot the function as a **contour plot**: evaluate $f$ on a grid (`expand.grid()` in R, `np.meshgrid()` in Python), then hand the grid to `geom_contour()`. Since we will need many of these grids, we write a small helper for that step (again: Chapter 3 habits paying off). Note that this is the first place the objective $f$ is actually evaluated: the plot needs it, the descent never did.

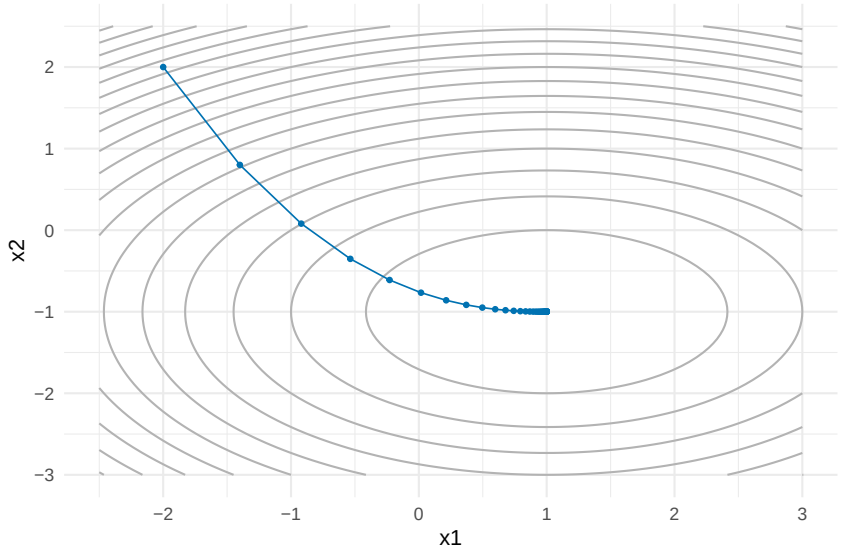
**Note for Python:** plotnine, ggplot2's Python twin from Chapter 4, has no `geom_contour()`. Every contour figure this week therefore falls back on matplotlib's `plt.contour()` (same pictures, different engine), and the faceted contour figures become `plt.subplots()` panels.

### R

```
library(ggplot2)

# evaluate f on a grid, ready for geom_contour
contour_data <- function(f, xlim, ylim, n = 100) {
  grid <- expand.grid(x1 = seq(xlim[1], xlim[2], length.out = n),
                      x2 = seq(ylim[1], ylim[2], length.out = n))
  grid$f <- apply(grid[, 1:2], 1, f)
  grid
}

# grey contours of f, with the trajectory drawn on top
ggplot(contour_data(f_bowl, c(-2.5, 3), c(-3, 2.5)), aes(x1, x2)) +
  geom_contour(aes(z = f), bins = 20, colour = "grey70") +
  geom_path(data = path_bowl, colour = "#0072B2", linewidth = 0.4) +
  geom_point(data = path_bowl, colour = "#0072B2", size = 0.9) +
  theme_minimal()
```



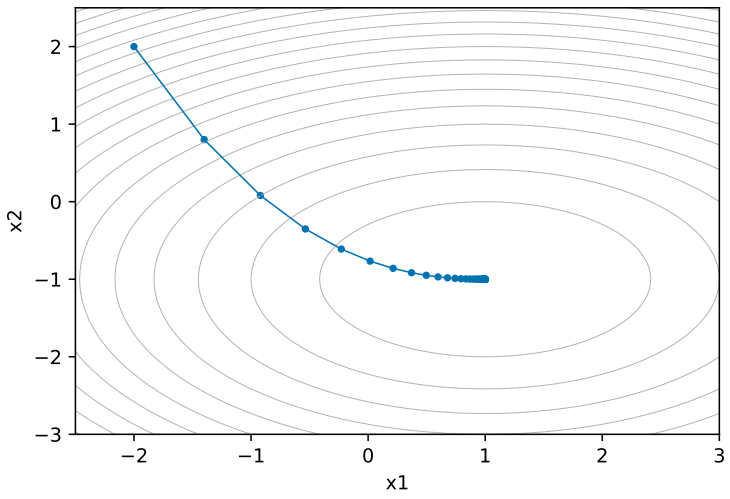

### Python

```
import matplotlib.pyplot as plt

# Unfortunately geom_contour does not work on plotnine!
# There are alternatives using matplotlib.
```

```
# evaluate f on a grid, ready for plt.contour
def contour_data(f, xlim, ylim, n=100):
    x1, x2 = np.meshgrid(np.linspace(xlim[0], xlim[1], n),
                         np.linspace(ylim[0], ylim[1], n))
    f_grid = np.array([f(np.array([a, b]))
                       for a, b in zip(x1.ravel(), x2.ravel())])
    return x1, x2, f_grid.reshape(x1.shape)

# grey contours of f, with the trajectory drawn on top
x1, x2, f_grid = contour_data(f_bowl, (-2.5, 3), (-3, 2.5))
fig, ax = plt.subplots()
ax.contour(x1, x2, f_grid, levels=20, colors="0.7", linewidths=0.5)
ax.plot(path_bowl["x1"], path_bowl["x2"], color="#0072B2", linewidth=0.8)
ax.scatter(path_bowl["x1"], path_bowl["x2"], color="#0072B2", s=8)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
plt.show()
```




The path marches straight into the bowl. Lovely.

### 5.1.1 Estimation is optimisation

So far $f$ has been a toy. Here is why statisticians care. A huge share of estimators are defined as the minimiser of an **average loss over the data**:

$$\hat{\theta} = \arg\min_{\theta} L(\theta), \qquad L(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\theta),$$

where $\ell_i(\theta)$ measures how badly the parameter $\theta$ explains observation $i$. You have met one member of this family already: least squares, with $\ell_i(\beta) = \frac{1}{2}(y_i - \mathbf{x}_i^\top \beta)^2$ and gradient $\nabla L(\beta) = \frac{1}{n} X^\top (X\beta - \mathbf{y})$. Least squares is also the family's great exception: setting its gradient to zero gives the normal equations $X^\top X\beta = X^\top \mathbf{y}$, a linear system that `solve()` (`np.linalg.solve()` in Python) dispatches in one line, with no iteration in sight. That is exactly the magic formula $\hat{\beta} = (X^\top X)^{-1} X^\top \mathbf{y}$ you used in Challenge 1.1 of Chapter 1, and the reason this chapter spends no more time on it. Almost no other model is so lucky. Our running example is the everyday kind, where no closed form exists and iterating is the only way down.

**Logistic regression (negative log-likelihood).** For binary $y_i \in \{0, 1\}$ the model is $\Pr(y_i = 1 \mid \mathbf{x}_i) = \sigma(\eta_i)$, where $\eta_i = \mathbf{x}_i^\top \beta$ is the **linear predictor** (the code below calls it `eta`) and $\sigma(z) = 1/(1 + e^{-z})$ is the **sigmoid** function, which squashes the real line into $(0, 1)$ so that $\eta_i$, which can be any real number, becomes a probability. Taking $-\log$ of the likelihood and averaging over the observations gives the loss; differentiating the loss gives the gradient:

$$L(\beta) = \frac{1}{n}\sum_{i=1}^{n} \Big[ \log(1 + e^{\mathbf{x}_i^\top \beta}) - y_i \, \mathbf{x}_i^\top \beta \Big], \qquad \nabla L(\beta) = \frac{1}{n} X^\top (\sigma(X\beta) - \mathbf{y}),$$

with the rows $\mathbf{x}_i^\top$ stacked into the design matrix $X$ and $\sigma$ applied elementwise. These two formulas are all the code will need. The gradient is worth reading aloud once: observation $i$ contributes $\nabla \ell_i(\beta) = (\sigma(\eta_i) - y_i)\, \mathbf{x}_i$, predicted probability minus observed outcome (a residual on the probability scale) times its own covariates, so observations the model already gets right contribute almost nothing while confidently wrong ones push hard; this per-observation piece is also exactly the single-sample gradient that SGD will draw at random in Section 5.1.2. Note how the least-squares gradient at the top of this section, $\frac{1}{n} X^\top (X\beta - \mathbf{y})$, has the same shape: both say "$X^\top$ times (prediction minus data)", the fingerprint of the *generalised linear model* family that R's `glm()` is named after. Tidy as the gradient is, setting it to zero gives equations with no closed-form solution: you *must* optimise numerically. This is the everyday reality of maximum likelihood.

If you're wondering where the two formulas come from, the proof below is **optional**. It is worth reading once in your life, though, in particular if you are planning to continue with your studies. This derivation you will redo, with a different likelihood each time, for every model you ever fit by maximum likelihood.

[OPTIONAL] Proof: from likelihood to loss to gradient

*Step 1* We write down the negative log-likelihood. Observation $i$ is a Bernoulli trial with success probability $p_i = \sigma(\eta_i)$, so its probability mass is $p_i^{y_i}(1-p_i)^{1-y_i}$ (check it: this equals $p_i$ when $y_i = 1$ and $1-p_i$ when $y_i = 0$). Independence multiplies the contributions, $-\log$ turns the product into a sum, and dividing by $n$ gives the average-loss form:

$$L(\beta) = -\frac{1}{n}\log\prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{1-y_i} = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big].$$

The parameters hide inside $p_i = \sigma(\eta_i)$, so before differentiating we rewrite everything in terms of $\eta_i$. Multiplying the top and bottom of $\sigma(\eta) = 1/(1+e^{-\eta})$ by $e^{\eta}$ gives $\sigma(\eta) = e^{\eta}/(1+e^{\eta})$, hence $1-\sigma(\eta) = 1/(1+e^{\eta})$, and taking logs:

$$\log\sigma(\eta) = \eta - \log\left(1+e^{\eta}\right), \qquad \log\left(1-\sigma(\eta)\right) = -\log\left(1+e^{\eta}\right).$$

Substituting both into the $i$-th summand $\ell_i$ of $L$:

$$\ell_i(\beta) = -y_i\big[\eta_i - \log\left(1+e^{\eta_i}\right)\big] + (1-y_i)\log\left(1+e^{\eta_i}\right) = \log\left(1+e^{\eta_i}\right) - y_i\,\eta_i,$$

the two logarithm terms merging because their coefficients $y_i$ and $1-y_i$ sum to one. Averaging the $\ell_i$:

$$L(\beta) = \frac{1}{n}\sum_{i=1}^{n}\Big[\log(1+e^{\mathbf{x}_i^\top\beta}) - y_i\,\mathbf{x}_i^\top\beta\Big].$$

*Step 2.* We differentiate, by first noting that the derivative of $\log(1+e^{\eta})$ (machine learning people call this function the *softplus*), is the sigmoid again:

$$\frac{d}{d\eta}\log\left(1+e^{\eta}\right) = \frac{e^{\eta}}{1+e^{\eta}} = \sigma(\eta).$$

Second, we note that $\eta_i = \mathbf{x}_i^\top\beta$ is linear in $\beta$ with $\nabla_\beta\,\eta_i = \mathbf{x}_i$, so a term $g(\eta_i)$ contributes $g'(\eta_i)\,\mathbf{x}_i$ to the gradient. Applying both to $\ell_i = \log(1+e^{\eta_i}) - y_i\eta_i$:

$$\nabla\ell_i(\beta) = \sigma(\eta_i)\,\mathbf{x}_i - y_i\,\mathbf{x}_i = \big(\sigma(\eta_i) - y_i\big)\,\mathbf{x}_i.$$

Averaging over the sample and stacking the rows $\mathbf{x}_i^\top$ into the design matrix $X$ packs the sum into one matrix product:

$$\nabla L(\beta) = \frac{1}{n}\sum_{i=1}^{n}\big(\sigma(\eta_i) - y_i\big)\,\mathbf{x}_i = \frac{1}{n}X^\top\big(\sigma(X\beta) - \mathbf{y}\big),$$

with $\sigma$ applied elementwise: exactly the two formulas claimed above. □

**Example.** Let us simulate a dataset to fit. We fix the random seed first (`set.seed()` in R, a seeded generator `np.random.default_rng()` in Python) so that you get the same data, plots

and fitted values as these notes.

**Note:** Remember that the two languages cannot share a random stream, so we will get slightly different results.

**R**

```
set.seed(7432)

# --- logistic regression data: intercept + one covariate --------------------
n_log <- 500
x_log <- rnorm(n_log)
X_log <- cbind(1, x_log)                  # design matrix: intercept + slope
beta_log_true <- c(-0.5, 2)
p_true <- 1 / (1 + exp(-as.vector(X_log %*% beta_log_true)))
y_log <- rbinom(n_log, 1, p_true)

sigmoid <- \(z) 1 / (1 + exp(-z))
nll <- \(b) {
  eta <- as.vector(X_log %*% b)
  mean(log(1 + exp(eta)) - y_log * eta)
}
nll_grad <- \(b) {
  p <- sigmoid(as.vector(X_log %*% b))
  as.vector(t(X_log) %*% (p - y_log)) / n_log
}
```

**Python**

```
rng = np.random.default_rng(7432)

# --- logistic regression data: intercept + one covariate --------------------
n_log = 500
x_log = rng.normal(size=n_log)
X_log = np.column_stack([np.ones(n_log), x_log])   # design matrix: intercept + slope
beta_log_true = np.array([-0.5, 2])
p_true = 1 / (1 + np.exp(-(X_log @ beta_log_true)))
y_log = rng.binomial(1, p_true)

sigmoid = lambda z: 1 / (1 + np.exp(-z))
def nll(b):
    eta = X_log @ b
    return np.mean(np.log(1 + np.exp(eta)) - y_log * eta)
```

```
def nll_grad(b):
    p = sigmoid(X_log @ b)
    return X_log.T @ (p - y_log) / n_log
```

Time to descend. The model has two parameters, so we can watch gradient descent walk over the loss surface of a real statistical model exactly as we watched it cross the toy bowls. We start from $\beta = (0, 0)$, which predicts every observation as a 50/50 coin toss, and take a hand-tuned $\gamma = 2$ (why this value works, and others would not, is the subject of Section 5.2.1).

**R**

```
path_nll <- grad_descent(nll_grad, x0 = c(0, 0), gamma = 2, n = 200)

beta_hat <- unlist(tail(path_nll, 1)[1:2])   # the final iterate
beta_hat
```

```
        x1         x2
-0.5081276  1.8236760
```

```
nll_grad(beta_hat)                           # its gradient: numerically zero
```

```
[1]  3.286926e-15 -1.348729e-14
```

**Python**

```
path_nll = grad_descent(nll_grad, x0=np.array([0.0, 0.0]), gamma=2, n=200)

beta_hat = path_nll.iloc[-1, :2].to_numpy()  # the final iterate
beta_hat
```

```
array([-0.63807379,  1.89155489])
```

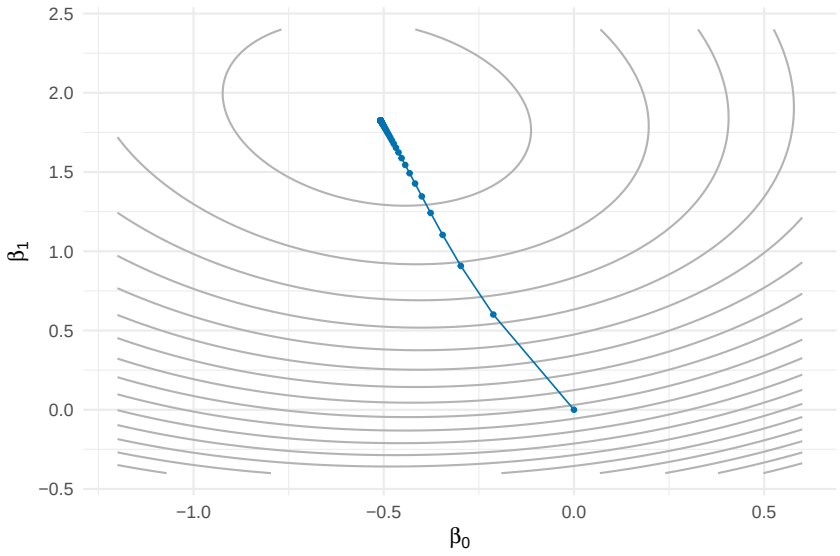
```
nll_grad(beta_hat)                           # its gradient: numerically zero
```

```
array([ 2.45948817e-14, -8.37838965e-14])
```

With no closed form there is no reference value to compare against, but the gradient itself testifies: at `beta_hat` it is zero to machine precision, so the surface is flat there in every direction and the descent has nowhere left to go. Note also that `beta_hat` lands near the true $(-0.5, 2)$ that generated the data but not *on* it: the gap is the sampling error of maximum likelihood, not a failure of the optimiser, and running more iterations would not shrink it.

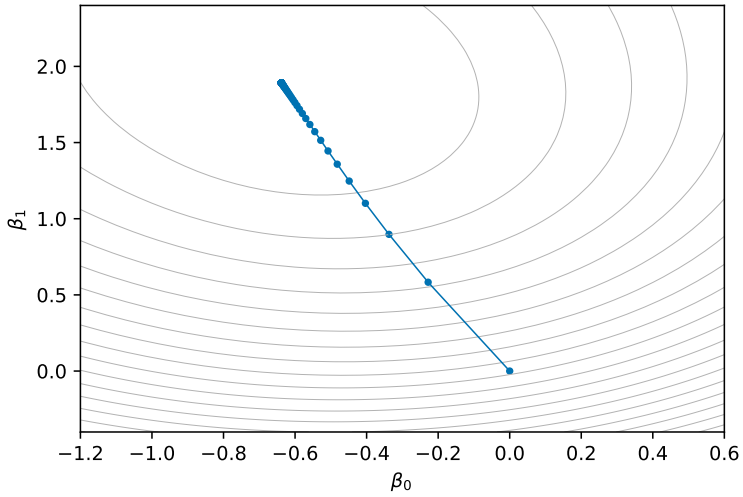
**R**

```r
ggplot(contour_data(nll, c(-1.2, 0.6), c(-0.4, 2.4)), aes(x1, x2)) +
  geom_contour(aes(z = f), bins = 20, colour = "grey70") +
  geom_path(data = path_nll, colour = "#0072B2", linewidth = 0.4) +
  geom_point(data = path_nll, colour = "#0072B2", size = 0.9) +
  labs(x = expression(beta[0]), y = expression(beta[1])) +
  theme_minimal()
```




**Python**

```python
x1, x2, f_grid = contour_data(nll, (-1.2, 0.6), (-0.4, 2.4))
fig, ax = plt.subplots()
ax.contour(x1, x2, f_grid, levels=20, colors="0.7", linewidths=0.5)
ax.plot(path_nll["x1"], path_nll["x2"], color="#0072B2", linewidth=0.8)
ax.scatter(path_nll["x1"], path_nll["x2"], color="#0072B2", s=8)
ax.set_xlabel(r"$\beta_0$")
ax.set_ylabel(r"$\beta_1$")
plt.show()
```




The contours are contours of the NLL $L(\beta_0, \beta_1)$: a picture worth keeping in mind every time you fit *any* model. The path marches straight into the bowl and stops at the maximum likelihood estimate. The fit itself is done; what the rest of the chapter adds is how to get there *cheaply* when $n$ is enormous (Section 5.1.2), why the step size is so delicate (Section 5.2.1), and how to stop hand-tuning it (Section 5.2.3).

#### 5.1.1.1 Approximate gradients

What if we cannot compute the gradient? Perhaps the loss is a black box written by someone else, or the likelihood is a page of algebra we would rather not differentiate. The descent seems stuck before it starts, but remember what a partial derivative *is*: the limit of a difference quotient. We can simply stop short of the limit. For a small $h$, each partial derivative is well approximated by its **central difference**:

$$\frac{\partial L}{\partial \beta_j}(\beta) \;\approx\; \frac{L(\beta + h\,\mathbf{e}_j) - L(\beta - h\,\mathbf{e}_j)}{2h},$$

where $\mathbf{e}_j$ has a one in position $j$ and zeros elsewhere. A loop over the coordinates (Chapter 2) wrapped in a function that takes `f` as an argument (Chapter 3) turns the definition into code. How close does it get? Since here we *do* know the exact gradient, we can measure the approximation error directly:

**R**

```r
# approximate each partial derivative of f by a central difference
num_grad <- function(f, b, h = 1e-6) {
  g <- numeric(length(b))
  for (j in seq_along(b)) {
    e <- numeric(length(b))
    e[j] <- h
    g[j] <- (f(b + e) - f(b - e)) / (2 * h)
  }
  g
}

b_test <- c(0.5, -1)
num_grad(nll, b_test) - nll_grad(b_test)   # approximation error at a test point
```

```
[1] -2.808398e-11 -7.505441e-12
```

**Python**

```python
# approximate each partial derivative of f by a central difference
def num_grad(f, b, h=1e-6):
    g = np.zeros(len(b))
    for j in range(len(b)):
        e = np.zeros(len(b))
        e[j] = h
        g[j] = (f(b + e) - f(b - e)) / (2 * h)
    return g

b_test = np.array([0.5, -1])
num_grad(nll, b_test) - nll_grad(b_test)   # approximation error at a test point
```

```
array([-5.39799039e-11,  4.99606467e-11])
```

An error of order $10^{-11}$: for all practical purposes, `num_grad` recovers the gradient without a single line of calculus. (Note that the same comparison, run in reverse, is a debugging tool: whenever you *have* derived a gradient by hand, it is always possible to check it against a numerical counterpart)

To descend with the approximate gradient, notice that `grad_descent()` never looks inside its `grad` argument; any function that maps a vector to a vector will do, including one manufactured on the spot from the loss itself. That is a job for a function factory (Section 3.5.2): curry the loss $L$ and the step $h$ into `num_grad`, receive a function of $\beta$ alone, and hand that straight to

`grad_descent()`. As an example, we rerun the descent and compare it with the exact-gradient run.

To compare runs fairly we need a small helper, which the rest of the chapter reuses: how many iterations until a path is within `tol` of a target?

**R**

```
steps_to_tol <- function(path, target, tol = 1e-3) {
  X <- as.matrix(path[seq_along(target)])   # the x1, ..., xd columns
  dist <- apply(X, 1, function(x) sqrt(sum((x - target)^2)))
  path$iter[which(dist < tol)[1]]           # NA if it never gets there
}
```

**Python**

```
def steps_to_tol(path, target, tol=1e-3):
    X = path.iloc[:, :len(target)].to_numpy()  # x1, ..., xd
    dist = np.sqrt(((X - target)**2).sum(axis=1))
    hit = np.nonzero(dist < tol)[0]
    return np.nan if len(hit) == 0 else int(path["iter"].iloc[hit[0]])
```

Now the rerun, next to the exact-gradient one:

**R**

```
# factory: fix f and h once, get back a gradient function of b alone
make_num_grad <- function(f, h = 1e-6) \(b) num_grad(f, b, h)

path_num <- grad_descent(make_num_grad(nll), x0 = c(0, 0), gamma = 2, n = 200)

c(steps_exact = steps_to_tol(path_nll, beta_hat),
  steps_num   = steps_to_tol(path_num, beta_hat))
```

```
steps_exact   steps_num
         47          47
```

```
rbind(exact     = beta_hat,
      numerical = unlist(tail(path_num, 1)[1:2]))
```

```
                  x1       x2
exact     -0.5081276 1.823676
numerical -0.5081276 1.823676
```

**Python**

```
# factory: fix f and h once, get back a gradient function of b alone
def make_num_grad(f, h=1e-6):
    return lambda b: num_grad(f, b, h)

path_num = grad_descent(make_num_grad(nll), x0=np.array([0.0, 0.0]), gamma=2, n=200)

{"steps_exact": steps_to_tol(path_nll, beta_hat),
 "steps_num":   steps_to_tol(path_num, beta_hat)}
```

```
{'steps_exact': 50, 'steps_num': 50}
```

```
pd.DataFrame([beta_hat, path_num.iloc[-1, :2].to_numpy()],
             index=["exact", "numerical"])
```

```
                  0         1
exact     -0.638074  1.891555
numerical -0.638074  1.891555
```

Same number of steps into a $10^{-3}$ ball around `beta_hat`, the same estimate to the printed precision (the true gap is about $10^{-10}$), and we never differentiated anything. The price is in the operation count: each step now costs $2d$ evaluations of the full loss (here $4 \times 200 = 800$ passes over the data, against 200 for the exact gradient), a bill that grows with the dimension; and the finite-difference error, harmless here, can bite on badly scaled problems. This trick is exactly what `optim()` and `minimize()` quietly do whenever you call them without supplying a gradient. It is also half of a bigger idea: if first derivatives can be built from differences of the *loss*, then curvature can be built from differences of *gradients*. That observation powers BFGS, the workhorse of the next section.

### 5.1.2 Stochastic gradient descent

Look again at the gradient of the average loss:

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta).$$

Every single step of gradient descent touches **all $n$ observations**. For our $n = 500$ example that is nothing; for $n = 10^8$ user records it means one step of GD costs a full pass over a dataset that may not even fit in memory. The sum structure suggests a bold shortcut: *don't*

*compute the whole sum.* Pick one index $I$ uniformly at random and use $\nabla \ell_I(\theta)$ as a stand-in for $\nabla L(\theta)$. It is a noisy stand-in, but it is **unbiased**: $\mathbb{E}[\nabla \ell_I(\theta)] = \nabla L(\theta)$. On average, it is the right direction. The update

$$\theta_{t+1} = \theta_t - \gamma_t \, \nabla \ell_{I_t}(\theta_t), \qquad I_t \sim \text{Uniform}\{1, \dots, n\},$$

is **stochastic gradient descent** (SGD).

**Example: one observation's gradient.** So far we have mostly written the logistic gradient in matrix form, $\frac{1}{n} X^\top (\sigma(X\beta) - \mathbf{y})$. SGD never builds that product: it works with one row of $X$ at a time. For our model with an intercept and one covariate, observation $i$ has $\mathbf{x}_i = (1, x_i)^\top$ and $\eta_i = \beta_0 + \beta_1 x_i$, so the per-observation gradient from Section 5.1.1, written out in full, is

$$\nabla \ell_i(\beta) = (\sigma(\eta_i) - y_i)\, \mathbf{x}_i = (\sigma(\beta_0 + \beta_1 x_i) - y_i) \begin{pmatrix} 1 \\ x_i \end{pmatrix} = \begin{pmatrix} \sigma(\beta_0 + \beta_1 x_i) - y_i \\ (\sigma(\beta_0 + \beta_1 x_i) - y_i)\, x_i \end{pmatrix} :$$

two numbers, computed from one $(x_i, y_i)$ pair only. The function `nll_grad_i` coded below is this formula, one line long.

The step size now carries a subscript $t$, and it matters. With the full gradient, steps shrink on their own near the minimum, because $\nabla L \to \mathbf{0}$. With SGD they don't: even at the minimiser, a single observation pushes the iterate *somewhere* (only the average push is zero). A constant $\gamma$ therefore never settles: the iterates reach the minimiser's neighbourhood quickly, then rattle around it in a "noise ball" whose radius grows with $\gamma$. The cure is a **decaying schedule** $\gamma_t$: shrink the steps to damp the noise, but not so fast that we run out of fuel. The classical recipe (Robbins and Monro, 1951) asks, informally, for

$$\sum_{t=1}^{\infty} \gamma_t = \infty \qquad \text{and} \qquad \sum_{t=1}^{\infty} \gamma_t^2 < \infty :$$

infinite total travel, so we can reach the minimiser from anywhere, but finite total squared steps, so the noise averages out. Schedules like $\gamma_t = a/(b+t)$ satisfy both.

A compromise between one sample and all $n$ is a **mini-batch**: average $\nabla \ell_i$ over a random subset $B$ of size $m$. Still unbiased, with variance shrinking like $1/m$, at cost proportional to $m$. The economics are the whole point:

- one GD step costs $O(n)$ per-observation gradients;
- one SGD step costs $O(1)$ (or $O(m)$ for a mini-batch), *regardless of* $n$.

When $n$ is huge, SGD has taken thousands of noisy-but-useful steps before GD finishes its first.

Algorithm 5.2 states the mini-batch recipe precisely, with single-sample SGD as the special case $m = 1$.

Algorithm 5.2: Stochastic gradient descent (mini-batch)

**Input:** objective $L(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\theta)$ with per-observation gradients $\nabla \ell_i$, start point $\theta_0$, step sizes $\gamma_t > 0$ (constant, or decaying like $\gamma_t = a/(b+t)$), batch size $m$, number of steps $n_{\text{steps}}$.
**Output:** the trajectory $\theta_0, \theta_1, \dots, \theta_{n_{\text{steps}}}$.

1. **for** $t = 1, 2, \dots, n_{\text{steps}}$:
   a. draw a subset $B_t \subseteq \{1, \dots, n\}$ of size $m$, uniformly at random, without replacement
   b. $\theta_t \leftarrow \theta_{t-1} - \gamma_t \frac{1}{m} \sum_{i \in B_t} \nabla \ell_i(\theta_{t-1})$
2. **return** $\theta_0, \theta_1, \dots, \theta_{n_{\text{steps}}}$

Single-sample SGD is the case $m = 1$: step (a) draws one index $I_t \sim \text{Uniform}\{1, \dots, n\}$ and step (b) becomes the update displayed above, $\theta_t \leftarrow \theta_{t-1} - \gamma_t \nabla \ell_{I_t}(\theta_{t-1})$. Note the index base: the step that *produces* $\theta_t$ uses $\gamma_t$, which is exactly the code's `gamma(t)`, so the first update uses $\gamma_1$.

Here is single-sample SGD in the same functional style. The step size enters as a *function* of the step number $t$, so that constant and decaying schedules fit the same interface, and the per-observation gradient `grad_i` we feed it is an old friend: exactly the $(\sigma(\eta_i) - y_i)\,\mathbf{x}_i$ derived in Section 5.1.1.

**R**

```r
sgd <- function(grad_i, x0, gamma, n_steps, n_data) {
  # grad_i(theta, i): gradient of the loss at observation i only
  # gamma(t): step size at step t
  path <- matrix(NA_real_, nrow = n_steps + 1, ncol = length(x0))
  path[1, ] <- x0
  for (t in 1:n_steps) {
    i <- sample(n_data, 1)
    path[t + 1, ] <- path[t, ] - gamma(t) * grad_i(path[t, ], i)
  }
  out <- as.data.frame(path)
  names(out) <- paste0("x", seq_along(x0))
```

```
  out$iter <- 0:n_steps
  out
}

# single-observation gradient of the NLL: (sigmoid(eta_i) - y_i) * x_i
nll_grad_i <- function(b, i) X_log[i, ] * (sigmoid(sum(X_log[i, ] * b)) - y_log[i])
```

**Python**

```
def sgd(grad_i, x0, gamma, n_steps, n_data):
    # grad_i(theta, i): gradient of the loss at observation i only
    # gamma(t): step size at step t
    path = np.full((n_steps + 1, len(x0)), np.nan)
    path[0] = x0
    for t in range(1, n_steps + 1):
        i = rng.integers(n_data)
        path[t] = path[t - 1] - gamma(t) * grad_i(path[t - 1], i)
    out = pd.DataFrame(path, columns=[f"x{j + 1}" for j in range(len(x0))])
    out["iter"] = range(n_steps + 1)
    return out

# single-observation gradient of the NLL: (sigmoid(eta_i) - y_i) * x_i
nll_grad_i = lambda b, i: X_log[i] * (sigmoid(X_log[i] @ b) - y_log[i])
```

**Note for Python:** `sample(n_data, 1)` draws from $\{1, \dots, n\}$; `rng.integers(n_data)` draws from $\{0, \dots, n-1\}$: each exactly the right range for its language's indexing. And where R's `sample()` quietly uses the global RNG state, the Python column leans on the global `rng` object in the same way.

Let us run it on the logistic loss twice: once with a constant step size, once with a decaying one.

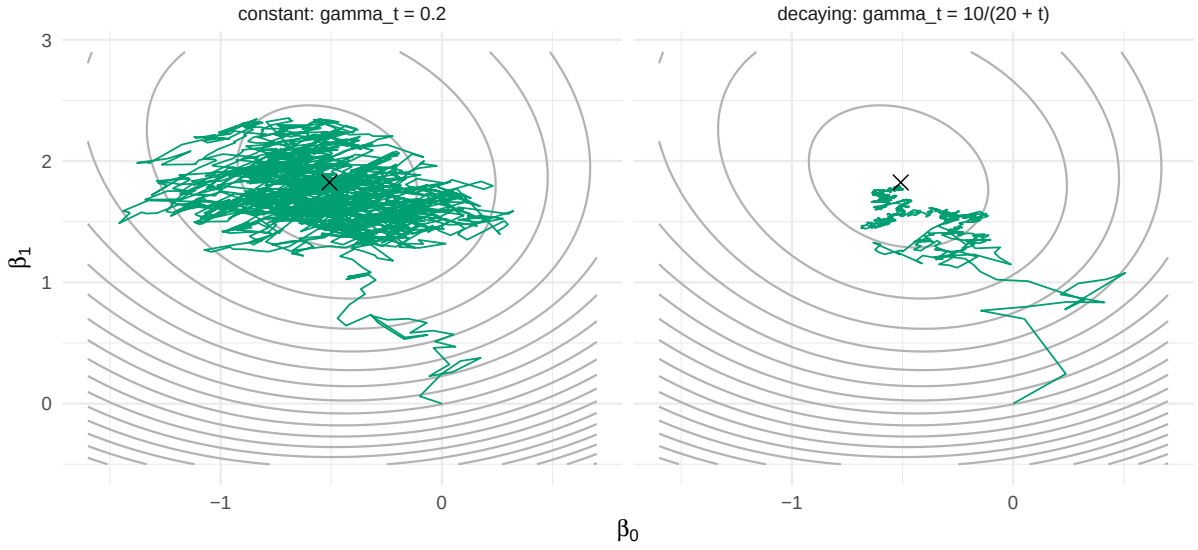
**R**

```
set.seed(1)
sgd_const <- sgd(nll_grad_i, c(0, 0), gamma = \(t) 0.2,           n_steps = 2000, n_data = n
sgd_decay <- sgd(nll_grad_i, c(0, 0), gamma = \(t) 10 / (20 + t), n_steps = 2000, n_data = n

sgd_const$schedule <- "constant: gamma_t = 0.2"
sgd_decay$schedule <- "decaying: gamma_t = 10/(20 + t)"
sgd_both <- rbind(sgd_const, sgd_decay)
```

```r
# the cross marks beta_hat, the minimiser gradient descent found above
mle_pt <- data.frame(x1 = beta_hat[1], x2 = beta_hat[2])

ggplot(contour_data(nll, c(-1.6, 0.7), c(-0.5, 2.9)), aes(x1, x2)) +
  geom_contour(aes(z = f), bins = 20, colour = "grey70") +
  geom_path(data = sgd_both, colour = "#009E73", linewidth = 0.4) +
  geom_point(data = mle_pt, shape = 4, size = 3, colour = "black") +
  facet_wrap(~ schedule) +
  labs(x = expression(beta[0]), y = expression(beta[1])) +
  theme_minimal()
```

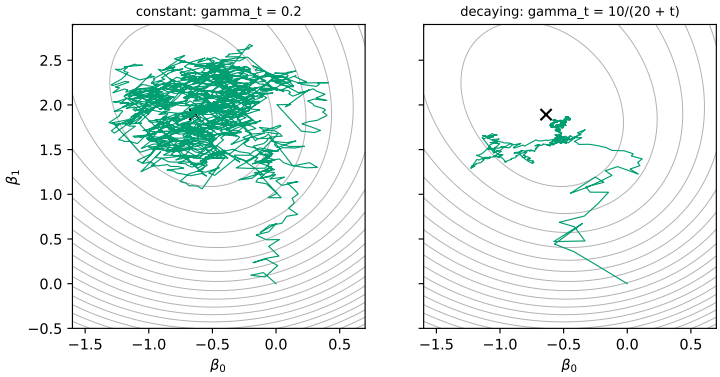



### Python

```python
rng = np.random.default_rng(1)
sgd_const = sgd(nll_grad_i, np.array([0.0, 0.0]), gamma=lambda t: 0.2,           n_steps=2000
sgd_decay = sgd(nll_grad_i, np.array([0.0, 0.0]), gamma=lambda t: 10 / (20 + t), n_steps=2000

# one subplot per schedule, same contours in each;
# the cross marks beta_hat, the minimiser gradient descent found above
titles = ["constant: gamma_t = 0.2", "decaying: gamma_t = 10/(20 + t)"]
x1, x2, f_grid = contour_data(nll, (-1.6, 0.7), (-0.5, 2.9))
fig, axes = plt.subplots(1, 2, figsize=(8, 3.8), sharey=True)
for ax, p, ttl in zip(axes, [sgd_const, sgd_decay], titles):
    ax.contour(x1, x2, f_grid, levels=20, colors="0.7", linewidths=0.5)
```

```
    ax.plot(p["x1"], p["x2"], color="#009E73", linewidth=0.8)
    ax.scatter([beta_hat[0]], [beta_hat[1]], marker="x", s=60, color="black")
    ax.set_title(ttl, fontsize=9)
    ax.set_xlabel(r"$\beta_0$")
axes[0].set_ylabel(r"$\beta_1$")
plt.show()
```




Both trajectories are noisy versions of the gradient descent path: that is the noise of estimating the gradient from one observation. And the endgames play out exactly as the theory promised: the **constant** step reaches the neighbourhood of $\widehat{\beta}$ quickly and then rattles around it forever in its noise ball, while the **decaying** step still moves fast early on and then funnels into the minimum as the shrinking $\gamma_t$ damps the noise.

Finally, the claim that actually sells SGD: progress *per data point touched*. One GD step costs $n = 500$ observation-gradients; one SGD step costs 1. Plot the loss against that budget:

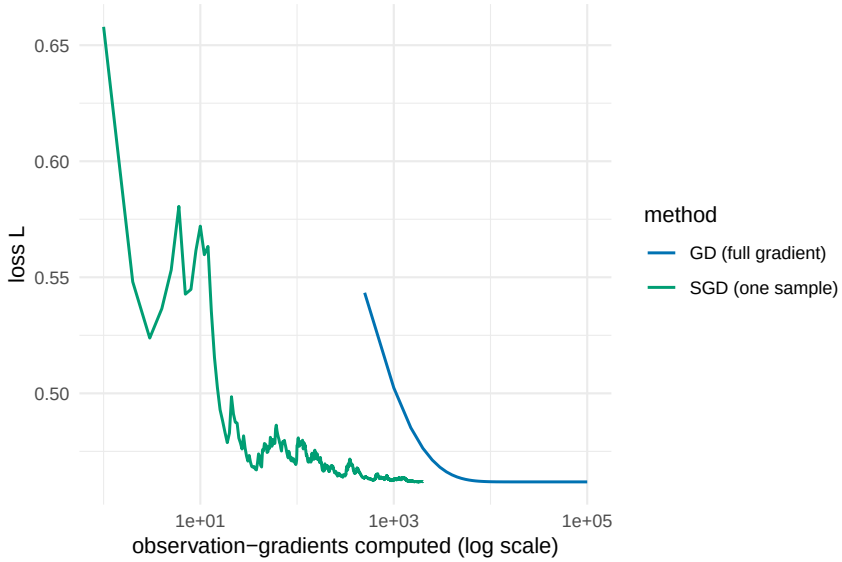
**R**

```
library(dplyr)

# the paths hold only iterates; evaluate the loss along each of them ourselves
path_nll$loss  <- apply(as.matrix(path_nll[, 1:2]), 1, nll)
sgd_decay$loss <- apply(as.matrix(sgd_decay[, 1:2]), 1, nll)

cost_df <- bind_rows(
  tibble(evals = path_nll$iter * n_log, loss = path_nll$loss,  method = "GD (full gradient)"),
  tibble(evals = sgd_decay$iter,        loss = sgd_decay$loss, method = "SGD (one sample)")
) |>
  filter(evals > 0)
```

```r
ggplot(cost_df, aes(evals, loss, colour = method)) +
  geom_line(linewidth = 0.7) +
  scale_x_log10() +
  scale_colour_manual(values = c("GD (full gradient)" = "#0072B2",
                                 "SGD (one sample)" = "#009E73")) +
  labs(x = "observation-gradients computed (log scale)", y = "loss L") +
  theme_minimal()
```



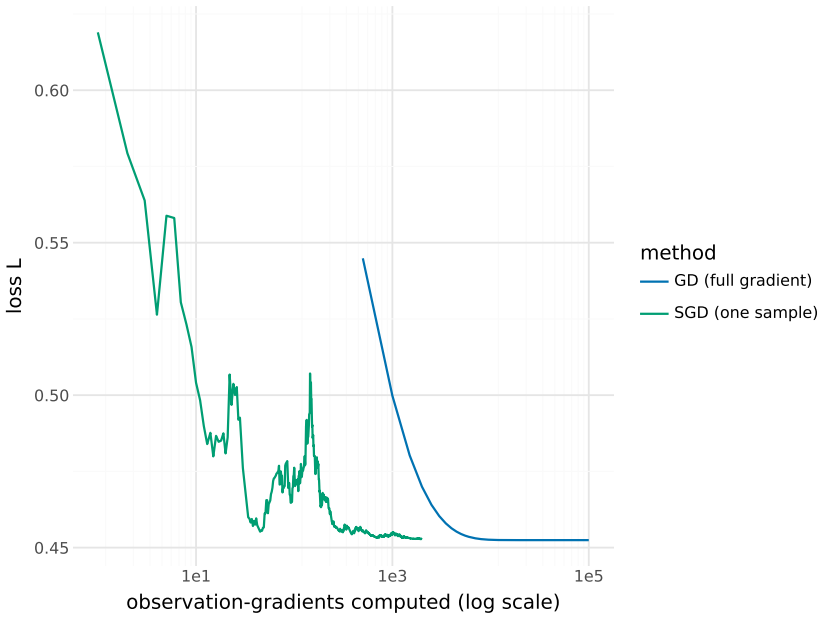

### Python

```python
from plotnine import (ggplot, aes, geom_line, scale_x_log10,
                      scale_colour_manual, labs, theme_minimal)

# the paths hold only iterates; evaluate the loss along each of them ourselves
path_nll["loss"]  = [nll(b) for b in path_nll[["x1", "x2"]].to_numpy()]
sgd_decay["loss"] = [nll(b) for b in sgd_decay[["x1", "x2"]].to_numpy()]

cost_df = pd.concat([
    pd.DataFrame({"evals": path_nll["iter"] * n_log, "loss": path_nll["loss"],
                  "method": "GD (full gradient)"}),
    pd.DataFrame({"evals": sgd_decay["iter"], "loss": sgd_decay["loss"],
                  "method": "SGD (one sample)"})
])
cost_df = cost_df[cost_df["evals"] > 0]
```

```
(ggplot(cost_df, aes("evals", "loss", colour="method"))
 + geom_line(size=0.7)
 + scale_x_log10()
 + scale_colour_manual(values={"GD (full gradient)": "#0072B2",
                               "SGD (one sample)": "#009E73"})
 + labs(x="observation-gradients computed (log scale)", y="loss L")
 + theme_minimal()).show()
```




**Note for Python:** pandas has no `bind_rows()` or `filter()` verbs; `pd.concat()` plus a boolean mask do the same job (Chapter 4). And plotnine still calls the line-width aesthetic `size`, which ggplot2 has since renamed `linewidth`.

By the time GD has finished its *first* step (500 evaluations), SGD has already collapsed most of the loss. Read the other end honestly too: GD then converges to the exact minimiser, while SGD closes in more slowly and less precisely. For small $n$ and high accuracy, GD (or better, Section 5.2.3) wins; for enormous $n$ and statistical-scale accuracy (remember the estimate itself has sampling error of order $1/\sqrt{n}$, so chasing the tenth decimal of the minimiser is vanity), SGD is the only affordable option. It is the workhorse that trains essentially every neural network.

Exercise 5.1: Mini-batch SGD

Fill in the blanks to turn `sgd()` into mini-batch SGD, then run it on the logistic loss. The batch gradient function is given. Note (on the R side) the `drop = FALSE`: when the batch holds a single index, `X_log[idx, ]` would silently collapse the one-row matrix to a plain vector, and `drop = FALSE` keeps the matrix shape so the algebra survives `batch = 1`:

**R**

```r
nll_grad_batch <- function(b, idx) {
  Xb <- X_log[idx, , drop = FALSE]
  p <- sigmoid(as.vector(Xb %*% b))
  as.vector(t(Xb) %*% (p - y_log[idx])) / length(idx)
}

sgd_batch <- function(grad_batch, x0, gamma, n_steps, n_data, batch = 10) {
  path <- matrix(NA_real_, nrow = n_steps + 1, ncol = length(x0))
  path[1, ] <- x0
  for (t in 1:n_steps) {
    idx <- sample(____, ____)          # draw a mini-batch of indices
    path[t + 1, ] <- ____              # the usual update, batch gradient
  }
  out <- as.data.frame(path)
  names(out) <- paste0("x", seq_along(x0))
  out$iter <- 0:n_steps
  out
}
```

**Python**

```python
def nll_grad_batch(b, idx):
    Xb = X_log[idx]
    p = sigmoid(Xb @ b)
    return Xb.T @ (p - y_log[idx]) / len(idx)

def sgd_batch(grad_batch, x0, gamma, n_steps, n_data, batch=10):
    path = np.full((n_steps + 1, len(x0)), np.nan)
    path[0] = x0
    for t in range(1, n_steps + 1):
        idx = rng.choice(____, ____, replace=False)   # draw a mini-batch of indices
        path[t] = ____                                # the usual update, batch gradient
    out = pd.DataFrame(path, columns=[f"x{j + 1}" for j in range(len(x0))])
    out["iter"] = range(n_steps + 1)
    return out
```

**Note for Python:** numpy dodges the `drop = FALSE` trap: indexing with an index *array* (`X_log[idx]`) always keeps the 2-D shape; only a bare scalar index drops the dimension. The trap on the Python side is `rng.choice()`, which samples **with** replacement unless told `replace=False`, whereas R's `sample()` is without replacement by default.
Run 2000 steps with constant `gamma = \(t) 0.2` (`lambda t: 0.2`) for batch sizes 1, 10 and 50, and compare the wobble of the final 500 iterates (e.g. via `sd()` of each coordinate, `.std()` in Python, or just by plotting the paths). What does the batch size buy you, and what does it cost?

## 5.2 Fixing the geometry: preconditioning and BFGS

This section is about the shape of the functions we minimise. First we see why their geometry makes the step size so delicate; then we fix that geometry twice: first by hand, then automatically.

### 5.2.1 Why the step size is hard

So far every descent in this chapter has run with a step size picked by hand, and behaved. Time to break it. The step size $\gamma$ is doing a lot of quiet work. To expose it, we use a function that is deliberately *ill-conditioned* (much steeper in one direction than the other):

$$f(x_1, x_2) = \frac{x_1^2}{2} + \frac{10\,x_2^2}{2}, \qquad \nabla f(x_1, x_2) = \begin{pmatrix} x_1 \\ 10\,x_2 \end{pmatrix}.$$

Think of a long narrow valley: gentle along $x_1$ (curvature 1), steep across it in $x_2$ (curvature 10). Watch what three different step sizes do, starting from $(-9, 1.5)$.

**R**

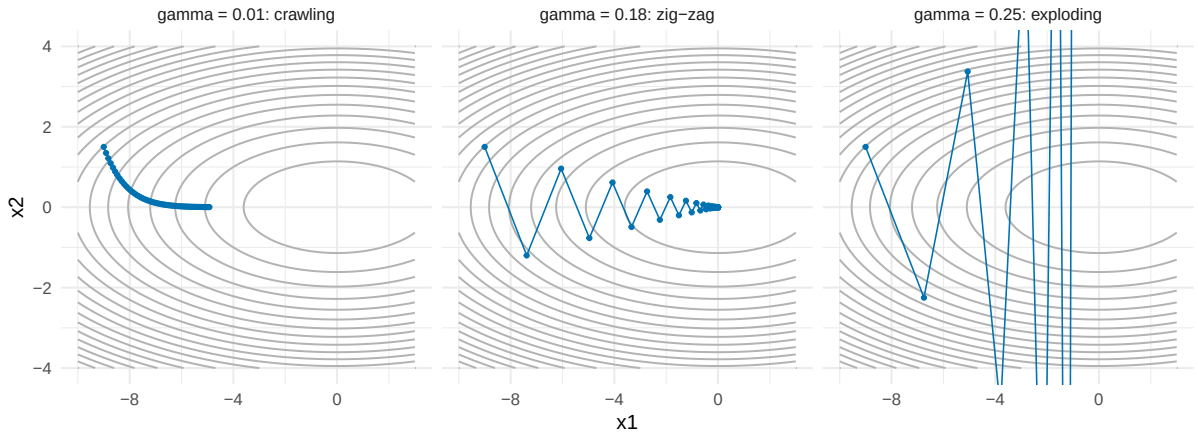
```
f_ill <- \(x) x[1]^2 / 2 + 10 * x[2]^2 / 2
grad_ill <- \(x) c(x[1], 10 * x[2])

path_small <- grad_descent(grad_ill, c(-9, 1.5), gamma = 0.01, n = 60)
path_zig   <- grad_descent(grad_ill, c(-9, 1.5), gamma = 0.18, n = 60)
path_exploding  <- grad_descent(grad_ill, c(-9, 1.5), gamma = 0.25, n = 8)

# one labelled data frame, one facet per step size (week 4 tricks)
path_small$gamma <- "gamma = 0.01: crawling"
path_zig$gamma   <- "gamma = 0.18: zig-zag"
path_exploding$gamma  <- "gamma = 0.25: exploding"
```

```r
paths3 <- rbind(path_small, path_zig, path_exploding)
paths3$gamma <- factor(paths3$gamma, levels = unique(paths3$gamma))

# the contour layer has no `gamma` column, so it is drawn in every facet
ggplot(contour_data(f_ill, c(-10, 3), c(-4, 4)), aes(x1, x2)) +
  geom_contour(aes(z = f), bins = 20, colour = "grey70") +
  geom_path(data = paths3, colour = "#0072B2", linewidth = 0.4) +
  geom_point(data = paths3, colour = "#0072B2", size = 0.9) +
  coord_cartesian(ylim = c(-4, 4)) +
  facet_wrap(~ gamma) +
  theme_minimal()
```




### Python

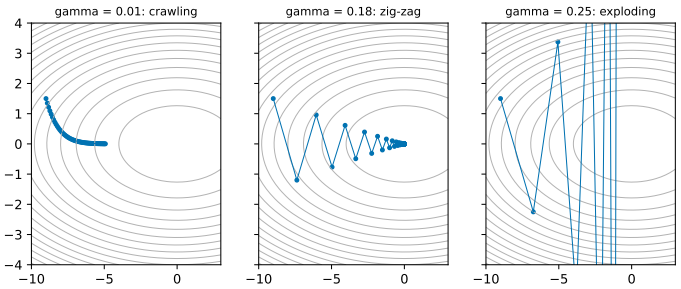
```python
f_ill = lambda x: x[0]**2 / 2 + 10 * x[1]**2 / 2
grad_ill = lambda x: np.array([x[0], 10 * x[1]])

path_small = grad_descent(grad_ill, np.array([-9, 1.5]), gamma=0.01, n=60)
path_zig   = grad_descent(grad_ill, np.array([-9, 1.5]), gamma=0.18, n=60)
path_exploding  = grad_descent(grad_ill, np.array([-9, 1.5]), gamma=0.25, n=8)

# one list of labelled paths, one subplot per step size (no facets in matplotlib)
titles = ["gamma = 0.01: crawling", "gamma = 0.18: zig-zag", "gamma = 0.25: exploding"]
paths3 = [path_small, path_zig, path_exploding]

# the same contours are drawn in every panel, then one trajectory per panel
x1, x2, f_grid = contour_data(f_ill, (-10, 3), (-4, 4))
```

```
fig, axes = plt.subplots(1, 3, figsize=(9, 3.4), sharey=True)
for ax, p, ttl in zip(axes, paths3, titles):
    ax.contour(x1, x2, f_grid, levels=20, colors="0.7", linewidths=0.5)
    ax.plot(p["x1"], p["x2"], color="#0072B2", linewidth=0.8)
    ax.scatter(p["x1"], p["x2"], color="#0072B2", s=8)
    ax.set_ylim(-4, 4)
    ax.set_title(ttl, fontsize=9)
plt.show()
```




It's like in Goldilocks and the Three Bears: the step size can be

- **Too small** ($\gamma = 0.01$): every step is safe and almost useless. After 60 iterations we have barely moved along the valley.
- **Near the limit** ($\gamma = 0.18$): the steep $x_2$ direction overshoots and bounces from one wall of the valley to the other (the famous **zig-zag**) while slowly drifting towards the minimum.
- **Too big** ($\gamma = 0.25$): each bounce is *bigger* than the last. The iterates diverge. (The panel is clipped: the true trajectory has already left the plotting window.)

By Arthur Rackham - http://www.gutenberg.org/etext/17034, Public Domain, Link

There is a clean piece of intuition behind this. For a quadratic, gradient descent multiplies the error along each principal axis of the bowl (for our valley, simply the two coordinate axes) by $(1 - \gamma\lambda)$ per step, where $\lambda$ is the curvature in that direction.

Read the three panels as the three trajectories above, one coordinate at a time.

#### 5.2.1.1 The condition number

These directional curvatures are exactly the *eigenvalues* of the **Hessian** of $f$, its symmetric matrix of second derivatives $\nabla^2 f$. Formally, collect the eigenvalues of the Hessian into the set $\{\lambda_1, \lambda_2, \dots, \lambda_d\}$ and pick out its two extremes:

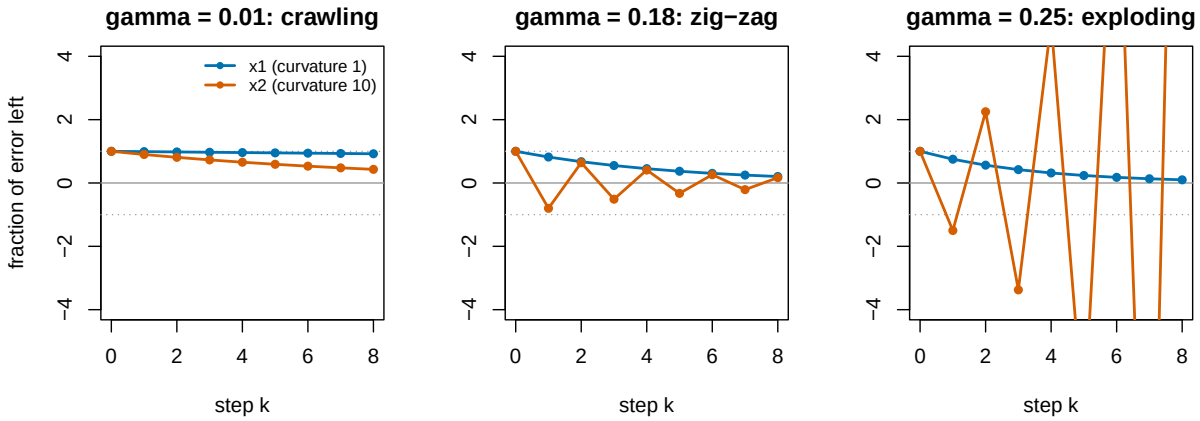



Figure 5.1: Fraction of the starting error left in each coordinate after $k$ steps, i.e. $(1-\gamma\lambda)^k$, for the valley with $\lambda = 1$ (along $x_1$) and $\lambda = 10$ (along $x_2$). The right panel is clipped to $[-4, 4]$.

$$\lambda_{\max} = \max_j \lambda_j \quad \text{(the steepest curvature)}, \qquad \lambda_{\min} = \min_j \lambda_j \quad \text{(the flattest)}.$$

Stability requires $|1-\gamma\lambda_j| < 1$ for *every* direction $j$, and the steepest one is the binding constraint:

$$\gamma < \frac{2}{\lambda_{\max}}.$$

Progress in the flattest direction, meanwhile, is governed by $\lambda_{\min}$: per step its error shrinks by $(1-\gamma\lambda_{\min})$, at best about $(1-\lambda_{\min}/\lambda_{\max})$. The tension between the two extremes is measured by their ratio,

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \qquad \text{(the condition number)},$$

which controls how many iterations you need: as $\kappa$ grows, the iteration count grows in proportion.

In our example, the valley has curvatures 1 (along $x_1$) and 10 (across it, along $x_2$), so $\lambda_{\max} = 10$ and the stability bound is $\gamma < 2/10 = 0.2$: exactly why 0.18 was stable, if only just, and 0.25 diverged. Its condition number is $\kappa = 10/1 = 10$, and that is already annoying: the zig-zag panel is what $\kappa = 10$ looks like.

The same reasoning explains, in hindsight, the hand-tuned $\gamma = 2$ of Section 5.1.1: large, but safe there, because the contours of that negative log-likelihood form a friendly, nearly round bowl (small $\kappa$).

[OPTIONAL] Condition numbers: why statistical problems are ill-conditioned

If you have met condition numbers in a linear algebra course, you may know a different-looking definition: $\kappa(A) = \sigma_{\max}(A)/\sigma_{\min}(A)$, a ratio of **singular values**, measuring how much solving $A\mathbf{x} = \mathbf{b}$ amplifies small errors in the inputs. There is no conflict. Our $\kappa$ is the condition number *of the Hessian*, and a Hessian is always symmetric (mixed partial derivatives commute) and, at a minimum, positive (semi-)definite; for such matrices the singular values *are* the eigenvalues, so $\lambda_{\max}/\lambda_{\min}$ and $\sigma_{\max}/\sigma_{\min}$ are the same number. (For a general symmetric matrix, $\sigma_i = |\lambda_i|$; eigenvalues and singular values genuinely part ways only for non-symmetric matrices, which a Hessian never is.) One notation warning for further reading: optimisation texts typically write $L = \lambda_{\max}$ and $\mu = \lambda_{\min}$, calling them the *smoothness* and *strong-convexity* constants, and state the condition number as $L/\mu$; same objects, different costume.

The singular-value view also explains why *statistical* problems are so often ill-conditioned. A model fit through a design matrix $X$ (Challenge 1.1's least squares; the logistic regression of Section 5.1.1, where a diagonal matrix of observation weights gets pinched between $X^\top$ and $X$) has a Hessian of the form $X^\top X/n$, and the eigenvalues of $X^\top X$ are the *squared* singular values of $X$. Whatever imbalance the columns of $X$ carry, the optimisation problem therefore suffers it twice over: one covariate in grams and another in kilograms (a scale ratio of just $10^3$) can already push $\kappa$ past $10^6$, and plain gradient descent simply stops being a usable tool. At the extreme $\sigma_{\min} = 0$, $\kappa$ is infinite and $X^\top X$ is exactly the *singular* matrix that Chapter 1 warned cannot be inverted; a huge $\kappa$ means nearly so.

Whether or not you open the box above, take this with you: covariates on very different scales make $\kappa$ huge and gradient descent crawl, which is why Exercise 5.3 will ask you to stare at the column scales of its design matrix. Ill-conditioning is the recurring obstacle for the rest of the chapter.

The `steps_to_tol()` helper of Section 5.1.1.1 puts a number on the crawl and on the zig-zag: the iterations each needs to get within $10^{-3}$ of the minimum.

**R**

```
c(crawl = steps_to_tol(grad_descent(grad_ill, c(-9, 1.5), 0.01, 5000), c(0, 0)),
  zig   = steps_to_tol(grad_descent(grad_ill, c(-9, 1.5), 0.18, 5000), c(0, 0)))
```

```
crawl   zig
  906    46
```

**Python**

```
{"crawl": steps_to_tol(grad_descent(grad_ill, np.array([-9, 1.5]), 0.01, 5000),
                       np.array([0, 0])),
 "zig":   steps_to_tol(grad_descent(grad_ill, np.array([-9, 1.5]), 0.18, 5000),
                       np.array([0, 0]))}
```

```
{'crawl': 906, 'zig': 46}
```

Exercise 5.2: Goldilocks and the step size

Consider $f(x_1, x_2) = x_1^2/2 + 25\,x_2^2/2$, with gradient $\nabla f = (x_1, 25x_2)$, starting point $(-8, 2)$, and true minimum $(0, 0)$.

a. Using the stability condition $\gamma < 2/\lambda_{\max}$, predict the largest stable step size. Check your prediction by running `grad_descent()` with $\gamma$ just below and just above it (a small `n`, say 100, is enough to see divergence).
b. Use `steps_to_tol()` to count the iterations needed to get within $10^{-3}$ of the minimum for $\gamma = 0.005, 0.04$ and $0.07$ (use a generous `n`, e.g. 20000). What do you conclude about how to choose $\gamma$?

### 5.2.2 Preconditioning

Back to the zig-zag valley of Section 5.2.1. The step size was capped by the steep direction, progress was throttled along the flat one, and the trajectory wasted its energy bouncing between walls. Time to fix it.

If the valley is ten times steeper in $x_2$ than in $x_1$, then take steps ten times smaller in $x_2$. Formally, pick a symmetric positive-definite matrix $P$ and update

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \gamma\, P \nabla f(\mathbf{x}_t).$$

$P$ reshapes the descent direction, stretching it in flat directions and shrinking it in steep ones. Plain gradient descent is $P = I$.

**Intuition: preconditioning is a change of coordinates.** Take the valley $f(x_1, x_2) = x_1^2/2 + 10\,x_2^2/2$ of Section 5.2.1, and the matrix $P = \mathrm{diag}(1, 1/10)$ (why exactly this one, shortly). Rather than changing the algorithm, change the ruler: measure the steep coordinate in units $\sqrt{10}$ times finer by setting

$$\mathbf{z} = P^{-1/2}\mathbf{x}, \quad \text{i.e.} \quad z_1 = x_1, \qquad z_2 = \sqrt{10}\,x_2.$$

(Why $\sqrt{10}$ and not 10? The Hessian holds *second* derivatives, so stretching $x_2$ by a factor $s$ divides its curvature by $s^2$: to turn curvature 10 into 1 we need $s^2 = 10$.)

Substituting $x_2 = z_2/\sqrt{10}$, the same function in the new coordinates is $f = z_1^2/2 + z_2^2/2$: the narrow valley has become a perfectly **round bowl**. In a round bowl the gradient points straight at the minimum, and plain gradient descent with $\gamma = 1$ jumps there in one step: $\mathbf{z} - 1 \cdot \nabla_{\mathbf{z}} f = \mathbf{z} - \mathbf{z} = \mathbf{0}$. Now translate that step back to $\mathbf{x}$. By the chain rule, $\nabla_{\mathbf{z}} f = P^{1/2} \nabla_{\mathbf{x}} f$ (here simply $\partial f/\partial z_2 = \frac{1}{\sqrt{10}}\, \partial f/\partial x_2$), and $\mathbf{x} = P^{1/2}\mathbf{z}$, so a plain step in $\mathbf{z}$ becomes

$$\mathbf{x} - \gamma\, P^{1/2}(P^{1/2}\nabla_{\mathbf{x}} f) = \mathbf{x} - \gamma\, P \nabla_{\mathbf{x}} f$$

in $\mathbf{x}$: exactly the preconditioned update. **Preconditioned gradient descent is plain gradient descent run in rescaled coordinates.** The animation below shows (a) the original step, where $-\nabla f$ is perpendicular to the elongated contours and so points *across* the valley instead of along it; then the rescaling, during which the contours turn into circles and the steepest-descent direction swings round to face the minimum; (b) the plain step in the round bowl; and finally the squeeze back to $\mathbf{x}$, where that same step is the preconditioned step $-P\nabla f$.

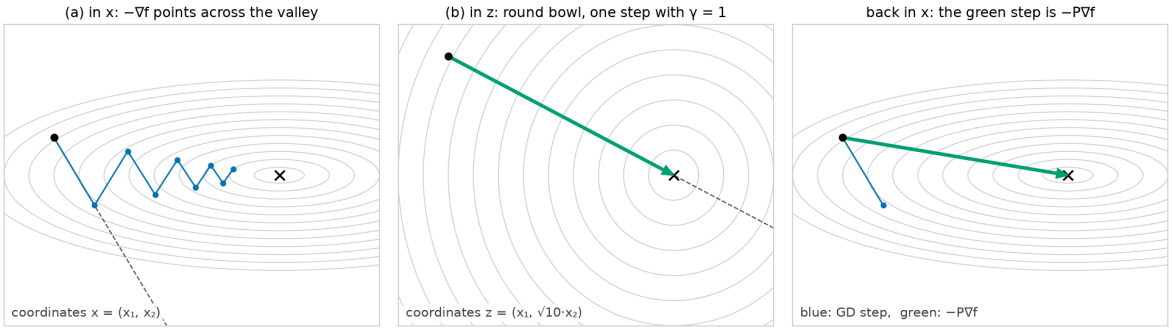



Figure 5.2: Preconditioning as a change of coordinates, on the valley $f = x_1^2/2 + 10\, x_2^2/2$ starting from $(-9, 1.5)$. (a) In the original coordinates, $-\nabla f$ (dashed) is perpendicular to the contours and points across the valley, so gradient descent ($\gamma = 0.18$, blue) zig-zags. (b) After stretching the $x_2$ axis by $\sqrt{10}$ the contours are circles, and a single plain step with $\gamma = 1$ (green) lands on the minimum. Right: back in $\mathbf{x}$, that green step is the preconditioned step $-P\nabla f$ with $P = \mathrm{diag}(1, 1/10)$. (The HTML version of these notes animates this.)

The same reading works for any symmetric positive-definite $P$: its square root $P^{1/2}$ exists (take the eigendecomposition from Chapter 1 and square-root the eigenvalues), and choosing $P$ amounts to choosing the coordinates in which plain gradient descent will run. A good $P$ is one that makes the bowl round.

Algorithm 5.3 is Algorithm 5.1 with one matrix slipped in.

> Algorithm 5.3: Preconditioned gradient descent
>
> **Input:** objective $f$, gradient $\nabla f$, start point $\mathbf{x}_0$, step size $\gamma > 0$, a symmetric positive-definite matrix $P$, number of steps $n_{\text{steps}}$ (the code's `n`).
> **Output:** the trajectory $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{n_{\text{steps}}}$.
>
> 1. **for** $t = 1, 2, \dots, n_{\text{steps}}$:
>    a. $\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \gamma P \nabla f(\mathbf{x}_{t-1})$
> 2. **return** $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{n_{\text{steps}}}$
>
> $P = I$ recovers Algorithm 5.1. Taking $P = (\nabla^2 f)^{-1}$, the inverse Hessian recomputed at every iterate, with $\gamma = 1$, is Newton's method (of which more shortly).

**R**

```r
grad_descent_pre <- function(grad, x0, gamma, n, P) {
  path <- matrix(NA_real_, nrow = n + 1, ncol = length(x0))
  path[1, ] <- x0
  for (i in 1:n) {
    path[i + 1, ] <- path[i, ] - gamma * as.vector(P %*% grad(path[i, ]))
  }
  out <- as.data.frame(path)
  names(out) <- paste0("x", seq_along(x0))
  out$iter <- 0:n
  out
}
```

**Python**

```python
def grad_descent_pre(grad, x0, gamma, n, P):
    path = np.full((n + 1, len(x0)), np.nan)
    path[0] = x0
    for i in range(n):
        path[i + 1] = path[i] - gamma * (P @ grad(path[i]))
    out = pd.DataFrame(path, columns=[f"x{j + 1}" for j in range(len(x0))])
    out["iter"] = range(n + 1)
    return out
```

What is the *ideal* $P$? Start from the Hessian of our valley. Differentiating $f(x_1, x_2) = x_1^2/2 + 10\,x_2^2/2$ twice,

$$\frac{\partial^2 f}{\partial x_1^2} = 1, \qquad \frac{\partial^2 f}{\partial x_2^2} = 10, \qquad \frac{\partial^2 f}{\partial x_1 \partial x_2} = 0 \qquad \Longrightarrow \qquad H = \begin{pmatrix} 1 & 0 \\ 0 & 10 \end{pmatrix} = \operatorname{diag}(1, 10)\,:$$

curvature 1 along $x_1$, 10 along $x_2$. The off-diagonal entries vanish because $f$ contains no term mixing $x_1$ and $x_2$: it is a function of $x_1$ alone plus a function of $x_2$ alone, so the slope in one coordinate never depends on the other. Geometrically, the axes of the elliptical contours line up with the coordinate axes. **This is a property of our example, chosen to keep the algebra light, not of Hessians in general**; we return to the general case below. Choosing $P = H^{-1} = \operatorname{diag}(1, 1/10)$ divides each direction by its own curvature, so *every* direction behaves like curvature 1: the valley becomes a perfectly round bowl, $\kappa = 1$. With $\gamma = 1$:

**R**

```r
P <- diag(c(1, 1 / 10))
path_pre <- grad_descent_pre(grad_ill, c(-9, 1.5), gamma = 1, n = 5, P = P)
round(path_pre, 10)
```

```
  x1  x2 iter
1 -9 1.5    0
2  0 0.0    1
3  0 0.0    2
4  0 0.0    3
5  0 0.0    4
6  0 0.0    5
```

**Python**

```python
P = np.diag([1, 1 / 10])
path_pre = grad_descent_pre(grad_ill, np.array([-9, 1.5]), gamma=1, n=5, P=P)
path_pre.round(10)
```

```
    x1   x2  iter
0 -9.0  1.5     0
1  0.0  0.0     1
2  0.0  0.0     2
3  0.0  0.0     3
4  0.0  0.0     4
5  0.0  0.0     5
```

One step. Not "converges quickly": *lands exactly on the minimum in one step.*

Taking $P = (\nabla^2 f)^{-1}$, recomputed at every iterate, with $\gamma = 1$, has a name: **Newton's method**,

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \left(\nabla^2 f(\mathbf{x}_t)\right)^{-1} \nabla f(\mathbf{x}_t),$$

which jumps straight to the minimiser of the local quadratic approximation of $f$ at $\mathbf{x}_t$. On a quadratic it finishes in one step; near the minimum of a smooth function it converges *quadratically* (the number of correct digits roughly doubles per iteration). So why isn't Newton the end of the chapter?

- **Cost.** In $d$ dimensions the Hessian has $d^2$ entries, and applying its inverse means solving a $d \times d$ linear system: $O(d^3)$ per step. For a neural network with $d = 10^6$ parameters, the Hessian alone would need terabytes of memory.
- **Fragility.** Away from the minimum the Hessian need not be positive definite, and then $-H^{-1}\nabla f$ may not even point downhill. Making Newton robust takes real engineering.

**What if the Hessian is not diagonal?** Rotate the same valley by 45°, so that it runs diagonally across the plane:

$$f(x_1, x_2) = \tfrac{1}{2}\,\mathbf{x}^\top H \mathbf{x}, \qquad H = \begin{pmatrix} 5.5 & 4.5 \\ 4.5 & 5.5 \end{pmatrix},$$

which expands to $f = 2.75\,x_1^2 + 4.5\,x_1 x_2 + 2.75\,x_2^2$. The cross term $x_1 x_2$ puts 4.5 off the diagonal. The eigenvalues of $H$ are still 10 and 1 (along the diagonal directions $(1, 1)$ and $(1, -1)$), so $\kappa = 10$ exactly as before; only the orientation has changed. Two things follow.

- The *full* inverse still works: $P = H^{-1}$ rescales along the eigenvectors of $H$, makes the bowl round, and lands on the minimum in one step, whatever the orientation. But that is Newton's method, at Newton's price.
- The obvious cheap way out, keeping only the *diagonal* of $H$, does not. Its diagonal is $\mathrm{diag}(5.5, 5.5)$, so $P = \mathrm{diag}(1/5.5, 1/5.5) = I/5.5$: a multiple of the identity, which only rescales $\gamma$ and leaves $\kappa = 10$, and the zig-zag, untouched.

A diagonal $P$ can only stretch along the coordinate axes. It fixes valleys aligned with those axes and is powerless against ones that are not.

Cheap compromises are everywhere in statistics and machine learning: use only the *diagonal* of the Hessian (exactly our $P$ above: for axis-aligned problems it is the whole story; for rotated ones, as we just saw, it can miss the point entirely, but it reliably fixes the most common statistical ailment, covariates on wildly different scales); or simply **standardise your covariates**, which is preconditioning by a diagonal matrix before the algorithm even starts. If you remember one practical trick from this section: *rescale your variables before optimising.* Exercise 5.3 shows what happens if you don't.

### 5.2.3 BFGS: learning the Hessian as you go

We now know what the ideal preconditioner wants: $P = H^{-1}$, the inverse of the Hessian, which makes the bowl round; recomputed at every iterate, it is Newton's method. We also know its price: $d^2$ second derivatives and a $d \times d$ linear system at every step. **Quasi-Newton** methods keep the preconditioned step of Algorithm 5.3, but replace the exact inverse Hessian by a matrix $B_k \approx H^{-1}$ that is *estimated* along the way, and refined a little at every step:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \gamma_k \, B_k \nabla f_k,$$

where $\nabla f_k$ is short for $\nabla f(\mathbf{x}_k)$ and the step length $\gamma_k$ is chosen by a line search. We start from $B_0 = I$, so the first step is a plain gradient step.[1]

But what is $B_k$, and how do we build it without ever computing a second derivative? Recall Section 5.1.1.1: a *difference of losses*, divided by the distance between the two points, approximates a first derivative. The same trick one level up turns a *difference of gradients* into second derivatives.

In one dimension, write $s_k = x_{k+1} - x_k$ for the step and $y_k = f'(x_{k+1}) - f'(x_k)$ for the change in slope. A first-order Taylor expansion of $f'$ around $x_k$ gives $y_k \approx f''(x_k)\, s_k$, so

$$f''(x_k) \approx \frac{y_k}{s_k} \qquad \text{and, inverting,} \qquad B_{k+1} = \frac{s_k}{y_k} \approx \frac{1}{f''(x_k)} :$$

the inverse curvature the step needs, from two derivatives the algorithm has computed anyway. ($y_k/s_k$ is the slope of the secant line through two points of the graph of $f'$, hence the name of what follows.)

In $d$ dimensions, with $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and $\mathbf{y}_k = \nabla f_{k+1} - \nabla f_k$, the same expansion reads $\mathbf{y}_k \approx H\mathbf{s}_k$ (exactly, for a quadratic $f$), that is $H^{-1}\mathbf{y}_k \approx \mathbf{s}_k$. We cannot divide by a vector, so instead we ask $B_{k+1}$ to satisfy this relation exactly for the step just taken, the **secant condition**:

$$B_{k+1}\, \mathbf{y}_k = \mathbf{s}_k,$$

which in one dimension is $B_{k+1} = s_k/y_k$ again.

In $d$ dimensions the secant condition does not determine $B_{k+1}$: it only fixes what the matrix does to one vector, $\mathbf{y}_k$, and many matrices do that. So BFGS does not build $B_{k+1}$ from scratch: it *updates* $B_k$, by the smallest correction that gives

1. **the secant condition**, $B_{k+1}\mathbf{y}_k = \mathbf{s}_k$, for the new step;

[1] Many textbooks write $B_k$ for an approximation of the Hessian $H$ itself, and $H_k$ for its inverse, so their formulas look the other way round. Here $B_k$ always approximates $H^{-1}$, the matrix the step actually uses.

2. **symmetry**, as for $H^{-1}$;
3. **positive definiteness**, so that the next step $-B_{k+1}\nabla f_{k+1}$ still points downhill.

Being a small correction, it keeps what $B_k$ learned from the earlier steps and adds what the new step teaches.

**BFGS** (Broyden–Fletcher–Goldfarb–Shanno, 1970: four people found it independently in the same year) gives this update in closed form:

$$B_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) B_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top, \qquad \rho_k = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k}.$$

We will not derive it ("smallest" is measured in a suitably weighted norm); the box below checks that it delivers the three properties, and cheaply.

[OPTIONAL] Checking the BFGS update

- **It satisfies the secant condition.** Apply it to $\mathbf{y}_k$. The right-hand bracket kills $\mathbf{y}_k$, $(I - \rho_k \mathbf{y}_k \mathbf{s}_k^\top)\mathbf{y}_k = \mathbf{y}_k - \mathbf{y}_k \frac{\mathbf{s}_k^\top \mathbf{y}_k}{\mathbf{y}_k^\top \mathbf{s}_k} = \mathbf{0}$, so the first term vanishes, and the second leaves $\rho_k \mathbf{s}_k (\mathbf{s}_k^\top \mathbf{y}_k) = \mathbf{s}_k$.
- **It stays symmetric.** The two brackets are transposes of each other, so the first term has the form $M B_k M^\top$ (with $M = I - \rho_k \mathbf{s}_k \mathbf{y}_k^\top$), and $\mathbf{s}_k \mathbf{s}_k^\top$ is symmetric.
- **It stays positive definite**, so every step $-B_k \nabla f_k$ points downhill, provided $\mathbf{y}_k^\top \mathbf{s}_k > 0$: along the step, the gradient grew in the direction we moved, i.e. $f$ curved upwards. For $\mathbf{v} \neq \mathbf{0}$, $\mathbf{v}^\top B_{k+1} \mathbf{v} = (M^\top \mathbf{v})^\top B_k (M^\top \mathbf{v}) + \rho_k (\mathbf{s}_k^\top \mathbf{v})^2$: both terms are non-negative, and they cannot both be zero (if $\mathbf{s}_k^\top \mathbf{v} = 0$ then $M^\top \mathbf{v} = \mathbf{v}$, and the first term is $\mathbf{v}^\top B_k \mathbf{v} > 0$). Implementations make sure the condition holds: the line search of `minimize()` picks $\gamma_k$ so that it does, and `optim()` checks it and restarts from $B = I$ when it fails.
- **It is cheap.** Multiplying out, $B_{k+1} - B_k$ is built from the two vectors $\mathbf{s}_k$ and $B_k \mathbf{y}_k$ alone: a *rank-two* correction, costing $O(d^2)$ operations, with no matrix ever inverted (Newton: $O(d^3)$).

The secant condition leaves room for other updates too: other choices of "closest", or other extra requirements, give other quasi-Newton methods with the same secant condition. **DFP** applies the same idea to an approximation of $H$ itself rather than of $H^{-1}$; **SR1** is the unique symmetric rank-*one* update, but it need not stay positive definite; the **Broyden family** blends BFGS and DFP. BFGS is the default in practice: it keeps $B_k$ positive definite, and it recovers best from a poor $B_k$.

To see it learn, run BFGS on the valley of Section 5.2.1, $H = \mathrm{diag}(1, 10)$, from $\mathbf{x}_0 = (-9, 1.5)$ with $B_0 = I$ and exact line searches. After the first update, $B_1 \approx \begin{pmatrix} 1.30 & 0.02 \\ 0.02 & 0.10 \end{pmatrix}$: after one step it has already found that $x_2$ is ten times steeper. After the second, $B_2 = \mathrm{diag}(1, 1/10) = H^{-1}$

exactly, and the iterate sits on the minimum. This is general: on a quadratic in $d$ dimensions with exact line searches, BFGS recovers $H^{-1}$ after $d$ steps. On other functions $H$ changes from point to point, and $B_k$ keeps tracking it: after a handful of steps BFGS is taking nearly Newton-quality steps while having only ever evaluated $f$ and $\nabla f$.

Algorithm 5.4 collects the whole loop in one place.

Algorithm 5.4: BFGS

**Input:** objective $f$, gradient $\nabla f$, start point $\mathbf{x}_0$, tolerance $\varepsilon > 0$.
**Output:** the final iterate, an approximate minimiser.

1. $B_0 \leftarrow I$
2. **for** $k = 0, 1, 2, \ldots$ until $\|\nabla f_k\| \leq \varepsilon$:
   a. $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \gamma_k B_k \nabla f_k$, with $\gamma_k$ from a line search
   b. $\mathbf{s}_k \leftarrow \mathbf{x}_{k+1} - \mathbf{x}_k$, $\mathbf{y}_k \leftarrow \nabla f_{k+1} - \nabla f_k$
   c. $B_{k+1} \leftarrow (I - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) B_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$, with $\rho_k = 1/(\mathbf{y}_k^\top \mathbf{s}_k)$, as defined above
3. **return** $\mathbf{x}_k$

$B_k$ has nothing to do with the mini-batches $B_t$ of Algorithm 5.2.

You will never implement BFGS yourself, because both languages ship it (R as `optim()`, Python inside `scipy.optimize.minimize()`):

**R**

```
optim(par, fn, gr, method = "BFGS")
```

**Python**

```
sp.optimize.minimize(fun, x0, jac=grad, method="BFGS")
```

with `par`/`x0` the starting point, `fn`/`fun` the function to minimise and `gr`/`jac` its gradient. Note that, unlike our `grad_descent()`, the built-in optimisers genuinely need the objective as well as the gradient: the line search inside BFGS evaluates `fn` to decide how far to step. The pieces we have been passing around all chapter slot straight in (this is why we wrote everything functional-style). Time to return to the logistic regression of Section 5.1.1 and retire our hand-tuned step sizes:

**R**

```r
fit_bfgs <- optim(par = c(0, 0), fn = nll, gr = nll_grad, method = "BFGS")
fit_bfgs$par          # estimated coefficients
```

```
[1] -0.5080097  1.8237002
```

```r
fit_bfgs$value        # NLL at the minimum
```

```
[1] 0.4618516
```

```r
fit_bfgs$counts       # evaluations of fn and gr
```

```
function gradient
      12       11
```

```r
fit_bfgs$convergence  # 0 means success
```

```
[1] 0
```

**Python**

```python
import scipy as sp

fit_bfgs = sp.optimize.minimize(nll, x0=np.zeros(2), jac=nll_grad, method="BFGS")
fit_bfgs.x            # estimated coefficients
```

```
array([-0.63803659,  1.89158111])
```

```python
fit_bfgs.fun          # NLL at the minimum
```

```
np.float64(0.45245131411477824)
```

```python
(fit_bfgs.nfev, fit_bfgs.njev)  # evaluations of fn and gr
```

```
(12, 12)
```

```
fit_bfgs.success      # True means success
```

```
True
```

**Note for Python:** the report fields differ in name and convention: `$par`, `$value`, `$counts` and `$convergence` become `.x`, `.fun`, `.nfev`/`.njev` and `.success`. And R signals success with `convergence == 0`, scipy with `success == True`. Always check it, whatever it is called.

About a dozen gradient evaluations in either run (each language on its own draw of the data). Let us line it up against our own algorithms, and against a reference fit: `glm()`, R's official logistic regression fitter, on the left; on the right, since Python has no built-in `glm()` (the usual tools live in the `statsmodels` package), BFGS rerun with a very strict tolerance, measuring cost in observation-gradients touched, as in Section 5.1.2:

**R**

```
# our GD: path_nll, already computed in @sec-week5-losses (200 steps, gamma = 2)

# our SGD, with a hand-tuned decay schedule
set.seed(3)
path_sgd_nll <- sgd(nll_grad_i, c(0, 0), gamma = \(t) 10 / (20 + t),
                    n_steps = 4000, n_data = n_log)

# the reference fit
fit_glm <- glm(y_log ~ X_log - 1, family = binomial)

results <- data.frame(
  method = c("glm()", "optim BFGS", "GD (200 steps)", "SGD (4000 steps)"),
  nll = c(nll(coef(fit_glm)), fit_bfgs$value,
          nll(beta_hat),
          nll(unlist(tail(path_sgd_nll, 1)[1:2]))),
  obs_gradients = c(NA, fit_bfgs$counts["gradient"] * n_log, 200 * n_log, 4000)
)
results$nll <- round(results$nll, 6)
results$obs_gradients <- format(results$obs_gradients, big.mark = ",", scientific = FALSE)
results
```

```
            method      nll obs_gradients
1            glm() 0.461852            NA
2       optim BFGS 0.461852         5,500
3   GD (200 steps) 0.461852       100,000
4 SGD (4000 steps) 0.461863         4,000
```

**Python**

```
# our GD: path_nll, already computed in @sec-week5-losses (200 steps, gamma = 2)

# our SGD, with a hand-tuned decay schedule
rng = np.random.default_rng(3)
path_sgd_nll = sgd(nll_grad_i, np.zeros(2), gamma=lambda t: 10 / (20 + t),
                   n_steps=4000, n_data=n_log)

# the reference fit (no glm() here: BFGS again, with a very strict tolerance)
fit_ref = sp.optimize.minimize(nll, np.zeros(2), jac=nll_grad, method="BFGS",
                               options={"gtol": 1e-10})

results = pd.DataFrame({
    "method": ["reference (tight BFGS)", "minimize BFGS", "GD (200 steps)", "SGD (4000 steps)"],
    "nll": [fit_ref.fun, fit_bfgs.fun,
            nll(beta_hat),
            nll(path_sgd_nll.iloc[-1, :2].to_numpy())],
    "obs_gradients": [np.nan, fit_bfgs.njev * n_log, 200 * n_log, 4000]
})
results["nll"] = results["nll"].round(6)
results["obs_gradients"] = ["NA" if np.isnan(v) else f"{v:,.0f}"
                            for v in results["obs_gradients"]]
results
```

| | method | nll | obs_gradients |
|---|---|---|---|
| 0 | reference (tight BFGS) | 0.452451 | NA |
| 1 | minimize BFGS | 0.452451 | 6,000 |
| 2 | GD (200 steps) | 0.452451 | 100,000 |
| 3 | SGD (4000 steps) | 0.452686 | 4,000 |

Read the table slowly: it summarises the whole chapter. (The two columns fit two different draws of the data, so the loss values differ between languages: the *pattern* is the same.)

- **BFGS matches the reference fit (`glm()` in R) to six decimals** at the cost of a few thousand observation-gradients, with *no tuning at all*: no $\gamma$, no schedule, no $\beta$. The curvature information it accumulates replaces all the knobs we spent this chapter turning by hand.
- **GD gets there too**, but only after we hand-tuned $\gamma$, and at over fifteen times the cost. On an ill-conditioned problem (Exercise 5.3) the gap grows by orders of magnitude.
- **SGD is nearly as accurate here at a fraction of the cost**, and its cost would be *unchanged* if $n$ were $10^8$, which is the regime it is built for.

Two closing remarks on `optim()` and `minimize()`. For problems with many parameters there is `method = "L-BFGS-B"` (same name in both), a low-memory variant that stores only the last few $(\mathbf{s}_k, \mathbf{y}_k)$ pairs instead of the full $d \times d$ matrix (and accepts box constraints on the parameters). And practically: **`optim()` with BFGS is what you reach for** when fitting a model by maximum likelihood in R (`minimize()` in Python): countless packages in both languages are, under the hood, exactly `optim(par, nll, gr, method = "BFGS")` or its `minimize()` twin wearing a nice interface.

Exercise 5.3: optim / minimize in the wild

Fit a logistic regression predicting a car's transmission type from its horsepower and weight, using the built-in `mtcars` data (`am` is 1 for manual; in Python the same data frame ships with plotnine):

**R**

```r
X_cars <- cbind(1, mtcars$hp, mtcars$wt)
y_cars <- mtcars$am
n_cars <- nrow(mtcars)

nll_cars <- \(b) {
  eta <- as.vector(X_cars %*% b)
  mean(log(1 + exp(eta)) - y_cars * eta)
}
nll_cars_grad <- \(b) {
  p <- 1 / (1 + exp(-as.vector(X_cars %*% b)))
  as.vector(t(X_cars) %*% (p - y_cars)) / n_cars
}
```

**Python**

```python
from plotnine.data import mtcars

X_cars = np.column_stack([np.ones(len(mtcars)), mtcars["hp"], mtcars["wt"]])
y_cars = mtcars["am"].to_numpy()
n_cars = len(mtcars)

def nll_cars(b):
    eta = X_cars @ b
    return np.mean(np.log(1 + np.exp(eta)) - y_cars * eta)

def nll_cars_grad(b):
    p = 1 / (1 + np.exp(-(X_cars @ b)))
    return X_cars.T @ (p - y_cars) / n_cars
```

a. Fit with `optim(c(0, 0, 0), nll_cars, nll_cars_grad, method = "BFGS")` and compare the NLL at the optimum with `glm(am ~ hp + wt, data = mtcars, family = binomial)` (compute `nll_cars(coef(...))`). Check `$convergence`: if it is not `0`, look up what that means in `?optim` and fix it via the `control` argument.
b. Now try your own `grad_descent()` with `gamma = 1e-3`, then with `gamma = 1e-4` and `n = 20000`. Describe what happens in each case.
c. Explain (b) in one or two sentences using Section 5.2.1.1. Hint: look at the scales of the columns of `X_cars`. Which section of this chapter offers a fix that would let gradient descent compete?

**Note for Python:** the fit in (a) is `sp.optimize.minimize(nll_cars, np.zeros(3), jac=nll_cars_grad, method="BFGS")`; the fields to check are `.success` and `.message`, and options such as iteration caps go in `options = {"maxiter": 500}`. With no `glm()` in Python, compare against a strict-tolerance refit (`options = {"gtol": 1e-10}`), as the reference row of the table above did.

## 5.3 Where you will see all this again

This chapter is a gateway. The same handful of ideas reappears, barely disguised, all over statistics and machine learning:

- **Maximum likelihood, everywhere.** Almost every model you will fit from now on is fitted by numerically minimising a negative log-likelihood. Sometimes the optimiser is visible (`optim()` or `minimize()` in a package's source code), sometimes hidden (R's `glm()` uses iteratively reweighted least squares, a Newton variant), but it is always there, and now you know what its knobs mean and what its failure modes look like. When a model "fails to converge", you have seen the picture: a too-large step diverging, or an ill-conditioned likelihood crawling.
- **Neural networks.** Training a network is minimising an average loss over millions of observations and millions of parameters: mini-batch SGD is the core engine. However, as the geometry of the loss landscape is often ill-conditioned in those ML models, we tend to find that a separate class of gradient descent algorithms, called **momentum** based SGD, which include Adam. In short, these accelerate convergence by using a moving average of past gradients to smooth out the zig-zagging and help navigate the loss landscape more effectively.
- **Sampling algorithms.** When you meet Markov chain Monte Carlo in weeks 8–10 of this module, you will find the same geometry problem in a new costume: a sampler exploring an elongated, ill-conditioned posterior zig-zags just like gradient descent did.
- **Automatic differentiation.** In Section 5.1.1.1 we traded the calculus for $2d$ loss evaluations per step. Modern software offers a third way: *automatic differentiation* reads the code that computes the loss and, applying the chain rule to each elementary operation

in it, returns the *exact* gradient, at a small multiple of the cost of evaluating the loss, however many parameters there are. It is what makes training neural networks with millions of parameters possible (there it goes by the name of *backpropagation*). You will meet it again in weeks 8–10 of this module, and in future modules.

# References

The books below go deeper than these notes do: Wickham (2019) on the R language itself, Wickham et al. (2023) and McKinney (2022) on data analysis in each of our two languages, and Nocedal and Wright (2006) for the optimisation theory behind week 5.

McKinney, Wes. 2022. *Python for Data Analysis*. 3rd ed. O'Reilly Media. https://wesmckinney.com/book/.

Nocedal, Jorge, and Stephen J. Wright. 2006. *Numerical Optimization*. 2nd ed. Springer.

Wickham, Hadley. 2019. *Advanced R*. 2nd ed. Chapman; Hall/CRC. https://adv-r.hadley.nz/.

Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. 2023. *R for Data Science*. 2nd ed. O'Reilly Media. https://r4ds.hadley.nz/.